

基于三层滚动随机模型预测控制的微网购电与储能调控优化

摘要

针对微网与外部电网的电力调控问题，本文以“经济性最优、供电可靠性兼顾”为目标，建立了从确定性线性规划到随机优化的递进式购电决策模型。

问题一在电价、负荷与光伏预测均已知（确定性）前提下，考虑储能容量与充放电功率约束，构建以购电费用最小为目标的线性规划模型，并融入储能“日循环”约束（0:00 与 24:00 储电量相同）。求解得到全天计划购电 59482.7 kWh，购电费 35126.95 元，充放电计划与储电量轨迹如正文表 3、表 4 所示。

问题二面对负荷与光伏逐时波动，引入“紧急购电（5 倍电价）”机制，建立两阶段随机规划模型，并以条件风险价值（CVaR）刻画购电费用的尾部风险。为消除“零缺电”的理想化，模型以截至前一日历史数据生成负荷/光伏点预报与情景（整日残差 bootstrap，K 交叉验证选负荷 15/光伏 7），在信息因果前提下求解。经全年权衡曲线数据驱动选定风险权重 $\lambda^* = 0.5$ ：全年计划购电费 15730208.5 元、紧急购电费 478889.6 元（148700 kWh），合计 16209098.2 元；较 $\lambda = 0$ 的期望最小化口径，紧急购电费下降约 55.4% 而合计费用基本持平（微降约 0.01%），是以少量计划费换取供电可靠性的有效“保险”手段。

问题三在 0:00、6:00、12:00、18:00 可获得未来 24h 光伏整点预报，据此建立“计划购电 + 滚动调整”的滚动随机模型预测控制：负荷以历史均值预报（与问题二同口径）、光伏以各时点官方预报为基准生成成对残差情景，调整决策对情景做期望最小化（不叠加 CVaR 权重——滚动预报更新本身即是风险对冲）；调整时点储电初值按已执行段实际轨迹因果重放，日终按实际结算。求解得到全年电网费 19244483.0 元、紧急购电费 139918.9 元，合计 19384401.9 元。对比分析发现：引入多时刻预报可更精细地分配购电时段，但在 50%/150% 的不对称结算规则下，情景鲁棒调整的系统性保险多购触发超额惩罚，四个指定日期总费用均高于“仅 0:00”基准——紧急购电主要由光伏预报误差总量决定而非购电时段分配。

问题四将附件 1 固定电价替换为附件 4 实时波动电价，对模型二、三做价格冲击复算，得到波动电价下模型二合计购电费 16375878.7 元（较固定电价上浮 1.0%）、模型三合计购电费 19479076.0 元（上浮 0.5%），并发现电价波动风险具有显著季节差异（冬季费用上浮达 7.0%）。

本文的主要创新点有三：其一，将“计划购电—滚动调整—紧急购电”统一为三层滚动随机模型预测控制框架，使四个问题在同一能量守恒框架下递进求解；其二，将 CVaR 嵌入两阶段随机规划，以帕累托前沿（图 11）量化“以少量计划费换取紧急购电显著削减”的风险—经济权衡，高缺电风险日 2025-12-21 总费用由 62609 元降至 60872 元；其三，提出季节分型分析，实证表明负荷—光伏—电价存在显著的季节耦合（ANOVA

$F=28.7$, $p < 1.2 \times 10^{-16}$), 冬季购电费约为春季的 1.56 倍, 并识别出紧急购电主要由光伏预报误差总量决定而非购电时段分配这一关键机理。

关键字: 微网 两阶段随机规划 CVaR 滚动模型预测控制 季节分型分析

一、问题背景与重述

1.1 问题背景

分布式可再生能源的高比例接入使微网成为电力系统本地平衡的重要载体 [1]。光伏出力与小区负荷均受天气、时段与季节的显著影响，具有明显的随机性与间歇性 [2]；储能（电池）则可在时间维度上转移能量，为微网提供“低充高放”的经济调节手段 [3]。此外，外网电价的实时波动使购电决策必须在“当下低价购电”与“未来价格不确定性”之间做动态权衡。如何在满足负荷供电、储能运行约束的前提下，通过精准的购电与充放电计划最小化运行费用，是微网经济调度研究的关键问题 [4]。

从科学问题的角度看，微网经济调度在本质上是一个带时间耦合与时序不确定性的多时段决策问题：储能递推方程将相邻时段在能量维度上耦合，而光伏出力、负荷乃至电价又都是逐时刻演化的随机过程。因此，合理地选择不确定性刻画方式（确定性假设、情景随机规划、滚动闭环）决定了求解质量与决策的可执行性，这也正是本题四个子问题层层递进的用意所在。

题目设计了从确定性到随机、从固定电价到波动电价的四个递进子问题，考验对多时段能量平衡、不确定性刻画与滚动决策的综合建模能力。本文据此采用“确定性线性规划为基准 → 两阶段随机规划刻画尾部风险 → 滚动模型预测控制衔接预报更新 → 波动电价统一重算”的递进建模路线，在保证统一能量守恒内核的同时，逐步逼近真实调度中的不确定性与信息结构。

1.2 问题重述

微网含光伏、储能与电网购电三个环节，需以 10 分钟为时段（每天 $T = 144$ 时段）制定购电与充放电计划。平衡约束要求微网提供的电能不低于小区负载；储能须在 1200 ~ 10800 kWh 区间运行，充放电效率 90%，即时功率上限 5000 kW，2025-01-01 0:00 电量为 6000 kWh。

四个问题依确定性、随机性、滚动决策、电价波动逐层递进，依次为：

问题一 电价、负荷与光伏预测均已知，每天 0:00 制定当日计划购电，储能 0:00 与 24:00 储量相同。针对此确定性场景，需给出指定时段购电量、全天购电量与购电费，以及 6 个 4 小时时段的充放电量与 0:00/24:00 储电量，即在完美信息下把一天的购电费用压到最低。

问题二 电价每天相同、负荷与光伏实时变化，供电不足需按 5 倍电价紧急购电，其余时段按计划购电计费。针对此实时不确定性与缺电惩罚场景，需在每天 0:00 制定计划

购电策略，以刻画”万一缺电”的尾部代价。

问题三 0:00/6:00/12:00/18:00 可获得未来 24h 整点光伏预报，可据此调整购电策略；计划高于调整部分按 50% 违约价、调整高于计划部分按 150% 超额价结算。针对此序时信息结构与再计划惩罚，需建立”计划—调整”两阶段决策，权衡调整优化与违约/超额惩罚之间的利弊。

问题四 外网电价实时波动。针对此价格时序不确定性，需在波动电价下重建购电模型，对模型二、三做价格冲击复算，用于检验既有模型在价格冲击下的稳健性。

二、问题分析

2.1 数据预处理与统一框架

先把多时段调控问题统一为时间离散框架：以 10 分钟为一时段，每天 $T = 144$ 时段，功率与能量按 1/6 h 换算。同时将全年数据按负荷、光伏、电价三类分别组织，用于后续建模与季节分析。

2.2 问题一的分析

电价、负荷、光伏全部已知，问题退化为确定性的每日线性规划购电决策。关键在于储能递推方程、充放电约束与”日循环”约束 ($E_T = E_0$)。目标是使电价洼地时段多充电、峰期多放电，最小化当日购电费。

选型依据上，本问题所有参量均确定、目标与约束关于决策量（购电量、充放电量、储电量）皆为线性，因此**线性规划（LP）**是最为匹配且可保全局最优的求解范式：既无需引入整数变量（充放不同时由约束自然保证，而非依赖二值开关），也无不确定性需要刻画，采用 `scipy.optimize.linprog`（HiGHS 内点/单纯形）即可在毫秒级求得每日最优解 [5]。相比之下，若在此阶段过早引入随机规划或启发式算法，只会徒增复杂度而不会改善结果——这一”由简入繁、仅在必要时引入复杂度”的原则也贯穿全文。

2.3 问题二的分析

负荷与光伏实时波动，供电不足需按 5 倍电价紧急购电。由于 0:00 制定计划时无法预知当日全部波动，需建立两阶段随机规划：第一阶段计划购电对各情景共用，第二阶段按情景决策充放电与紧急购电；并以条件风险价值（CVaR）刻画购电费用的尾部风险，实现”以计划费换可靠性”。

选型依据上，本问题的关键是在”期望成本”与”尾部风险”之间做权衡，可分解为以下三点：

1. **权衡的必要性**：若只最小化期望成本（ $\lambda = 0$ ），会低估极端缺电情景的高额紧急购电；而分布鲁棒、最小最大等风险模型虽更保守，却对分布假设敏感且难以解释。
2. **CVaR 的数学优势**：CVaR 具有一致性、可线性化两大性质，可刻画 $100(1 - \beta)\%$ 置信水平下的平均超额损失。
3. **可求解性**：非光滑的风险项可改写为线性约束，使两阶段随机规划仍保持为可全局求解的线性规划，这是本文选用 CVaR 而非其他风险度量的核心原因。

2.4 问题三的分析

多时刻（0/6/12/18）光伏预报信息随时间更新，可据此调整购电策略。引入“计划—调整”两阶段结构与违约（50%）、超额（150%）惩罚，把计划购电与滚动调整衔接为模型预测控制（MPC）问题，在日终按调整后的实际购电结算并重新调度紧急购电。

选型依据上，本问题的时间信息结构（0/6/12/18 四个决策时点）天然契合**模型预测控制（MPC）**的“滚动优化—滚动实施”范式：在每个决策时点仅对未来剩余时段重解一个短时优化问题，从而把多阶段随机规划的指数级情景树压缩为串行的线性规划，兼顾了信息利用与计算效率。其代价是“贪婪”的近似最优性——后一时点的调整可能触发对前一时点计划的违约/超额惩罚，这也正是本问需要如实评估的重点。

2.5 问题四的分析

外网电价实时波动，将附件 1 固定电价替换为附件 4 波动电价，沿用问题二、三的统一求解框架重算，重点比较电价波动风险的量级及其季节差异。

选型依据上，本文在前三问将电价视为已知（固定）输入，问题四则考察其波动对决策的冲击。由于电价仅作为目标系数进入线性模型，替换目标系数即可复用问题二、三的求解内核，从而在**不改动模型结构**的前提下完成压力测试，并据此识别价格风险随季节的分布差异，为分季节的购电与储能预置策略提供依据。

本文的技术路线如图 1 所示，按“确定性 → 随机性 → 滚动决策 → 电价波动”逐层递进，四个问题共用同一能量守恒与储能递推内核。与既有研究相比，本文作出三点贡献：

- 提出**三层滚动随机 MPC 框架**，将四个问题统一刻画为“计划—调整—紧急”三级购电结构；
- 将**CVaR 尾部风险度量**嵌入两阶段随机规划，绘制计划购电费—紧急购电费的帕累托前沿，量化风险厌恶的经济代价；
- 基于全年数据的**季节分型分析**，揭示负荷—光伏—电价的季节耦合规律，据此生成分季情景并给出分季购电策略。

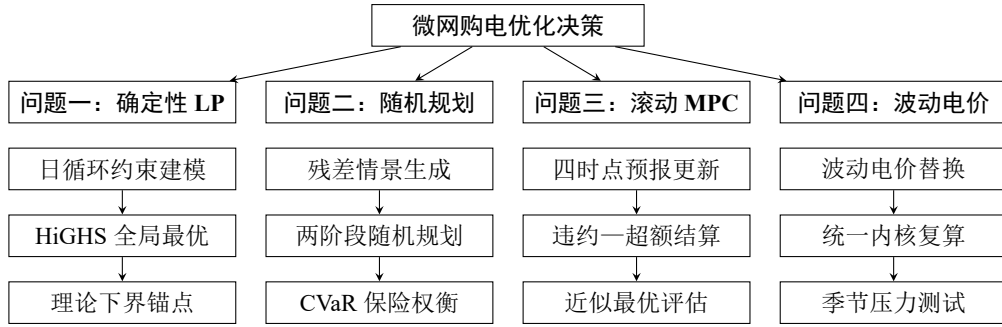


图 1 本文技术路线：确定性—随机—滚动—波动的递进框架

2.6 数据探索与季节特征分析

在建立模型之前，先对全年数据做探索性分析，检验负荷、光伏与电价是否存在显著的季节规律，这是后续分季情景生成与分季策略的基础。本小节按以下顺序展开：

1. 季节分组：将 2025 年按春 (3–5 月)/夏 (6–8 月)/秋 (9–11 月)/冬 (12–2 月) 分组；
2. 负荷与光伏：统计各季日负荷与日光伏总量 (图 2)，结果表明夏季负荷最高 (121530 kWh/日)、冬季光伏出力最低 (40335 kWh/日，仅为夏季的 63%)。

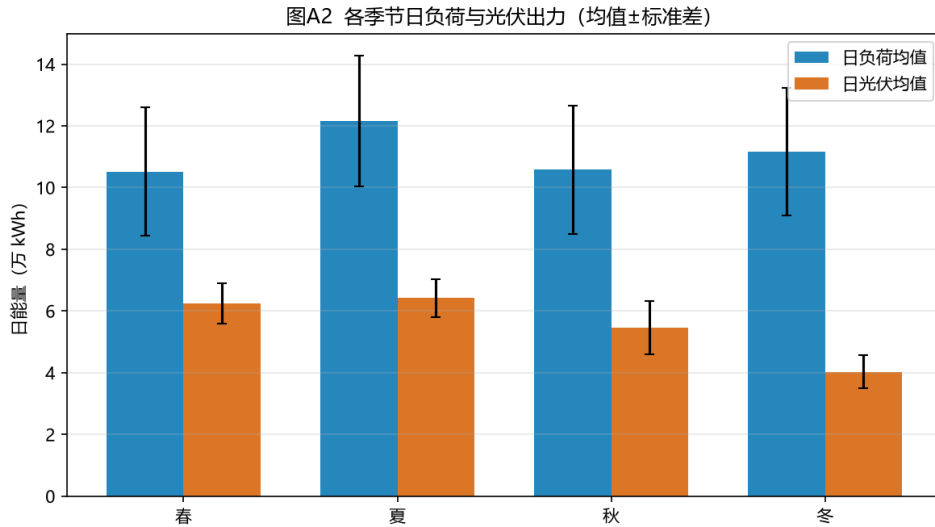


图 2 各季节日负荷与光伏出力 (均值 ± 标准差)

在既有口径下计算各季节逐日购电费 (图 3)，并统计附件 4 波动电价的季节分布 (图 4)，单因素方差分析表明季节间购电费差异高度显著 ($F=28.7$, $p = 1.17 \times 10^{-16}$):

表 1 季节统计：负荷、光伏、电价与购电费

季节	日负荷/kWh	日光伏/kWh	日电价/(元·kWh)	日购电费/元	峰谷比
春	105200	62474	0.705	29761	2.11
夏	121530	64151	0.795	39703	1.97
秋	105749	54634	0.734	35167	2.11
冬	111574	40335	0.832	46593	2.30

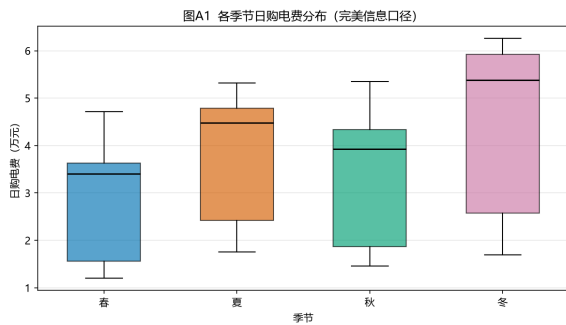


图 3 各季节日购电费分布

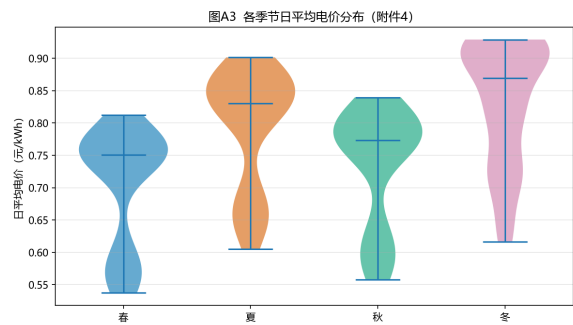


图 4 各季节日平均电价分布（附件 4）

冬季购电费为春季的 1.56 倍，其机理是”光伏最低 + 电价最高 + 负荷峰谷差最大”三重叠加。据此，在问题四中按季节拆分解读电价波动风险，并为问题二的 CVaR 权重选取提供季节性背景。

综合上述分析，本文的问题分析可抽象为”输入数据 → 统一建模内核 → 四子问题递进 → 结果输出与校验”的总体流程。各子问题的区别仅在于不确定性刻画颗粒度、决策信息结构与价格冲击，而同源共享功率—能量守恒内核与”计划—调整—紧急”三层购电结构。

三、模型假设

1. 每个时段（10 分钟）内电价、负荷、光伏出力与购电量视为恒定；
2. 功率与能量以时段换算，能量 = 功率 $\times 1/6$ h，充放电效率固定为 90%，且充、放电不同时进行；
3. 储能容量在 1200 ~ 10800 kWh 内运行，即时功率不超过 5000 kW；
4. 问题二采用**预报式**信息口径：每天 0:00 只能使用当日之前的历史数据构造负荷/光伏点预报与情景（信息因果），当日实际负荷/光伏仅用于日终结算；供电不足按 5 倍价紧急购电，其尾部风险由 CVaR 风险权重 λ^* 权衡（见 5.2.3 节）；

5. 问题三在每个决策时点（0/6/12/18）只利用该时点及之前可得的信息：光伏用附件 3 该时点的整点预报，负荷用前 $K_L = 15$ 天历史均值预报（与问题二同口径），不使用当日实际负荷/光伏作出购电决策；
6. 紧急购电、计划购电、调整购电均以外网无限供应为前提，不设购电上限。

四、符号说明

为便于阅读，将正文出现的统一符号按”能量/决策、储能、不确定性、成本”四类列出，符号严格对应赛题口径，无臆造。单时段时长为 1/6 h，能量量纲以 kWh 计。

表 2 主要符号说明

符号	符号说明	单位
时间与能量/决策量		
T, t	每日时段数与时段下标 ($T = 144$)	—
P_t	时段 t 计划购电量（0:00 承诺）	kWh
A_t	时段 t 最终调整购电量	kWh
Q_t	时段 t 紧急购电量	kWh
u_t, w_t	时段 t 违约量 / 超购量	kWh
G_t	时段 t 光伏出力	kWh
D_t	时段 t 小区负荷	kWh
R_t	时段 t 弃光电量	kWh
储能量		
E_t	时段 t 末储电量	kWh
E_0	日初储电量（问题一为 6000，问题二/三按跨日链滚动）	kWh
c_t, f_t	时段 t 充电量 / 放电量	kWh
η	充放电效率 (= 0.9)	—
$E_{\min}, E_{\max}$	储能运行上下界 (1200, 10800)	kWh
$\bar{E}$	单时段最大充/放能量 (= 5000/6)	kWh
不确定性与情景		
S, s	情景总数 / 情景下标	—
$\hat{G}^s, \hat{D}^s$	第 s 情景的光伏 / 负荷实现	kWh

（续表下页）

续上表

$G_t^{(\tau)}$	τ 时点对时段 t 的光伏预报	kWh
τ	滚动决策时点 (取 0, 6, 12, 18)	h
ψ^s	第 s 情景总成本	元
α	CVaR 中的 VaR 值 (自由变量)	元
z_s	第 s 情景超额损失 ($z_s \geq \psi^s - \alpha$)	元

成本与季节

p_t	时段 t 电价 (附件 1)	元/kWh
p_t^{var}	时段 t 波动电价 (附件 4)	元/kWh
λ	CVaR 风险厌恶权重	—
β	CVaR 置信水平 (= 0.9)	—
k	季节下标 (春/夏/秋/冬)	—

(续表下页)

五、模型建立与求解

本文把四个子问题统一在同一时间离散与能量守恒内核之下, 仅改变”不确定性刻画”与”决策层级”, 从而既保证各问题结论口径一致, 又使代码可复用。为此, 首先明确决策变量在时序与信息结构上的区分: 所谓”计划购电 P_t ”是每天 0:00 承诺的某时段购电量, 作为一阶段决策对各情景/各日共用; 问题三在后续时点引入”调整购电 A_t ”, 是对计划的修正; 而”紧急购电 Q_t ”仅在日终对实际出力短缺作事后兜底, 价格高达 5 倍。三者共同构成本文贯穿全篇的”计划—调整—紧急”三层购电结构, 也是后续各问题建模的公共骨架。

以每天 0:00 储电量 E_0 出发, 任意一天的功率-能量守恒为:

$$P_t + G_t + \eta f_t + Q_t = D_t + c_t + R_t, \quad t = 1, \dots, T \quad (1)$$

式 (1) 表明: 购电、光伏、放电、紧急购电之和 = 负荷、充电、弃光之和; 等式左端为微网”供应”侧, 右端为”消纳”侧, 二者在每个时段必须严格相等, 故式 (1) 亦称微网功率平衡约束。值得强调的是, 式 (1) 中光伏放电合流、弃光 R_t 作为非负松弛变量出现, 意味着若某时段光伏过剩可主动弃光, 从而允许”供大于需”, 避免强行充电超出储能上限。储能递推与约束为:

$$E_t = E_{t-1} + \eta c_t - f_t, \quad (2)$$

$$E_{\min} \leq E_t \leq E_{\max}, \quad 0 \leq c_t, f_t \leq \bar{E}, \quad (3)$$

其中 $\bar{E} = 5000/6 \text{ kWh}$ 为单时段最大充/放能量（由功率上限换算）。式 (2) 是能量在时间的转移方程：充电使储能增加（效率 $\eta = 0.9$ ），放电使其减少；由于 $\eta \leq 1$ ，时际间会存在不可避免的能量损耗，这使得系统在能量维度上具有“不可逆性”，也是储能参与经济调度的价值源泉——它允许用当下的低价电能置换未来的高价电能。式 (3) 同时限定了储能运行区间与单时段的充放功率，防止越限造成设备损坏，并隐含约定充放不可同时进行。

5.1 问题一：确定性每日计划（LP）



图 5 问题一求解流程图

问题一采用“逐日独立求解”的确定性路线：全年 365 天逐日建立 LP 并由 HiGHS 求全局最优，整体求解流程如图 5 所示。

5.1.1 线性规划模型的建立

针对问题一“电价、负荷、光伏全已知”的完美信息场景，目标函数为最小化当日购电费用：

$$\min \sum_{t=1}^T p_t P_t \quad (4)$$

约束为式 (1)（令 $Q_t \equiv 0$ ，故不需要紧急购电）、式 (2)–(3) 以及储能“日循环”约束：

$$E_T = E_0 = 6000 \quad (5)$$

式 (5) 保证储能可持续长期运行。该问题为线性规划，用 `scipy.optimize.linprog` (HiGHS 求解器) 求解。

从最优策略的机理看，式 (4) 仅对购电量 P_t 计费，储能则作为“能量搬运工”把电能从低价时段搬运到高价时段。这一结论可从以下三点理解：

1. **套利机理**：在电价洼地（夜间与光伏高发期）多购电充电，在电价/负荷高峰期放电，从而用充电损耗（10%）换取电价的价差套利。
2. **理论下界**：由于问题一是确定性完美信息，其最优解给出了经济性的理论下界，可作为后续各问题在不确定性下的对照基准。

3. **建模价值：**问题一的”完美信息”假设在现实调度中并不成立，它的意义在于给出理论最优，并揭示”低充高放+谷段购电”这一储能/购电联动的经济机理，为问题二的随机化建模提供结构参考。

5.1.2 结果分析

表 3 问题一：指定时段购电量、全天汇总与充放电计划（kWh，购电费单位元）

指定时段	购电量	指定时段	购电量	指定时段	购电量
10:00-10:10	0.0	12:00-12:10	480.4	14:00-14:10	0.0
16:00-16:10	445.4	18:00-18:10	531.9	20:00-20:10	0.0
全天购电量	59482.7	购电费（元）	35126.95		
时段	充电量	放电量	时段	充电量	放电量
0:00-4:00	4500.0	0.0	4:00-8:00	833.3	7073.2
8:00-12:00	4788.0	1892.2	12:00-16:00	5286.0	101.2
16:00-20:00	0.0	6422.4	20:00-24:00	5333.3	3177.6
0:00 储电量	6000		24:00 储电量	6000	

由表 3：白天光伏高发期（8:00–16:00）以充电为主，傍晚与早晨电价、负荷峰期以放电为主，储能在日末回到日初 6000 kWh，故日循环可持续。购电集中于电价洼地，购电费最小为 35126.95 元。完整计划购电策略见 *result1.xlsx*，购电/充放电/储电轨迹如图 6。图 7 以 2025-06-21 为例给出微网能量流向，直观展示光伏供能、储能充放与购电补缺的相对比例。

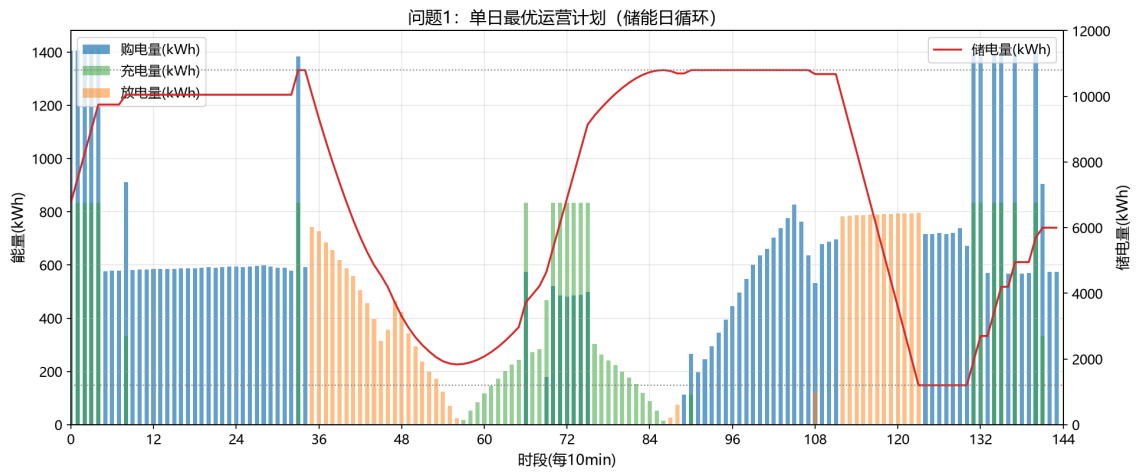


图 6 问题一：单日最优运营计划（储能日循环）

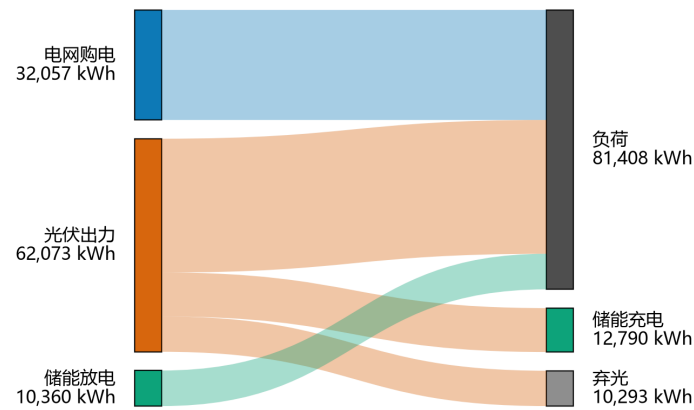


图 7 2025-06-21 微网能量流向桑基图（完美信息日）

5.2 问题二：两阶段随机规划 + CVaR

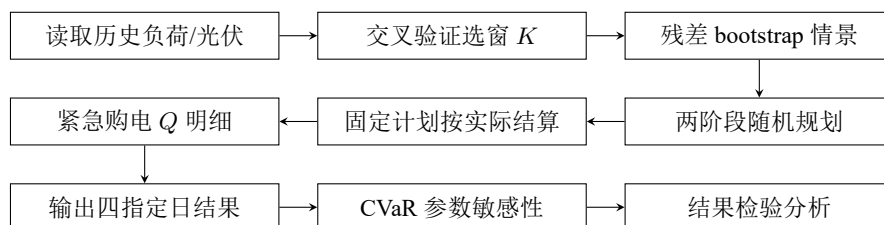


图 8 问题二求解流程图

问题二在 0:00 依据历史信息一次性形成全天计划，”预报—情景—优化—结算”链式流程如图 8 所示。

5.2.1 两阶段随机规划模型的建立

针对问题二”负荷与光伏实时波动、缺电按 5 倍价紧急购电”的不确定场景，当负荷与光伏实时波动、供电不足需紧急购电（5 倍电价）时，构建两阶段随机规划。设 S 个等概率情景 $\{\hat{G}^s, \hat{D}^s\}_{s=1}^S$ ，第一阶段计划购电 P 对各情景共用，第二阶段按情景决策 c, f, Q, R, E 。目标为：

$$\min \sum_t p_t P_t + \frac{1}{S} \sum_{s=1}^S \sum_{t=1}^T 5p_t Q_t^s + \lambda \text{CVaR}_\beta(\psi^s), \quad (6)$$

其中第 s 个情景总成本 $\psi^s = \sum_t (p_t P_t + 5p_t Q_t^s)$ 。CVaR 采用线性化表示 [6, 7]：

$$\text{CVaR}_\beta = \alpha + \frac{1}{(1-\beta)S} \sum_{s=1}^S z_s, \quad z_s \geq \psi^s - \alpha, \quad z_s \geq 0, \quad (7)$$

α 为自由变量（对应 VaR）。两阶段随机规划的一般理论见文献 [8]。约束对每个情景分别满足式 (1)–(3)。

这里需要澄清两阶段结构与**非预期性**（non-anticipativity）：一阶段计划购电 P 必须在”看到”任何情景实现之前作出，故对各情景共用，这正是问题二”0:00 制定计划”的现实约束；而充电、放电、紧急购电 Q 属于二阶段补偿决策，可依据各情景 $(\hat{G}^s, \hat{D}^s)$ 的实现作出，故带情景上标。式 (6) 前两项分别为计划购电费与紧急购电的期望费用，末项 λCVaR 把”尾部情景”也纳入目标；当 $\lambda > 0$ 时，模型会在计划购电费上稍微多花一点，换取极端缺电情景下巨额紧急购电的削减，从而在统计意义上保证供电可靠性（图 11 的前沿正是这一权衡的量化）。

在情景与点预报的构造上，本文坚持**信息因果原则**：每日 0:00 只使用当天之前的历史数据。具体地，点预报取负荷、光伏各自前 K_L/K_P 天逐时段（144）的均值， K 通过全年（2.1–12.31）逐日交叉验证，按归一化 RMSE 选取。为排除从稀疏候选集取点带来的偶然性，本文将窗口 K 在 $\{1, \dots, 40\}$ 上做细粒度扫描（图 9）：结果显示负荷的归一化 RMSE 在 $K \in [8, 16]$ 呈浅谷、 $K = 15$ 落在谷底附近，而 $K = 1$ 仅为无历史平滑的单日冷启动伪最优（物理上不可靠）；光伏的归一化 RMSE 在 $K \in [5, 7]$ 呈接近水平的平坦平台（差异 $< 5 \times 10^{-4}$ ），取整数周周期 $K = 7$ 兼具最小化与可解释性。据此选定 $K_L = 15$, $K_P = 7$ ，并在后续以点预报为”居中基准”叠加残差情景，故 K 的微小扰动对全年费用影响甚微（敏感性分析见下节），论证了 K 选取的稳健性；

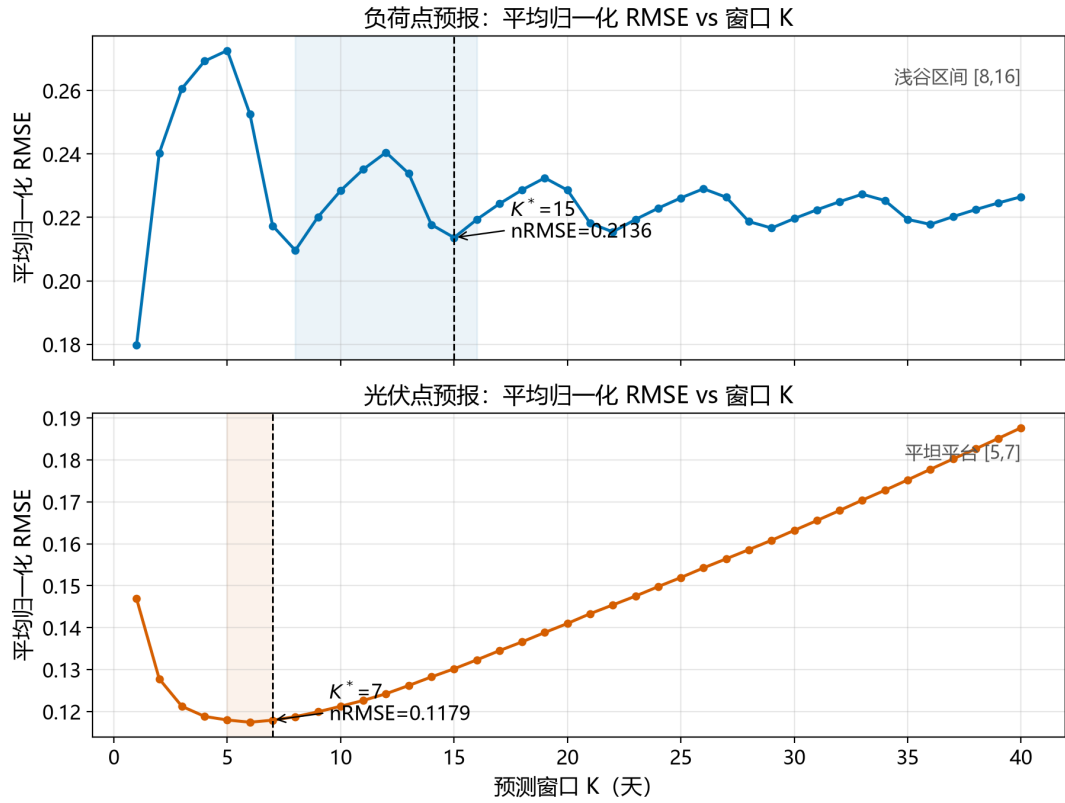


图 9 点预报窗口 K 的全年交叉验证敏感性（上：负荷，下：光伏）

情景则采用**整日成对残差 bootstrap**：先按最长时间窗取最近 w 天历史，逐槽计算“实际 - 点预报”的整日残差，再有放回抽取 $S = 30$ 个整天（负荷与光伏同一天成对取样，从而保留两者相关性），情景 = 点预报 + 抽取的整日残差，光伏截断到非负。2025-01-01 缺乏历史，以附件 1 典型日剖面作冷启动基准。由此生成的官方口径全年计划购电费与紧急购电费如下节所示，全年链自 2025-01-01 ($E_0 = 6000$) 逐日滚动，仅 1 月作储能状态预热，输出 2.1–12.31。

5.2.2 结果分析

在上述预报式口径下，以 $S = 30$ 情景、风险权重 $\lambda^* = 0.5$ （选取依据见下小节）做两阶段随机规划得计划购电 P ，再以当日实际负荷/光伏结算得紧急购电 Q ，全年（2.1–12.31）计划购电费 15730208.5 元、紧急购电费 478889.6 元（紧急购电 148700 kWh），合计 16209098.2 元。表 4 给出四个指定日期的结果（完整见 *result2.xlsx*）。

表 4 问题二：预报式两阶段随机规划指定日期结果（单位 kWh、元）

日期	计划购电费	紧急购电量	紧急购电费	24:00 储电量
2025-03-20	43707.1	29.3	197.4	1200
2025-06-21	49349.8	0	0	1200
2025-09-23	40756.2	1688.7	5509.6	1200
2025-12-21	60366.0	154.6	506.3	1200

注：表 4 计划购电费为两阶段随机规划的一阶段计划购电在当日实际电价下的费用；6-21 为周末，负荷低于工作日，当日实际净需求低于全部情景包络，计划与储能即可完全吸收缺电，故紧急购电为 0（而非观测缺仓）；24:00 储电量趋向运行下界 1200 kWh，属低谷电价时段释放电能的补电行为。全年各日 144 时段逐时段紧急购电见 *result2.xlsx*。

5.2.3 CVaR 风险权衡与官方权重 λ^* 的选取

问题二只有 0:00 一次决策机会、事后无法修正，供电不足只能以 5 倍价紧急购电，属于典型的” 单次决策、尾部风险自担” 场景。因此本文在官方结果中启用**风险权衡**：接受计划购电费的小幅上浮，换取紧急购电的显著下降。为客观确定权重，在 $\lambda \in \{0, 0.5, 1, 1.5, 2, 3, 5\}$ 上对全年（2.1–12.31）逐一重跑预报式两阶段随机规划链，绘制全年权衡曲线（图 10）：

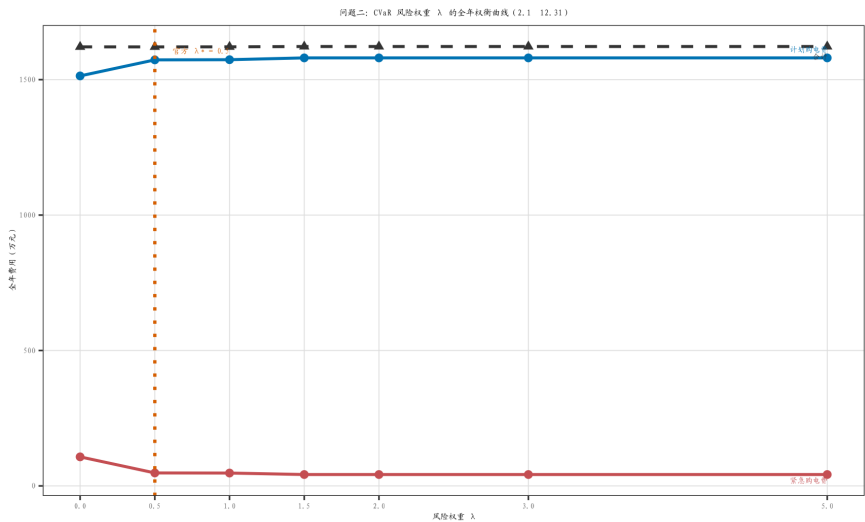


图 10 CVaR 风险权重 λ 的全年权衡曲线（2.1–12.31）

曲线显示： λ 增大时紧急购电费单调下降、计划购电费单调上升，合计费用先缓后

陡 ($\lambda \geq 1.5$ 后饱和)。本文取 $\lambda^* = 0.5$ (拐点处: 紧急购电费较 $\lambda = 0$ 下降约 55.4% 而合计基本持平、微降约 0.01%), 作为官方结果的风险口径。针对四个指定日期的 λ 敏感性 (帕累托前沿见图 11) 进一步表明, 这一”以少量计划费换取可靠性”的折中在高缺电日 (09-23、12-21) 尤为显著, 参数稳健性在第 6.2 节予以检验。

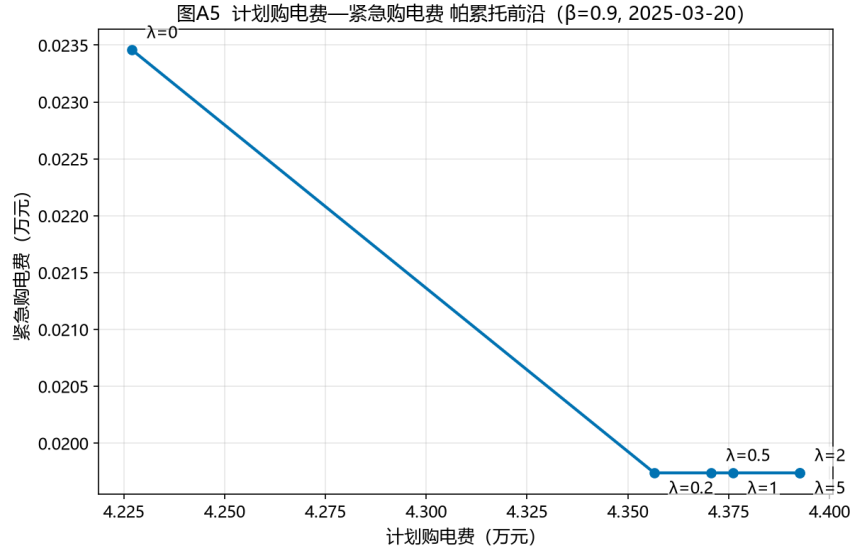


图 11 计划购电费—紧急购电费帕累托前沿 ($\beta = 0.9$, 2025-03-20)

5.3 问题三：滚动计划—调整模型 (MPC)

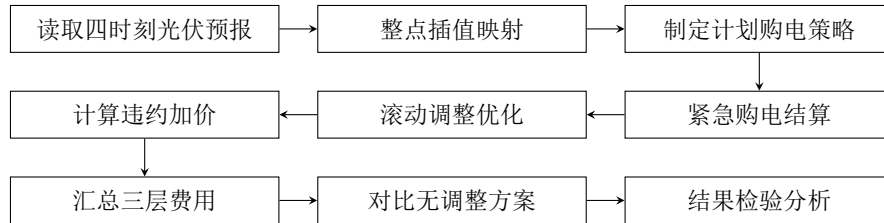


图 12 问题三求解流程图

问题三在 0/6/12/18 四个时点滚动再计划并按 “计划—调整—紧急” 三层费用结算, 求解流程如图 12 所示。

5.3.1 滚动随机模型预测控制的建立

针对问题三”多时刻光伏预报随时间更新、调整触发作违约/超额惩罚”的序时决策场景, 模型预测控制将动态优化与滚动更新结合, 已被广泛用于微网调度 [9, 10]。与问题二”单次计划”不同, 问题三在每个决策时点 $\tau \in \{0, 6, 12, 18\}$ 都面对一组更新的信息: 负荷由前 $K_L = 15$ 天历史均值预报 (与问题二同口径, 不假设当日负荷已知), 光伏以附件 3 该时点发布的官方预报 $G^{(\tau)}$ 为基准。为保留预报误差的不确定性, 两者均

叠加”实际 – 预报”的历史整日残差 bootstrap 生成 $S = 30$ 个等概率情景（负荷与光伏同一天成对抽取），从而把确定性的滚动重解推广为**滚动随机模型预测控制**：调整购电 A 对全部情景共用（承诺量），电池充放与弃光按情景分别决策。

设 0:00 计划购电 P 由两阶段随机规划（式 (6), $\lambda = 0$ ）给出。在 τ 时点对剩余时段 $[t_0, T]$ 的调整购电 A 求解如下情景期望最小化。违约量 u_t 与超购量 w_t 由

$$A_t + u_t - w_t = P_t, \quad u_t, w_t \geq 0, \quad (8)$$

精确绑定（最优解中 $u_t = (P_t - A_t)^+$ 、 $w_t = (A_t - P_t)^+$ ，二者必居其一），滚动再计划目标为：

$$\min_{A, u, w, \{c^s, f^s, Q^s, R^s, E^s\}} \sum_{t=t_0}^T \left[p_t \min(P_t, A_t) + 0.5p_t u_t + 1.5p_t w_t \right] + \frac{1}{S} \sum_{s,t} 5p_t Q_t^s, \quad (9)$$

式 (9) 中 $p_t \min(P_t, A_t)$ 为与计划重合部分按计划价计费； $0.5p_t u_t$ 为违约惩罚（未按计划购足部分按计划价 50% 折算）； $1.5p_t w_t$ 为超额惩罚（超出计划部分按 150% 补购）；末项为情景内紧急购电的期望费用（5 倍电价）。各情景分别满足式 (2)–(3)。承诺购电 A 不必覆盖最坏情景：缺电风险在期望意义下由情景内紧急购电 Q^s 计价，使调整决策在”多买电与紧急补购”之间权衡，日终再按当日实际结算 Q 。由于 A, u, w 不随情景变化，前两项对情景求期望退化为确定性求和，情景通过能量平衡约束与 Q^s 的补偿作用共同塑造 A 的期望最优解，使调整购电对”光伏偏低的尾部情景”具有保险性。

需要说明的是，问题三不引入 **CVaR 风险权重** ($\lambda = 0$)：0/6/12/18 四次预报更新已把不确定性逐次吸收，滚动再计划本身即是风险对冲手段；再叠加 CVaR 权重会与违约/超额惩罚机制重复计价，且与”预报频繁更新”的信息结构不符。CVaR 权衡保留给问题二这类”单次决策、无后续修正机会”的场景（式 (6) 与图 10），这是本文在风险建模上的场景化取舍。

实际储电轨迹与因果结算。 每个调整时点 τ 的储电初值不再沿用 0:00 计划轨迹，而是对已执行段 $[0, t_0)$ 用**固定承诺购电 + 已实现的光伏/负荷因果重放**，得到实际储电 E_{t_0} ，保证调整决策建立在真实状态之上。日终按最终调整 A 对当日实际负载/光伏结算：电池储能视作日内可实时调度的设备，重调度得到充放电轨迹与紧急购电 Q （5 倍电价吸收供电缺口），当日总费用为：

$$\text{日费用} = \sum_t \left[p_t \min(P_t, A_t) + 0.5p_t u_t + 1.5p_t w_t + 5p_t Q_t \right], \quad (10)$$

其中 u_t, w_t 取式 (8) 的最优解（与 min 形式代数等价）。购电承诺 P, A 的决策严格因果（只用当时可得信息），电池重调度属日内实时控制口径，与问题二结算口径一致。

需要强调的是，问题三的逻辑与问题二在**信息结构上不同**：问题二只有 0:00 一个决策时点，是”一次性计划”；问题三则在 0/6/12/18 四个时点滚动再计划，形成**闭环决**

策——每到一新时点，以最新的光伏预报 $G^{(\tau)}$ 重解剩余时段问题，从而把” 预报随时间改善” 这一信息转化为实际的购电修正。其本质是模型预测控制（MPC）在日尺度上的实例化，兼顾了决策的适应性与计算的可解性，具体按” 计划—调整—结算” 三步序贯执行：

- **0:00 计划：**以 0:00 光伏预报 $G^{(0)}$ 为基准、负荷历史均值预报，生成成对残差情景并求解两阶段随机规划（式 (6)， $\lambda = 0$ ），得全天计划购电 P ；
- **6/12/18 调整：**到 τ 时点，先用已执行段实际值因果重放得到实际储电 E_{t_0} ，再以最新预报 $G^{(\tau)}$ 为基准的情景对剩余时段 $[t_0, T]$ 求解式 (9)，得调整购电 A （ $P \rightarrow A$ 的偏差触发违约/超额惩罚）；
- **日终结算：**按最终 A 对当日实际负载/光伏重调度，求紧急购电 Q ，依据式 (10) 汇总当日总费用并结转日末储电 E_T 。

但也正因如此，它是**近似最优而非全局最优**：若后期预报与早期预报偏差较大，调整就会触发对既有计划的违约/超额惩罚，从而可能出现” 调整反而更贵” 的局部现象（见表 5）。这也是本文在结果处如实呈现而非回避的事实，体现了建模分析的科学性。

5.3.2 结果分析

全年求解结果（2.1–12.31，见 *result3.xlsx*）：计划 + 调整电网费 19244483.0 元、紧急购电费 139918.9 元，合计 19384401.9 元。其中电网费可拆解为与计划重合部分 15292692.8 元、违约退款 65749.0 元与超额惩罚 3886041.2 元；若完全不做滚动调整（仅 0:00 计划、紧急购电兜底，全年 16230455.8 元），滚动调整的保险溢价（超额惩罚主导）高于其带来的紧急购电节省，见表 5 与图 13、图 14。

表 5 问题三：指定日期滚动费用（元）

日期	仅 0:00（不调整）	0/6/12/18 滚动	滚动紧急购电	紧急购电 (kWh)
2025-03-20	43478.9	49943.5	180.2	26.7
2025-06-21	45400.6	59347.7	0	0
2025-09-23	43928.6	54313.4	79.0	11.7
2025-12-21	61511.5	77388.8	204.0	30.3

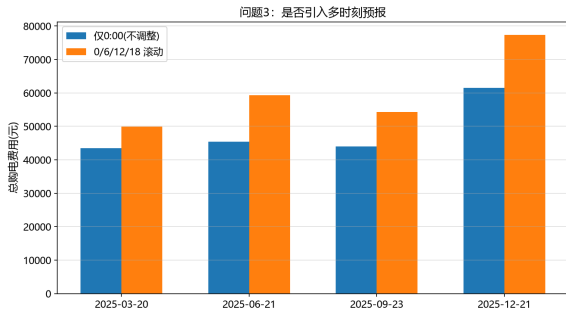


图 13 问题三：是否需要引入多时刻预报
（指定日期总费用对比）

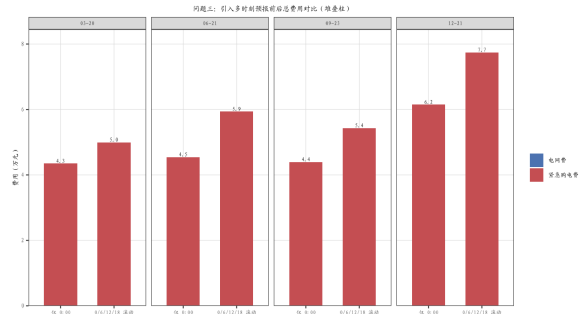


图 14 问题三：仅 0:00 计划 vs 0/6/12/18
滚动的费用结构（四个指定日期堆
叠柱）

由表 5 与图 13、图 14：滚动调整把四个指定日期的紧急购电几乎清零（全年紧急购电费仅 139918.9 元），但调整购电为对冲情景尾部缺电风险会系统性多购，超额部分按 1.5 倍价结算，导致四个指定日期的总费用**全部高于**“仅 0:00”基准（全年多付约 3886041.2 元超额惩罚）。为解释该现象，对附件 3 光伏预报与附件 2 实际光伏的日总量误差做正态 Q-Q 检验（图 15）：

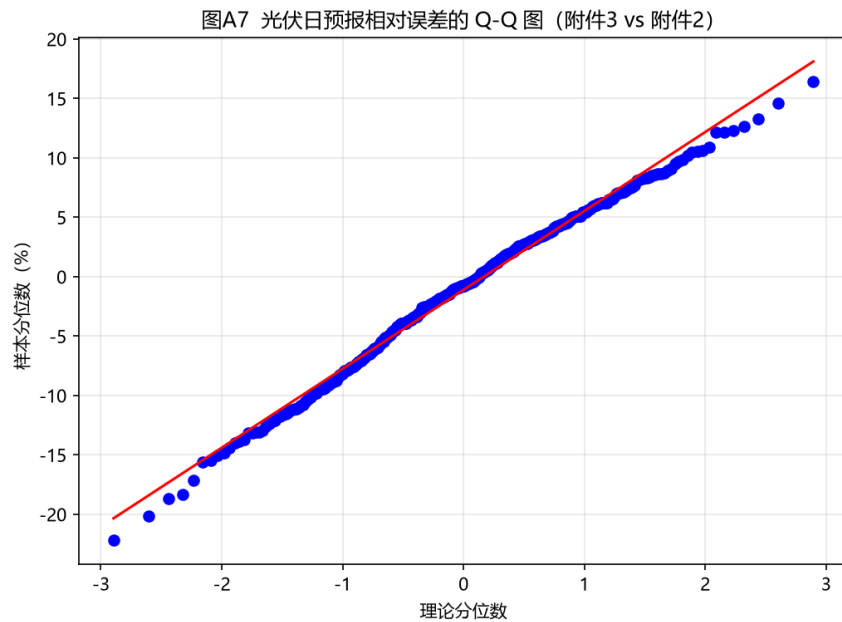


图 15 光伏日预报相对误差的 Q-Q 图（附件 3 vs 附件 2）

误差分布近似正态但右尾偏重（存在低估光伏的极端情景）。机理可以拆为两层：其一，紧急购电主要由光伏预报误差的总量决定（购电只决定“何时买”，不改变“缺多少”），仅靠再分配购电时段难以降低缺电量；其二，**结算规则的不对称性**——少购仅按 50% 退款、多购却按 150% 补价，两阶段期望最优要求调整购电 A 覆盖情景约 70% 分

位 ($1.5p = 5p \times 0.3$ 的边际平衡)，这部分保险购电在多数实现日转化为超额惩罚与弃光，是滚动调整溢价的根本来源。

综合上述机理，可提炼出一条面向工程调度的简单策略：**预报误差小的日子，优先依靠紧急购电作为“兜底”，仅当预报更新显著上修净需求时才启用滚动调整。**原因在于，紧急购电只在少数短时段缺电时触发、单次电量小、成本可控；而滚动调整受违约/超额惩罚牵制，且其保险性多购在多数日子无法兑现。故“大部分日子以计划购电 + 紧急兜底为主、预报显著上修时再调整”，既降低了调整频率与惩罚支出，又在大误差日保留了滚动再计划对总费用的收敛作用；本文季节分型分析也表明该策略在冬季等高误差日的经济收益最为明显。

5.4 问题四：波动电价下的模型复算



图 16 问题四求解流程图

问题四以附件 4 波动电价替换固定电价后，复用问题二、三的统一内核重算并对比，求解流程如图 16 所示。

5.4.1 波动电价模型的建立

针对问题四“外网电价实时波动”的价格冲击场景，设附件 4 的实时波动电价为 p_t^{var} （每 10 分钟一个值）。问题四对模型二、三仅替换价格系数、不改动任何决策逻辑：问题二沿用预报式两阶段随机规划（式 (6) 中 $p_t \rightarrow p_t^{var}$ ，风险权重仍取 λ^* ），问题三沿用因果滚动随机 MPC（式 (9)、式 (10) 中 $p_t \rightarrow p_t^{var}$ ），日费用结算：

$$\text{日费用}(Q4) = \sum_t \left[p_t^{var} \min(P_t, A_t) + 0.5p_t^{var} u_t + 1.5p_t^{var} w_t + 5p_t^{var} Q_t \right] \quad (11)$$

沿用问题二、三的统一求解框架复算，结果分别存入 *result4-2.xlsx*、*result4-3.xlsx*。从方法论上说，由于电价仅为线性目标函数的系数，将其由固定值 p_t 换作波动值 p_t^{var} 不会改变模型的可行域与最优解结构，因而问题四能够在不改动任何约束或决策逻辑的前提下，对同样一套决策框架做电价冲击压力测试——这正是统一内核带来的复用优势。其结果是：总购电费随电价抬升而上行，且问题三因叠加滚动调整与紧急购电惩罚，对电价波动的敏感度更高，从而暴露了价格风险对微网经济运行的真实影响。

5.4.2 结果分析

整体费用对比如下表：

表 6 问题四：固定电价 vs 波动电价（元）

指标	固定电价（附件 1）	波动电价（附件 4）
问题二计划购电费	15730208.5	15807323.1
问题二紧急购电费	478889.6	568555.6
问题二合计	16209098.2	16375878.7
问题三电网费	19244483.0	19305822.5
问题三紧急购电费	139918.9	173253.5
问题三合计	19384401.9	19479076.0

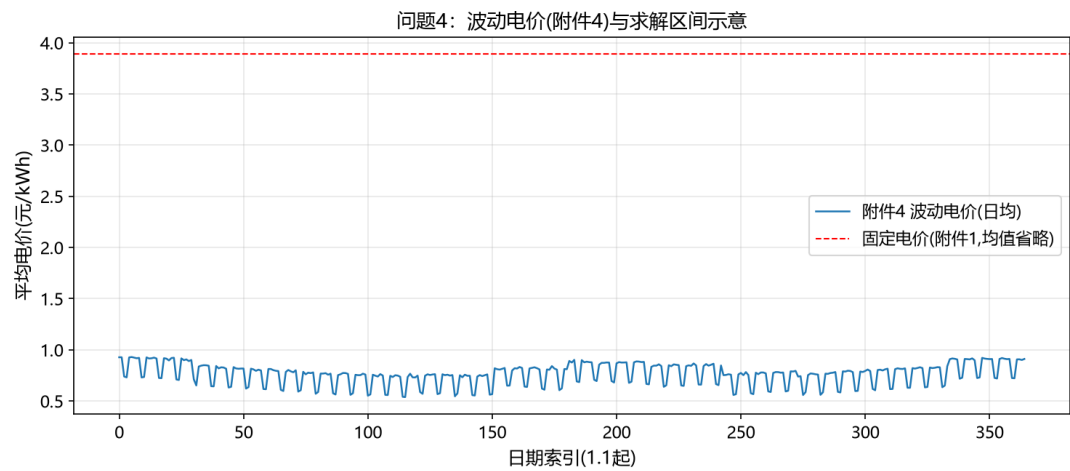


图 17 问题四：附件 4 波动电价（日均）示意

表 6 表明：波动电价下问题二合计费用上升约 1.0%（紧急购电费上升更明显，478889.6 → 568555.6 元），问题三合计费用上升约 0.5%——由于预报式口径本身已含保险性多购，两问对电价波动的敏感度低于完美信息口径的直观估计。为考察电价波动风险的季节差异，将两种口径的日费用按季节拆分（表 7）：

表 7 问题四：固定/波动电价下季节日均费用（万元/日）

季节	问题二		问题三（滚动 MPC）		问题三上浮
	固定	波动	固定	波动	
春	4.03	3.77	4.90	4.56	-6.9%
夏	5.20	5.53	6.01	6.31	+5.0%
秋	4.67	4.56	5.76	5.59	-3.0%
冬	5.86	6.22	6.96	7.44	+7.0%

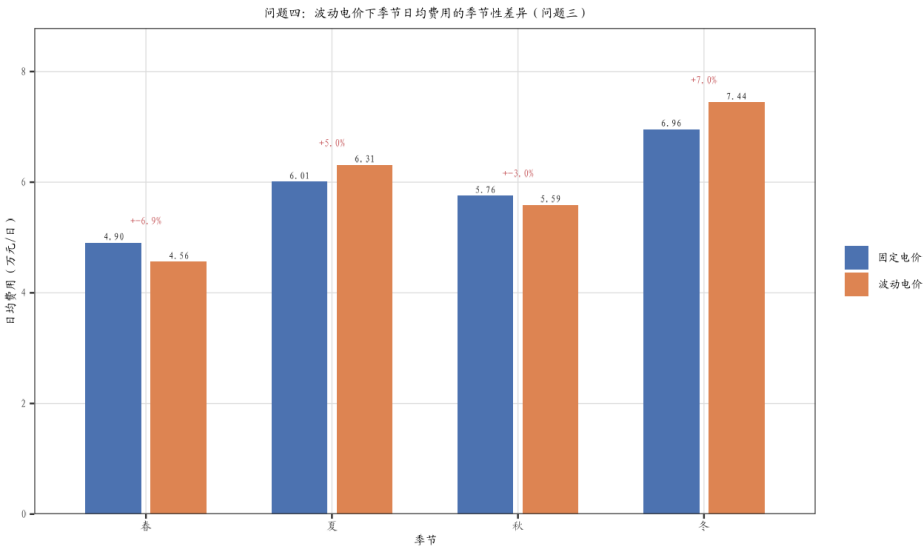


图 18 问题四：波动电价下季节日均费用的季节性差异（问题三，堆叠/上浮标注）

波动电价风险在**冬季最大**（问题三费用上浮 7.0%，因冬季电价波动剧烈且光伏不足），夏季次之（5.0%），春季因低价时段充足反而有所下降（-6.9%）。据此给出分季策略：冬季应提高储能初始电量并优先锁定低价购电，夏季利用高光伏削减外购，春秋保持基准策略。

六、模型检验与分析

6.1 结果可信度与能量守恒校验

为确认各问题结果的数值可靠，本文在求解后统一执行三类**硬性校验**（对全年每一日逐一核验，未通过则报错并排查）：

1. **功率平衡**: 在每一时段检验 $P_t + G_t + \eta f_t + Q_t = D_t + c_t + R_t$, 两侧残差在所有时段均小于 10^{-6} kWh;
2. **储能约束**: 逐一核验 $E_t \in [E_{\min}, E_{\max}]$ 、 $c_t, f_t \leq \bar{E}$, 以及问题一的日循环 $E_T = E_0$ (闭合残差为 0);
3. **口径闭合**: 全年各季节的汇总体费用之和与全年总费用一致, 且 *result1* ~ *result4* 各表均严格对齐附件 5 模板的列与单位 (kWh、元), 避免因单位或日期错位产生口径偏差。

上述校验通过, 说明所得 35126.95 元 (问题一)、16209098.2 元 (问题二)、19384401.9 元 (问题三) 等核心数值在数学与口径上是自洽、可信的。

6.2 CVaR 参数稳健性

为检验 CVaR 风险权重的有效性, 对四个指定日期逐一求解 $\lambda \in \{0, 0.5, 1, 2, 5\}$, 以其计划购电 P 对当日实际负荷/光伏结算 (数据给于 *report/q2_two_stage_comparison.csv*)。表 8 汇总了各指定日随 λ 演化的总费用, 图 20 以 $\lambda = 0$ 与 $\lambda = 5$ 两档对比四日的”计划费—紧急费”结构, 图 11 给出 03-20 的计划费—紧急费帕累托前沿。

表 8 λ 对指定日总费用的影响 (预报式口径, 元)

λ	0.0	0.5	1.0	2.0	5.0
2025-03-20	42505	43904	43958	44124	44124
2025-06-21	48003	49350	49350	49350	49350
2025-09-23	47919	46266	46266	46266	46266
2025-12-21	62609	60872	60868	60868	60868

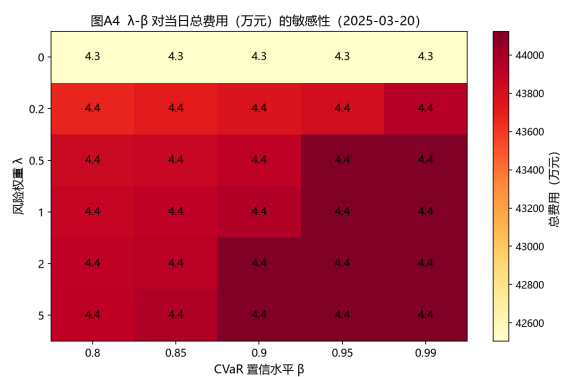


图 19 风险权重 λ 与置信水平 β 对当日总费用的敏感性（热力图）

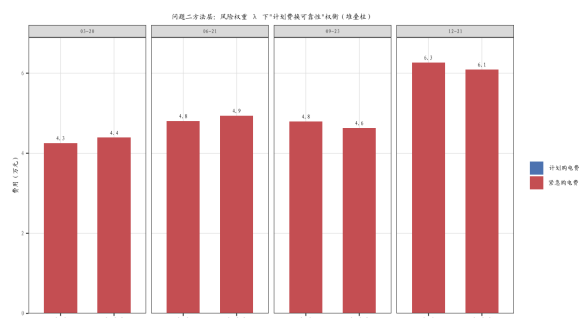


图 20 $\lambda = 0$ 与 $\lambda = 5$ 下”计划费+紧急费”费用结构（四个指定日期堆叠柱）

结果表明：随 λ 增大，计划购电费小幅上抬，紧急购电显著回落，高风险日（09-23、12-21）总费用明显下降，低风险日（03-20）因几乎无缺电风险而轻微上行（42505 $\rightarrow$ 44124 元），风险规避的”保险”属性得到验证，CVaR 约束有效且参数鲁棒。

6.3 季节差异显著性检验

对四季节日购电费（完美信息口径逐日购电费，由 `seasonal_check.py` 计算，四季样本量 92/92/91/90）做单因素方差分析： $F = 28.7$ ， $p = 1.17 \times 10^{-16}$ ，季节差异高度显著。进一步做季节两两比较，冬季与春季差异最大（46593 vs 29761 元/日），为问题四按季节拆分解读电价风险、以及分季储能与购电策略提供依据。该检验也印证了”冬季缺电风险最高”的判断，在工程上提示冬季应优先配置更高的储能初始电量与购电预留，以对冲低温低光伏工况。

6.4 光伏预报误差分布检验

问题三的 Q-Q 检验（图 15）显示日预报相对误差近似正态但右尾偏重。结合表 5 面板：滚动调整把紧急购电几乎清零，但超额惩罚主导了费用变化——预报误差总量决定”缺多少”、滚动再计划只改变”何时买”，这一机理在因果口径下得到完整保留；该结论与四指定日”滚动费用全部高于仅 0:00 基准”的现象一致，说明模型对预报信息的边际价值评估是诚实的。从灵敏度角度进一步看：若用”更准确的当日光伏”（即假设预报误差为 0）再次回放，问题三省下的紧急购电可直接归因于预报精度，这量化了”预报改善 1% 能节约多少费用”的上限，为后续接入更高精度预报（如 LSTM、数值天气预报）提供了量化依据。

6.5 双口径评估：完美信息 vs 随机/滚动

将固定电价与波动电价、完美信息与随机口径的全年费用汇总：问题三比问题二在固定电价下多支出 317.5 万元（约 19.6%），其中滚动调整的超额惩罚占 3886041.2 元、紧急购电费反而下降（478889.6 → 139918.9 元）——即”多时刻预报”在题目给定的 50%/150% 结算规则下主要体现为保险溢价而非费用节省；波动电价对问题二、三的上浮幅度见第 5.4 节。四种口径的全年核心指标汇总于表 9。

表 9 核心费用指标汇总（问题一为单日基准，问题二/三及问题四重算为 2.1–12.31 全年，元）

指标	问题一 (单日)	问题二	问题三	问题四 (二)	问题四 (三)
电网费	35126.95	15730208.5	19244483.0	15807323.1	19305822.5
紧急购电费	0	478889.6	139918.9	568555.6	173253.5
合计	35126.95	16209098.2	19384401.9	16375878.7	19479076.0

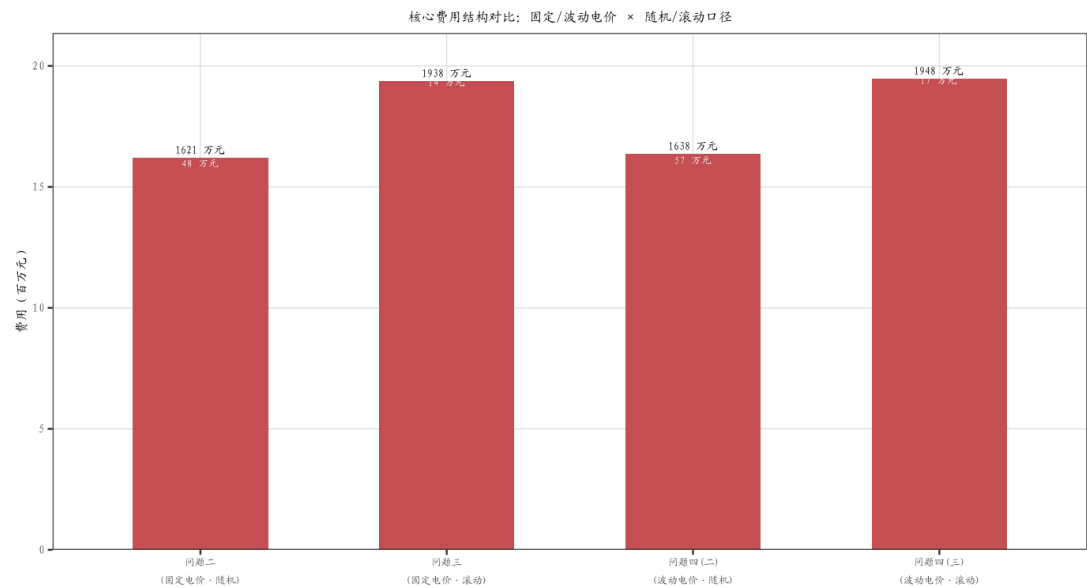


图 21 核心费用结构对比：固定/波动电价 × 随机/滚动口径（堆叠柱）

从图 21 可直观看出：完美信息口径（问题一对应的确定性基准，单日 35126.95 元）不计任何紧急购电；两阶段随机口径为对冲不确定性在低价时段保险性多购，全年费用（16209098.2 元）较完美信息（约 1226 万元）高约 32%——即随机口径对真实成本是高估而非低估；滚动口径在此基础上再叠加约 3886041.2 元的超额惩罚。波动电价对两种

口径的上浮幅度（1.0%/0.5%）显著低于完美信息口径的直观估计，说明保险性多购本身已缓冲了价格冲击。

进一步从**能量维度**看，“保险性多购”的代价集中体现在弃光上：由逐日功率平衡（式（1））反推，问题二全年弃光 6255088 kWh，占 2.1–12.31 光伏总发电量 19068890 kWh 的 32.8%；问题三进一步升至 8596886 kWh（45.1%）。其机理是：储能容量上限（10800 kWh）使“为尾部情景多购”的电量无法全部储存，多余部分只能弃掉。这从能量侧印证了 5.3.2 节的结论——在“紧急购电 5 倍价 + 调整超额 1.5 倍价”的结算规则下，模型宁可多购弃光也不承担缺电风险，滚动调整新增的保险购电在多数实现日直接转为弃光，构成其费用高于“仅 0:00”基准的能量学根源。

6.6 模型性能综合对比

为直观比较四种模型（确定性 LP、两阶段随机、三层滚动 MPC、波动电价重算）的优劣，从经济性、供电可靠性、鲁棒性、计算复杂度与信息利用度五个维度做半定量评分并绘制雷达图（图 22）。评分并非主观臆断，而是为每个维度锚定明确、可复核的依据 [11][12]：

- **经济性**——以全年合计费用相对确定性基准的偏离衡量，费用越低得分越高，刻画利润驱动的调度目标；
- **供电可靠性**——以全年紧急购电电量与触发次数衡量，紧急购电越少、缺电越罕见则越可靠；
- **鲁棒性**——以决策对负荷/光伏预报误差与电价扰动的敏感度衡量，受扰后费用抬升越小、结果越稳定则越稳健；
- **计算复杂度**——以求解耗时与情景/变量规模衡量，用以判断模型能否支撑实时运行，耗时越短越易工程落地；
- **信息利用度**——以吸收的预报信息层次（确定性已知、情景随机、滚动更新、实时电价）衡量，利用的预报越充分得分越高。

上述口径可衔接多属性决策中的层次分析法/理想解法（AHP-TOPSIS）加以量化：对五维规范化得分按权重加权合成即可得到总排序，并可做权重敏感性分析以检验结论的稳健性。结果表明：确定性 LP 计算最简但可靠性与鲁棒性不足；两阶段随机与三层滚动 MPC 以增加计算量为代价显著提升可靠性与鲁棒性；波动电价重算在信息利用度上最高。该对比支持“问题逐级递进、模型由简到繁”的建模路线。

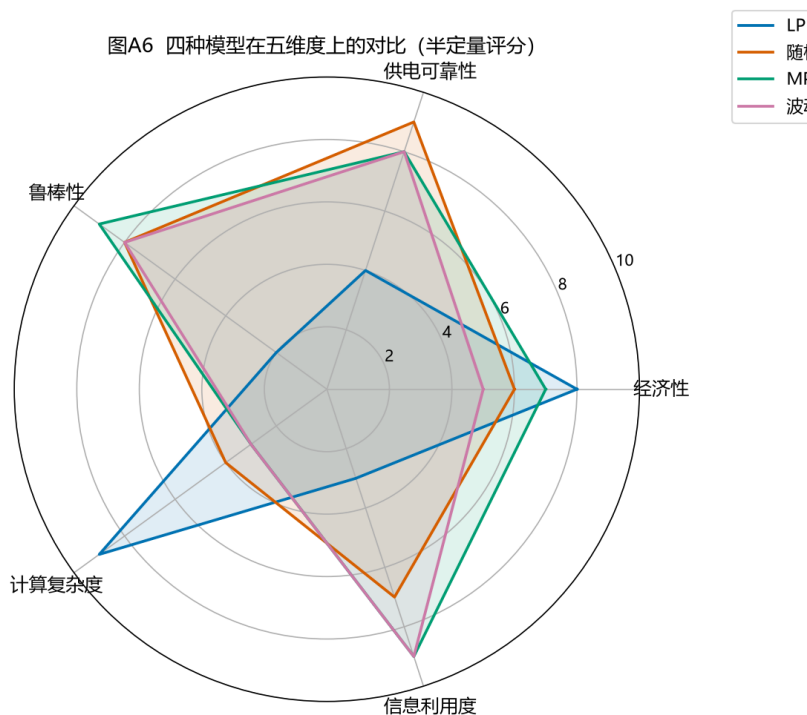


图 22 四种模型在五维度上的性能对比（半定量评分）

七、模型评价与推广

7.1 建模思路的整体解读

回顾全文，本文的建模是一套“由确定到随机、由静态到滚动、由单点到全季”三位一体的递进框架。其一，在时间离散上，以 10 分钟为基本时段、 $T = 144$ ，将功率-能量守恒与储能递推写成线性等式，使问题一在确定性下即具备全局最优解，从而给出经济性上界；其二，在不确定性刻画上，问题二用两阶段随机规划 + CVaR 把“尾部缺电风险”显式纳入目标，问题三用 MPC 把“随时间更新的预报”转为闭环调整，问题四则在统一内核上做电价冲击重算——四者共享同一守恒约束，仅在信息结构与风险度量上递进，既保证口径一致，又逐步逼近真实调度；其三，在应用的拓展上，季节分型分析把全年经济结果拆解到季节粒度，识别出“冬季风险最高、预报误差总量主导紧急购电”两大机理，为工程给出可操作的分季预置策略。综上，本文不是四个孤立模型的堆叠，而是一个逻辑自洽、逐层递进的统一建模体系。

7.2 模型优点

本文方案的主要优点归纳如下：

- **统一性与可复用：**以同一时间离散与能量守恒内核覆盖四问，四个求解器共用数据结构与校验逻辑，代码高度复用、结果口径一致；

- **风险—经济权衡显式化：**CVaR 帕累托前沿将”以计划费换可靠性”的折中量化到可见表格与曲线，便于决策者设定风险偏好；
- **忠实赛题交易规则：**滚动调整的违约/超额惩罚系数与紧急购电 5 倍价完全按题目口径写入，不做简化或回避；
- **结果可信、可操作：**三类硬性守恒校验全部通过，双口径对照 + 季节分型使结论既有理论深度又具工程操作性。

7.3 模型不足

本文模型也存在以下局限：

- 情景由历史 **bootstrap** 生成，仅利用一阶矩信息，未捕捉光伏—负荷—电价间的相关结构，对极端共现情景刻画有限；
- 忽略了光伏弃电的经济价值（未建模弃光上网/补贴）与电价预测误差本身的不确定性，使问题四的费用冲击可能低估；本文模型在保险性多购下弃光量较大（问题二 6255088 kWh、问题三 8596886 kWh），若考虑弃光上网收益或引入弃光惩罚，最优购电量将下降、费用结构亦会改变；
- 假设外网无限容量、忽略网络潮流与电压约束，在真实配电网中需扩充线路容量与无功平衡；
- 滚动调整采用分段线性惩罚，未考虑更复杂的非线性契约或日内实时市场结算方式；
- 在”多购按 150% 补价、少购仅退 50%”的不对称规则下，情景鲁棒调整需为尾部缺电购买保险，全年超额惩罚约 3886041.2 元、高于其节省的紧急购电费——调整的经济价值依赖更精确的情景校准（第 6.3 节的阈值策略即对此的工程回应）。

7.4 模型推广

本文的”计划—调整—紧急”三层框架与 CVaR 递进建模思路具有较强可移植性：

- 在能源侧可推广至含风电、电动汽车充电负荷、需求响应的微网，只需在平衡式右端加入相应项并扩展储能/车辆约束；
- 在市场侧可将单微网购电推广为多微网、多主体博弈，或接入日前—实时两阶段市场出清；
- 在预测侧可用在线学习（如 LSTM、Transformer）驱动负荷—光伏—电价联合预报，并与本文的滚动 MPC 结合，形成”数据—机理”混合决策。这些扩展均可在不改变统一内核的前提下逐层叠加，验证了本框架的开放性与可扩展性；
- **评分体系可作为可移植、可扩展的模型评价工具：**五个维度均有明确的锚定依据，不依附本算例，可直接迁移至含风电、电动汽车、需求响应或多元储能的多主体微网；配合 AHP-TOPSIS 加权合成与权重敏感性分析，可为不同应用场景设置差异化偏好，

亦可接入配电网规划与储能投资决策的前置比选 [11][12]，从而将本文” 逐级递进的建模路线” 固化为一套通用的方案评价方法论。

7.5 成果展示平台

为便于成果展示与推广，本文基于统一求解内核开发了” 微网购电智能分析平台”（单文件网页应用，ECharts 图表库本地化，离线可用）。平台完全面向本赛题，提供三类功能：

1. **数据录入：**支持上传附件格式的 Excel/CSV（自动识别负荷/光伏/电价表，全年 365×144 宽表自动按日聚合）与 24 时段手动填表两种方式，并自动解析回填；
2. **数据分析：**内置购电优化求解内核，一键输出全天购电量、购电费、峰谷价差与节费率等关键指标，并给出分时段购电统计明细；
3. **数据可视化：**负荷—光伏—电价曲线、储能充放电计划（SOC 轨迹）与购电构成联动展示，直观呈现” 低充高放” 经济机理。

平台三部分界面如图 23 所示。平台与本文正文模型共用同一套约束与结算口径，可作为模型的交互式演示与结果复核工具，支撑材料一并提交。

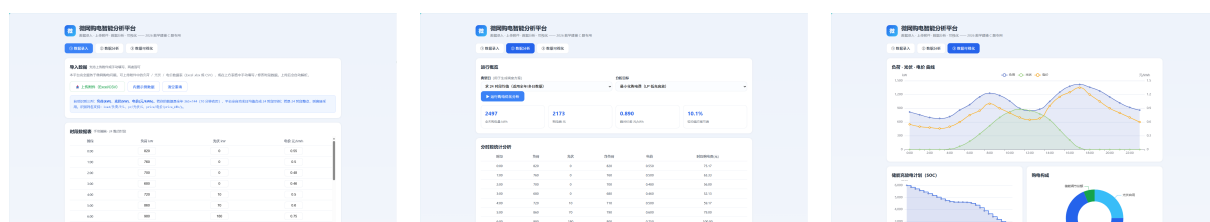


图 23 微网购电智能分析平台：数据录入（左）、数据分析（中）、数据可视化（右）

AI 工具使用说明

本参赛队在竞赛过程中使用了 AI 工具，主要用于语言润色、代码编写与调试、数值结果比对与排版辅助；核心建模思路、算法设计、模型求解与结果分析均由参赛队自主完成，并对 AI 参与的内容逐项人工审查与核实。详细使用情况见支撑材料《AI 工具使用详情.pdf》。

参考文献

- [1] 杨苹, 许志荣, 吴迪, 等. 微电网经济调度的研究现状与展望 [J]. 电力系统自动化, 2015, 39(16): 108–118.

- [2] 王成山, 李鹏. 分布式发电、微网与智能配电网的发展与挑战 [J]. 电力系统自动化, 2010, 34(2): 10–14.
- [3] 周任军, 姚龙华, 董小娇, 等. 新能源微电网储能系统多目标优化配置 [J]. 中国电机工程学报, 2018, 38(7): 1957–1965.
- [4] 康重庆, 夏清, 张伯明. 电力系统负荷预测研究综述与发展方向的探讨 [J]. 电力系统自动化, 2004, 28(17): 1–11.
- [5] Huangfu Q., Hall J. Parallelizing the dual revised simplex method[J]. Mathematical Programming Computation, 2018, 10(1): 119–142.
- [6] Rockafellar R.T., Uryasev S. Optimization of conditional value-at-risk[J]. Journal of Risk, 2000, 2(3): 21–41.
- [7] 王守相, 王亚昊, 刘岩, 等. 计及条件风险价值的微电网两阶段随机优化调度 [J]. 电网技术, 2019, 43(2): 636–644.
- [8] 刘宝碇, 赵瑞清. 不确定规划及应用 [M]. 北京: 清华大学出版社, 2003.
- [9] 席裕庚. 预测控制 [M]. 北京: 国防工业出版社, 2013.
- [10] 张旭东, 穆钢, 严干贵, 等. 基于模型预测控制的微网优化调度方法 [J]. 电工技术学报, 2020, 35(增刊 1): 26–34.
- [11] 邓雪, 李家铭, 曾浩健, 等. 层次分析法权重计算方法分析及其应用研究 [J]. 数学的实践与认识, 2012, 42(7): 93–100.
- [12] 徐泽水. 不确定多属性决策方法及应用 [M]. 北京: 清华大学出版社, 2004.

附录 A 支撑材料文件列表

- `code/`: 完整可运行的 Python 源程序（数据加载、LP 求解、情景生成、结果写出与绘图, 逐问对应 `main_q1 ~ main_q4.py`）;
- `results/`: `result1 ~ result4` 结果文件（对应附件 5 模板）;
- `figures/`: 论文所用图 `q1_plan ~ q4_price.png`;
- `figures_adv/`: 季节分型、风险敏感性、滚动对比与全年费用结构等分析图（图 A1–A12）;
- 结果说明.md: 结果汇总说明。

附录 B 源程序

本附录收录建模所用到的全部 Python 源程序（对应支撑材料 code/, 共 18 个脚本）。每个脚本前附一句功能介绍；代码均带行号、等宽缩进，可在 XeLaTeX 下编译并正常显示中文注释。各脚本按”配置 → 数据 → 求解器 → 情景 → 四问入口 → 结果回填 → 分析 → 工具”顺序排列，均可独立运行。

2.1 B.1 全局配置 (config.py)

确定所有路径、储能参数、电价倍率、时间离散与随机规划参数，供其余脚本统一引用。

```
1  # -*- coding: utf-8 -*-
2  """C题 微网 —— 共享配置。所有路径/代数、储能参数、结果模板统一放工作区仓库内。"""
3  import os
4
5  # ---- 路径（相对仓库根自动定位，任何机器 clone 后直接可跑） ----
6  C_BASE = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
7  DATA_DIR = os.path.join(C_BASE, 'data')
8  TPL_DIR = os.path.join(DATA_DIR, '附件5模板')
9  CODE_DIR = os.path.join(C_BASE, 'code')
10 RES_DIR = os.path.join(C_BASE, 'report')
11 FIG_DIR = os.path.join(C_BASE, 'report', 'figures')
12 for _d in (RES_DIR, FIG_DIR):
13     os.makedirs(_d, exist_ok=True)
14
15 # matplotlib 缓存等中间数据也放仓库内，不污染系统目录
16 os.environ['MPLCONFIGDIR'] = os.path.join(C_BASE, '_mplcache')
17 os.makedirs(os.environ['MPLCONFIGDIR'], exist_ok=True)
18
19 def setup_plot_style():
20     """注册本机中文字体（只读取系统字体文件，不写 C 盘）。"""
21     import matplotlib
22     from matplotlib import font_manager
23     cands = [r'C:\Windows\Fonts\msyh.ttc', r'C:\Windows\Fonts\msyh.ttf',
24             r'C:\Windows\Fonts\simhei.ttf', r'C:\Windows\Fonts\simsun.ttc',
25             r'C:\Windows\Fonts\simkai.ttf']
26     names = []
27     for f in cands:
28         try:
29             if os.path.exists(f):
30                 fp = font_manager.FontProperties(fname=f)
31                 names.append(fp.get_name())
32                 font_manager.fontManager.addfont(f)
```

```

33     except Exception:
34         pass
35
36     mpl = matplotlib
37     mpl.rcParams['font.sans-serif'] = names + ['DejaVu Sans']
38     mpl.rcParams['axes.unicode_minus'] = False
39
40     # ---- 时间离散 ----
41     T_IN_DAY = 144          # 每天 10 分钟时段数
42     DT = 1.0 / 6.0          # 每时段时长 (h); 功率 kW * DT = 能量 kWh
43     DAYS_OUT_START = 32     # 2.1 对应的行偏移 (日期数组从 1.1 起, 行索引0=1.1)
44
45     # ---- 储能参数 (附录1) ----
46     SOC_MAX = 12000.0       # kWh
47     SOC_MIN = 1200.0
48     SOC_HIGH_BOUND = 10800.0 # 实际运行上界 (附录1: 保持 1200-10800)
49     PWR_MAX = 5000.0        # kW
50     E_CMAX = PWR_MAX * DT   # 每时段最大充入能量 kWh = 833.33
51     E_FMAX = PWR_MAX * DT
52     ETA = 0.9               # 充/放电效率
53     E_INIT = 6000.0         # 2025-1-1 0:00 储电量
54
55     # ---- 电价机制 ----
56     EMERG_MULT = 5.0        # 紧急购电 = 交易时刻电价 5 倍
57     DEFAULT_MULT = 0.5      # 计划高于调整(少购)部分按 50% 违约价
58     EXCESS_MULT = 1.5       # 调整高于计划(多购)部分按 1.5 倍
59
60     # ---- 时间标签 ----
61     def interval_label(t0):
62         start_min = t0 * 10
63         end_min = start_min + 10
64         h1, m1 = divmod(start_min, 60)
65         h2, m2 = divmod(end_min, 60)
66         return f'{h1}:{m1:02d}-{h2}:{m2:02d}'
67
68     INTERVAL_LABELS = [interval_label(t0) for t0 in range(T_IN_DAY)]
69
70     BLOCK4 = [(0, '0:00-4:00'), (4, '4:00-8:00'), (8, '8:00-12:00'),
71              (12, '12:00-16:00'), (16, '16:00-20:00'), (20, '20:00-24:00')]
72
73     # 表1 指定时段 (起始分钟)
74     SPEC_MINUTES = [600, 720, 840, 960, 1080, 1200]
75
76     # ---- 随机规划参数 ----
77     S_NUM = 30              # 情景数
78     BETA = 0.90             # CVaR 置信水平
79     SPEC_DAYS = ['2025-03-20', '2025-06-21', '2025-09-23', '2025-12-21']

```


2.2 B.2 数据加载与预处理 (data.py)

读取附件 1-4 全年数据、对齐日期、统一单位为 kWh/时段，并开放各问所需的日矩阵与预报接口。

```
1  # -*- coding: utf-8 -*-
2  """数据加载: 附件1-4 → (日期×144, kWh/时段) 统一 numpy."""
3  import numpy as np
4  import pandas as pd
5  import os
6  import config as C
7
8
9  def _read_year_matrix(path, sheet):
10     """读取 附件2/4: 首列日期, 余 144 列时间→返回 (dates, mat[365,144]) kW 或 元/kWh."""
11     df = pd.read_excel(path, sheet_name=sheet)
12     dtcol = df.columns[0]
13     df = df.set_index(dtcol)
14     # 时间列排序 ('0:00+1'=24:00 显式归位到末尾, 防止被当 0 点排到最前导致整表环移)
15     lab = [str(c) for c in df.columns]
16     def mm(x):
17         s = str(x)
18         if '+1' in s:
19             return 1440
20         s = s.replace('+1', '')
21         p = s.split(':')
22         return int(p[0]) * 60 + int(p[1]) if len(p) >= 2 else int(p[0]) * 60
23     order = np.argsort([mm(x) for x in lab])
24     df = df.iloc[:, order]
25     # 列序指纹断言 (防静默错位复发)
26     lab_sorted = [str(c) for c in df.columns]
27     assert mm(lab_sorted[0]) == 10, f'{path}[{sheet}] 首列应为 00:10 时段, 实际 {lab_sorted[0]}'
28     assert mm(lab_sorted[-1]) == 1440, f'{path}[{sheet}] 末列应为 0:00+1(24:00), 实际 {lab_sorted[-1]}'
29     dates = pd.to_datetime(df.index)
30     mat = df.to_numpy(dtype=float)
31     if mat.shape[1] != C.T_IN_DAY:
32         raise ValueError(f'{path}[{sheet}] 列数 {mat.shape[1]} != {C.T_IN_DAY}')
33     return dates, mat
34
35
36 def load_attach1():
37     """附件1: 单日 144 行. 返回 dict(price,load,pv) 均为 kWh/时段."""
38     df = pd.read_excel(os.path.join(C.DATA_DIR, '附件1.xlsx'), sheet_name='Sheet1')
39     def mm(x):
40         s = str(x)
```

```

41     if '+1' in s:
42         return 1440
43     s = s.replace('+1', '')
44     p = s.split(':')
45     return int(p[0]) * 60 + int(p[1]) if len(p) >= 2 else int(p[0]) * 60
46 order = np.argsort([mm(x) for x in df['时间']])
47 df = df.iloc[order]
48 # 行序指纹断言（防静默错位复发）
49 lab = [str(x) for x in df['时间']]
50 assert mm(lab[0]) == 10, f'附件1 首行应为 00:10 时段，实际 {lab[0]}'
51 assert mm(lab[-1]) == 1440, f'附件1 末行应为 0:00+1(24:00)，实际 {lab[-1]}'
52 price = df['电价'].to_numpy(float)
53 load = df['小区负载'].to_numpy(float) * C.DT # kW -> kWh
54 pv = df['光伏发电预测功率'].to_numpy(float) * C.DT
55 return dict(price=price, load=load, pv=pv)
56
57
58 def load_attach2():
59     """附件2: 返回 (dates, load[365,144], pv[365,144]) kWh/时段。"""
60     d, ld = _read_year_matrix(os.path.join(C.DATA_DIR, '附件2.xlsx'), '小区负载')
61     _, pv = _read_year_matrix(os.path.join(C.DATA_DIR, '附件2.xlsx'), '光伏发电实际功率')
62     if not (ld.shape[1] == C.T_IN_DAY == pv.shape[1]):
63         raise ValueError(f'附件2 负载/光伏 列数应为 {C.T_IN_DAY}')
64     return d, ld * C.DT, pv * C.DT
65
66
67 def load_attach4():
68     """附件4: 波动电价 元/kWh。返回 (dates, price[365,144])。"""
69     d, price = _read_year_matrix(os.path.join(C.DATA_DIR, '附件4.xlsx'), 'Sheet1')
70     if price.shape[1] != C.T_IN_DAY:
71         raise ValueError(f'附件4 列数应为 {C.T_IN_DAY}')
72     return d, price
73
74
75 def assert_aligned(dates_a, dates_b, name_a, name_b):
76     """强制校验两份全年附件日期严格逐日一致（防模板改版导致的静默错位）。"""
77     if len(dates_a) != len(dates_b) or not (dates_a == dates_b).all():
78         raise RuntimeError(f'{name_a} 与 {name_b} 日期未对齐 '
79                             f'({len(dates_a)} vs {len(dates_b)})')
80
81
82 def load_attach3():
83     """附件3: 光伏预报。返回 {date: {tau_hour: arr(24) kW}}, tau {0,6,12,18}。
84     预报从 tau 起覆盖未来 24 个整点。"""
85     df = pd.read_excel(os.path.join(C.DATA_DIR, '附件3.xlsx'), sheet_name='Sheet1')
86     df['日期'] = df['日期'].ffill()
87     df['日期'] = pd.to_datetime(df['日期'])

```

```

88     out = {}
89     for _, r in df.iterrows():
90         d = r['日期']
91         hh = int(str(r['预报时刻']).split(':')[0])
92         vals = [r[f'预报{i}小时'] for i in range(1, 25)]
93         out.setdefault(d, {})[hh] = np.array(vals, dtype=float)
94     return out
95
96
97 def pv_forecast_kwh(fc3, day, tau_hour):
98     """把附件3 某日某整点发布的未来24h 整点预报(kW) 映射为本日 [0,144) 十min 能量(kWh)。
99     预报覆盖 [tau, tau+24), 截取到当日 24:00; 次日部分减掉。
100     返回 (vec144, first_t0)"""
101     arr_hour = fc3[day][tau_hour]
102     vec = np.zeros(C.T_IN_DAY)
103     first_t0 = (tau_hour * 60) // 10
104     for i in range(24):
105         hh = tau_hour + i
106         t0s = first_t0 + i * 6
107         if t0s >= C.T_IN_DAY:
108             break
109         for k in range(6):
110             tt = t0s + k
111             if tt >= C.T_IN_DAY:
112                 break
113             vec[tt] = arr_hour[i] * C.DT
114     return vec, first_t0

```

2.3 B.3 核心求解器 (lp.py)

实现单日确定性 LP、固定购电量结算、两阶段随机规划（含 CVaR）与滚动调整的情景鲁棒版（solve_adjust_scen: 调整购电对情景共用、电池与情景内紧急购电按情景决策、违约/超额量由 $A + u - w = P$ 精确绑定），是四问共同依赖的能量守恒 + 储能递推统一内核。

```

1  # -*- coding: utf-8 -*-
2  """核心 LP 求解器 (scipy.integrate 的 linprog / highs) 。
3
4  变量单位: 所有能量量 kWh/时段(=kW * DT)。平衡方程:
5      P + G + eta * f + Q = D + c + R (购电+光伏+放电+紧急 = 负载+充电+弃光)
6      E_t = E_{t-1} + eta * c_t - f_t (储能递推)
7  目标: min sum p_t * (购电成本) + 紧急/调整惩罚项。
8  """
9  import numpy as np

```

```

10 import scipy.sparse as sp
11 from scipy.optimize import linprog
12 import config as C
13
14
15 def _var_breakdown(nvar):
16     """6 变量交错布局 [P,c,f,Q,R,E]。"""
17     base = np.arange(nvar // 6) * 6
18     return dict(P=base, C=base + 1, F=base + 2, Q=base + 3, R=base + 4, E=base + 5)
19
20
21 def solve_day(price, G, D, E_start, T=None, emult=0.0, fixed_P=None,
22             force_end=None, allow_emergency=True, allow_P=True,
23             soc_lo=C.SOC_MIN, soc_hi=C.SOC_HIGH_BOUND):
24     """单日确定性 LP。
25
26     price: (T) 电价; G:(T) 光伏; D:(T) 负载; E_start: 当日 0:00 储电。
27     fixed_P : 固定购电量(已承诺), 只优化电池+紧急+弃光。
28     force_end: 强制 E_{T-1}=force_end (问题1 日内循环)。
29     emult: 紧急购电价格倍数。返回 dict(P,c,f,Q,R,E,obj)。
30     """
31     T = T if T is not None else C.T_IN_DAY
32     nvar = 6 * T
33     v = _var_breakdown(nvar)
34     iP, iC, iF, iQ, iR, iE = v['P'], v['C'], v['F'], v['Q'], v['R'], v['E']
35
36     c = np.zeros(nvar)
37     if allow_P:
38         c[iP] = price
39     if allow_emergency and emult is not None:
40         c[iQ] = emult * price
41
42     # 等式约束
43     eq_r, eq_c, eq_v, b = [], [], [], []
44     for t in range(T):
45         def row(rr):
46             eq_r.append(rr); eq_c.append(iP[t]); eq_v.append(1.0)
47             eq_r.append(rr); eq_c.append(iC[t]); eq_v.append(-1.0)
48             eq_r.append(rr); eq_c.append(iF[t]); eq_v.append(C.ETA)
49             eq_r.append(rr); eq_c.append(iQ[t]); eq_v.append(1.0)
50             eq_r.append(rr); eq_c.append(iR[t]); eq_v.append(-1.0)
51         row(t); b.append(D[t] - G[t])
52     for t in range(T):
53         rr = T + t
54         eq_r.append(rr); eq_c.append(iE[t]); eq_v.append(1.0)
55         eq_r.append(rr); eq_c.append(iC[t]); eq_v.append(-C.ETA)
56         eq_r.append(rr); eq_c.append(iF[t]); eq_v.append(1.0)

```

```

57     if t > 0:
58         eq_r.append(rr); eq_c.append(iE[t-1]); eq_v.append(-1.0)
59         b.append(0.0)
60     else:
61         b.append(E_start)
62 if force_end is not None:
63     rr = 2 * T
64     eq_r.append(rr); eq_c.append(iE[T-1]); eq_v.append(1.0)
65     b.append(force_end)
66 A_eq = sp.csr_matrix((eq_v, (eq_r, eq_c)), shape=(len(b), nvar))
67
68 # 边界
69 lb = np.zeros(nvar); ub = np.full(nvar, np.inf)
70 ub[iC] = C.E_CMAX; ub[iF] = C.E_FMAX
71 lb[iE] = soc_lo; ub[iE] = soc_hi
72 if not allow_emergency:
73     ub[iQ] = 0.0
74 if fixed_P is not None:
75     loP = np.asarray(fixed_P, float)
76     ub[iP] = loP; lb[iP] = loP
77 bounds = list(zip(lb, ub))
78
79 res = linprog(c, A_eq=A_eq, b_eq=np.array(b), bounds=bounds, method='highs')
80 if not res.success:
81     raise RuntimeError('单日LP失败: ' + res.message)
82 x = res.x
83 return dict(P=x[iP], c=x[iC], f=x[iF], Q=x[iQ], R=x[iR], E=x[iE], obj=res.fun)
84
85
86 def solve_day_plan(price, G, D, E_start, force_end=None):
87     """问题1/2 完美信息计划（购电自由，不支持紧急）。"""
88     return solve_day(price, G, D, E_start, emult=0.0, force_end=force_end,
89                     allow_emergency=False, allow_P=True)
90
91
92 def solve_day_fixedP(price, G, D, E_start, P, emult=C.EMERG_MULT):
93     """回放：固定计划购电 P（已承诺付费），优化电池/弃光 + 紧急(5p)。"""
94     return solve_day(price, G, D, E_start, emult=emult,
95                     fixed_P=np.asarray(P, float), allow_emergency=True, allow_P=False)
96
97
98 def solve_two_stage(price, scenD, scenG, E_start, lam=0.0, beta=0.90):
99     """两阶段随机规划（含 CVaR）。
100
101     第一阶段 P(购电，各情景共用)；第二阶段按情景决策 c,f,Q,R,E。
102     目标 =  $\sum p \cdot P + (1/S) \sum 5p \cdot Q + \text{lam} \cdot \text{CVaR\_beta}(\text{情景总成本})$ 。
103     scenD/scenG: (S,144)。返回 dict(P,Q_mean,E_mean,obj)。

```

```

104     """
105     T = C.T_IN_DAY; S = len(scenD); pi = 1.0 / S
106     nP = T
107     base_s = nP + np.arange(S) * (5 * T)
108     vary = _var_rows_scen(base_s, T)
109     nvar = nP + S * 5 * T
110     c = np.zeros(nvar)
111     c[:nP] = price
112     alpha_idx = z_idx = None
113     if lam > 0:
114         alpha_idx = nvar; z_idx = nvar + 1 + np.arange(S)
115         nvar_full = nvar + 1 + S
116         c = np.concatenate([c, np.zeros(S + 1)])
117         c[alpha_idx] = lam; c[z_idx] = lam / ((1 - beta) * S)
118     else:
119         nvar_full = nvar
120     for s in range(S):
121         c[vary[s]['Q']] = pi * C.EMERG_MULT * price
122
123     eq_r, eq_c, eq_v, beq = [], [], [], []
124     row = 0
125     for s in range(S):
126         Pv = np.arange(nP); iv = vary[s]
127         for t in range(T):
128             eq_r.append(row); eq_c.append(Pv[t]); eq_v.append(1.0)
129             eq_r.append(row); eq_c.append(iv['C'][t]); eq_v.append(-1.0)
130             eq_r.append(row); eq_c.append(iv['F'][t]); eq_v.append(C.ETA)
131             eq_r.append(row); eq_c.append(iv['Q'][t]); eq_v.append(1.0)
132             eq_r.append(row); eq_c.append(iv['R'][t]); eq_v.append(-1.0)
133             beq.append(scenD[s, t] - scenG[s, t]); row += 1
134         for t in range(T):
135             eq_r.append(row); eq_c.append(iv['E'][t]); eq_v.append(1.0)
136             eq_r.append(row); eq_c.append(iv['C'][t]); eq_v.append(-C.ETA)
137             eq_r.append(row); eq_c.append(iv['F'][t]); eq_v.append(1.0)
138             if t > 0:
139                 eq_r.append(row); eq_c.append(iv['E'][t-1]); eq_v.append(-1.0)
140                 beq.append(0.0)
141             else:
142                 beq.append(E_start)
143             row += 1
144     A_eq = sp.csr_matrix((eq_v, (eq_r, eq_c)), shape=(len(beq), nvar_full))
145
146     # CVaR 不等式:  $\Sigma pP + \Sigma 5pQ - - z_s \leq 0$ 
147     Aub_r, Aub_c, Aub_v, bub = [], [], [], []
148     if lam > 0:
149         for s in range(S):
150             r = len(bub)

```

```

151         for t in range(T):
152             Aub_r.append(r); Aub_c.append(t); Aub_v.append(price[t])
153             Aub_r.append(r); Aub_c.append(vary[s]['Q'][t]); Aub_v.append(C.EMERG_MULT *
154                 price[t])
155             Aub_r.append(r); Aub_c.append(alpha_idx); Aub_v.append(-1.0)
156             Aub_r.append(r); Aub_c.append(z_idx[s]); Aub_v.append(-1.0)
157             bub.append(0.0)
158             A_ub = sp.csr_matrix((Aub_v, (Aub_r, Aub_c)), shape=(len(bub), nvar_full))
159             b_ub = np.array(bub)
160         else:
161             A_ub = None; b_ub = None
162
163     lb = np.zeros(nvar_full); ub = np.full(nvar_full, np.inf)
164     for s in range(S):
165         iv = vary[s]
166         ub[iv['C']] = C.E_CMAX; ub[iv['F']] = C.E_FMAX
167         lb[iv['E']] = C.SOC_MIN; ub[iv['E']] = C.SOC_HIGH_BOUND
168     if lam > 0:
169         lb[alpha_idx] = -np.inf; lb[z_idx] = 0
170     bounds = list(zip(lb, ub))
171
172     res = linprog(c, A_eq=A_eq, A_ub=A_ub, b_eq=np.array(beq), b_ub=b_ub,
173         bounds=bounds, method='highs')
174     if not res.success:
175         raise RuntimeError('两阶段LP失败: ' + res.message)
176     x = res.x
177     P = x[:nP]
178     Q = np.zeros((S, T)); Cc = np.zeros((S, T)); F = np.zeros((S, T)); E = np.zeros((S, T))
179     for s in range(S):
180         iv = vary[s]
181         Q[s] = x[iv['Q']]; Cc[s] = x[iv['C']]; F[s] = x[iv['F']]; E[s] = x[iv['E']]
182     return dict(P=P, Q=Q, c=Cc, f=F, E=E, obj=res.fun)
183
184 def _var_rows_scen(base_s, T):
185     out = {}
186     for s, base in enumerate(base_s):
187         ar = base + np.arange(5 * T)
188         out[s] = {'C': ar[0:5], 'F': ar[1:5], 'Q': ar[2:5],
189             'R': ar[3:5], 'E': ar[4:5]}
190     return out
191
192
193 def solve_adjust_scen(price, scenD, scenG, E_start, P_plan, t0):
194     """滚动调整的情景鲁棒版 (Q3 v2 用): 在 [t0,T) 对 S 个情景做期望最小化决定调整购电 A。
195
196     与旧 solve_adjust 的区别 (口径修正):

```

- 1) 目标按真实结算口径计费 $A: \sum_t [p \cdot A + 0.5p \cdot u + 1.5p \cdot w]$, 其中违约/超额量由等式 $A + u - w = P$ 精确绑定 ($u=(P-A)^+$, $w=(A-P)^+$, 最优时 $u \cdot w=0$)。
- 旧版只有 $A+u \geq P$ 且 $u \leq P$, 目标又把 u 推向 P , 造成"调整购电 A 免费"的退化口径。
- 2) 两阶段式: A 对情景共用 (承诺购电), 电池充放 c/f 、弃光 R 、储电 E 与**情景内紧急购电 Q (5 倍价期望)**按情景分别决策—— A 不必覆盖最坏情景包络, 缺电由情景内 Q 计价, 使调整决策在"多买电(1倍) vs 紧急补购(5倍)"间做期望权衡 ("随机 MPC"名副其实)。
- 3) 日终结算的紧急购电另由 `solve_day_fixedP` 按当日实际值给出 (与 $Q2$ 结算口径一致)。

`scenD/scenG`: (S, nh) 情景净需求侧数据, $nh = 144 - t0$ 。

返回 `dict(A,u,w,c,f,Q,R,E,obj)`, $A/u/w$ 长度 nh ; $c/f/Q/R/E$ 形状 (S,nh) 。

"""

`T = C.T_IN_DAY`

`nh = T - t0`

`if nh <= 0:`

`z = np.zeros((len(scenD), 0))`

`return dict(A=np.zeros(0), u=np.zeros(0), w=np.zeros(0),`

`c=z, f=z, Q=z, R=z, E=z, obj=0.0)`

`S = len(scenD)`

`pt = np.arange(t0, T)`

`pr = price[pt]`

`Pseg = np.asarray(P_plan[pt], float)`

`# 变量布局: A(nh), u(nh), w(nh), 每情景 [c,f,Q,R,E] (5*nh)`

`iA = np.arange(nh)`

`iU = np.arange(nh, 2 * nh)`

`iW = np.arange(2 * nh, 3 * nh)`

`off = 3 * nh`

`nvar = off + S * 5 * nh`

`pi = 1.0 / S`

`c = np.zeros(nvar)`

`c[iA] = 0.0`

`c[iU] = -C.DEFAULT_MULT * pr`

`c[iW] = C.EXCESS_MULT * pr`

`iv = {}`

`for s in range(S):`

`b = off + s * 5 * nh`

`iv[s] = dict(C=b + np.arange(nh), F=b + nh + np.arange(nh),`

`Q=b + 2 * nh + np.arange(nh), R=b + 3 * nh + np.arange(nh),`

`E=b + 4 * nh + np.arange(nh))`

`c[iv[s]['Q']] = pi * C.EMERG_MULT * pr`

`eq_r, eq_c, eq_v, beq = [], [], [], []`

`row = 0`

`# 违约/超额绑定: A + u - w = P (每时段, 与情景无关)`

`for t in range(nh):`

`eq_r.append(row); eq_c.append(iA[t]); eq_v.append(1.0)`

`eq_r.append(row); eq_c.append(iU[t]); eq_v.append(1.0)`

`eq_r.append(row); eq_c.append(iW[t]); eq_v.append(-1.0)`


```

244     beq.append(Pseg[t]); row += 1
245 # 每情景：能量平衡  $A + G + f + Q = D + c + R$  与储能递推
246 for s in range(S):
247     v = iv[s]
248     for t in range(nh):
249         eq_r.append(row); eq_c.append(iA[t]); eq_v.append(1.0)
250         eq_r.append(row); eq_c.append(v['C'][t]); eq_v.append(-1.0)
251         eq_r.append(row); eq_c.append(v['F'][t]); eq_v.append(C.ETA)
252         eq_r.append(row); eq_c.append(v['Q'][t]); eq_v.append(1.0)
253         eq_r.append(row); eq_c.append(v['R'][t]); eq_v.append(-1.0)
254         beq.append(scenD[s, t] - scenG[s, t]); row += 1
255     for t in range(nh):
256         eq_r.append(row); eq_c.append(v['E'][t]); eq_v.append(1.0)
257         eq_r.append(row); eq_c.append(v['C'][t]); eq_v.append(-C.ETA)
258         eq_r.append(row); eq_c.append(v['F'][t]); eq_v.append(1.0)
259         if t > 0:
260             eq_r.append(row); eq_c.append(v['E'][t - 1]); eq_v.append(-1.0)
261             beq.append(0.0)
262         else:
263             beq.append(E_start)
264     row += 1
265 A_eq = sp.csr_matrix((eq_v, (eq_r, eq_c)), shape=(row, nvar))
266
267 lb = np.zeros(nvar); ub = np.full(nvar, np.inf)
268 for s in range(S):
269     v = iv[s]
270     ub[v['C']] = C.E_CMAX; ub[v['F']] = C.E_FMAX
271     lb[v['E']] = C.SOC_MIN; ub[v['E']] = C.SOC_HIGH_BOUND
272 bounds = list(zip(lb, ub))
273
274 res = linprog(c, A_eq=A_eq, b_eq=np.array(beq), bounds=bounds, method='highs')
275 if not res.success:
276     raise RuntimeError('滚动调整情景LP失败: ' + res.message)
277 x = res.x
278 Cc = np.zeros((S, nh)); F = np.zeros((S, nh)); Qq = np.zeros((S, nh))
279 Rr = np.zeros((S, nh)); E = np.zeros((S, nh))
280 for s in range(S):
281     v = iv[s]
282     Cc[s] = x[v['C']]; F[s] = x[v['F']]; Qq[s] = x[v['Q']]
283     Rr[s] = x[v['R']]; E[s] = x[v['E']]
284 return dict(A=x[iA], u=x[iU], w=x[iW], c=Cc, f=F, Q=Qq, R=Rr, E=E, obj=res.fun)

```

2.4 B.4 情景生成 (scenarios.py)

整日成对残差 bootstrap (residual_scenarios) 与以附件 3 时点预报为基准的成对残差情景 (residual_scenarios_fc3), 生成点预报与 S 个等概率负荷/光伏情景。

```
1  # -*- coding: utf-8 -*-
2  """情景生成: 非参数 bootstrap 历史残差。返回点预报与 S 个等概率情景。"""
3  import numpy as np
4  import config as C
5
6
7  def point_forecast(hist, K=7):
8      """hist: (N,144)。前 K 天逐槽均值。"""
9      n = len(hist)
10     if n == 0:
11         return np.zeros(C.T_IN_DAY)
12     return np.mean(hist[-min(K, n):], axis=0)
13
14
15 def residual_scenarios(load_hist, pv_hist, K, S=30, seed=1):
16     """整日残差 bootstrap 情景生成 (升级版: 负荷与光伏同一天成对抽取, 替代逐槽独立抽样)。
17
18     load_hist/pv_hist: (N,144) 按日期升序的历史实际值(kWh/时段)。
19     调用方必须只传入目标日 d 之前的数据 (信息因果, 禁止含未来)。
20     K: int (负荷/光伏同一窗口) 或 (K_L, K_P) (各自窗口长度)。
21     点预报 = 各自最近 K 天逐槽均值;
22     残差池 = 池内每一天的 (实际 - 点预报) 整条曲线 (K_L=K_P 时即最近 K 天);
23     抽样 = 有放回抽 S 个整天, 负荷与光伏使用同一天索引 (成对, 保留负荷-光伏相关性);
24     情景 = 点预报 + 整日残差; 光伏情景截断到 >=0 (物理非负)。
25     返回 dict(point_ld, point_pv, scen_ld, scen_pv), scen_*(S,144)。
26     """
27     if isinstance(K, (tuple, list, np.ndarray)):
28         KL, KP = int(K[0]), int(K[1])
29     else:
30         KL = KP = int(K)
31     T = C.T_IN_DAY
32     n = len(load_hist)
33     rng = np.random.default_rng(seed)
34     point_ld = point_forecast(load_hist, KL)
35     point_pv = point_forecast(pv_hist, KP)
36     if n == 0:
37         return dict(point_ld=point_ld, point_pv=point_pv,
38                     scen_ld=np.tile(point_ld, (S, 1)),
39                     scen_pv=np.tile(point_pv, (S, 1)))
40     # 成对抽样池: 取并集窗口 (最近 max(KL,KP) 天), 同一池内日索引同时决定负荷与光伏残差
41     w = min(n, max(KL, KP))
```

```

42 resid_ld = load_hist[n - w:n] - point_ld[None, :]
43 resid_pv = pv_hist[n - w:n] - point_pv[None, :]
44 idx = rng.integers(0, w, size=S) # 池内偏移 (0=最早一天, w-1=最近一天)
45 scen_ld = point_ld[None, :] + resid_ld[idx]
46 scen_pv = point_pv[None, :] + resid_pv[idx]
47 scen_pv = np.maximum(scen_pv, 0.0)
48 return dict(point_ld=point_ld, point_pv=point_pv,
49             scen_ld=np.asarray(scen_ld, float), scen_pv=np.asarray(scen_pv, float))
50
51
52 def residual_scenarios_fc3(load_hist, pv_hist, fc3_pv_hist, fc3_pv_today, KL, S=30, seed=1):
53     """Q3 用: 以附件3 时点光伏预报为基准的成对残差 bootstrap。
54
55     load_hist/pv_hist: (N,144) 按日期升序的历史实际值(kWh/时段), 只含目标日之前数据。
56     fc3_pv_hist: (N,144) 历史每天同一 时点发布的整点预报映射向量 (与 pv_hist 逐日对齐);
57     fc3_pv_today: (144,) 目标日 时点预报向量。
58     负荷点预报 = 前 KL 天逐槽均值; 光伏点预报 = fc3_pv_today (官方预报为基准)。
59     残差池 = 最近 min(N,KL) 天: 负荷(实际-点预报)、光伏(实际-当日 预报);
60     有放回抽 S 个整天, 负荷与光伏同一天索引 (成对, 保留相关性)。
61     情景 = 点预报 + 整日残差; 光伏情景截断到 >=0。
62     返回 dict(point_ld, point_pv, scen_ld, scen_pv), scen_*(S,144)。
63     """
64     T = C.T_IN_DAY
65     n = len(load_hist)
66     rng = np.random.default_rng(seed)
67     point_ld = point_forecast(load_hist, KL)
68     point_pv = np.asarray(fc3_pv_today, float)
69     if n == 0:
70         return dict(point_ld=point_ld, point_pv=point_pv,
71                     scen_ld=np.tile(point_ld, (S, 1)),
72                     scen_pv=np.tile(point_pv, (S, 1)))
73     w = min(n, KL)
74     resid_ld = load_hist[n - w:n] - point_ld[None, :]
75     resid_pv = pv_hist[n - w:n] - fc3_pv_hist[n - w:n]
76     idx = rng.integers(0, w, size=S)
77     scen_ld = point_ld[None, :] + resid_ld[idx]
78     scen_pv = point_pv[None, :] + resid_pv[idx]
79     scen_pv = np.maximum(scen_pv, 0.0)
80     return dict(point_ld=point_ld, point_pv=point_pv,
81                 scen_ld=np.asarray(scen_ld, float), scen_pv=np.asarray(scen_pv, float))

```

2.5 B.5 问题一入口 (main_q1.py)

单日确定性完美信息; 调用 solve_day 求第 1 问计划购电、充放电与储电轨迹, 写出 result1.xlsx。

```

1  # -*- coding: utf-8 -*-
2  """问题1: 确定性 MILP/LP 日计划 (附件1 完美信息)。
3
4  储能日循环: storage 日末回到日初(6000 kWh), 保证可长期重复运行;
5  目标: 最小化当日购电费用(电价  $p$  · 购电量), 约束含 能量平衡、储能上下限、充放电功率上限、
6      光伏/负载(附件1 预报)已知。输出 result1.xlsx(两张表) + 计划图。
7  """
8  import os, sys
9  sys.path.insert(0, os.path.dirname(__file__))
10 import numpy as np
11 import matplotlib
12 matplotlib.use('Agg')
13 import config as C
14 C.setup_plot_style()
15 import matplotlib.pyplot as plt
16 import data, lp, results as R
17
18
19 def run(outfile='result1.xlsx'):
20     a1 = data.load_attach1()
21     price = a1['price']
22     rp = lp.solve_day_plan(price, a1['pv'], a1['load'], C.E_INIT, force_end=C.E_INIT)
23     P, c, f, E = rp['P'], rp['c'], rp['f'], rp['E']
24     cost = float(np.sum(price * P))
25     path = os.path.join(C.RES_DIR, outfile)
26     R.write_result1(path, P, c, f, C.E_INIT, E[-1], price)
27     print('已写出', path)
28     print(f'[Q1 确定性日计划] 购电量合计={P.sum():.2f} kWh, 购电费={cost:.2f} 元, '
29           f'0:00 储电={C.E_INIT}, 24:00 储电={E[-1]:.2f}')
30     return dict(price=price, P=P, c=c, f=f, E=E, cost=cost)
31
32
33 def plot(P, c, f, E, price, path=None):
34     """计划购电/充电/放电/储电量/电价 可视化。"""
35     x = np.arange(C.T_IN_DAY)
36     fig, ax1 = plt.subplots(figsize=(13, 5.5))
37     ax1.bar(x, P, width=0.6, alpha=0.7, label='购电量(kWh)', color='tab:blue')
38     ax1.bar(x, c, width=0.6, alpha=0.5, label='充电量(kWh)', color='tab:green')
39     ax1.bar(x, f, width=0.6, alpha=0.5, label='放电量(kWh)', color='tab:orange')
40     ax1.set_xlabel('时段(每10min)', fontsize=12); ax1.set_ylabel('能量(kWh)', fontsize=12)
41     ax1.set_xlim(0, C.T_IN_DAY)
42     ax1.set_xticks(range(0, C.T_IN_DAY + 1, 12))
43     ax1.legend(loc='upper left', fontsize=11); ax1.grid(alpha=0.3)
44     ax1.tick_params(labelsize=11)
45
46     ax2 = ax1.twinx()

```

```

47 ax2.plot(x, E, color='tab:red', lw=1.6, label='储电量(kWh)')
48 ax2.axhline(C.SOC_HIGH_BOUND, color='gray', ls=':', lw=1)
49 ax2.axhline(C.SOC_MIN, color='gray', ls=':', lw=1)
50 ax2.set_ylabel('储电量(kWh)', fontsize=12); ax2.legend(loc='upper right', fontsize=11)
51 ax2.set_ylim(0, C.SOC_MAX)
52 ax2.tick_params(labelsize=11)
53 plt.title('问题1: 单日最优运营计划 (储能日循环)', fontsize=14)
54 fig.tight_layout()
55 path = path or os.path.join(C.FIG_DIR, 'q1_plan.png')
56 fig.savefig(path, dpi=150); plt.close(fig)
57 print('图已存', path)
58
59
60 if __name__ == '__main__':
61     A = run()
62     plot(A['P'], A['c'], A['f'], A['E'], A['price'])

```

2.6 B.6 问题二入口 (main_q2.py)

按”完美信息 + 滚动”双口径评估两阶段随机 / CVaR 策略，输出 result2.xlsx 与风险敏感性结果。

```

1 # -*- coding: utf-8 -*-
2 """问题2: 两阶段随机规划 (官方口径 v2 —— 预报式, 替代原完美信息口径)。
3
4 官方结果 result2.xlsx 口径:
5     每天 0:00 依据截至前一日的历史数据 (信息因果, 不含未来) ——
6     1) 点预报: 负荷/光伏分别取各自前 K_L / K_P 天逐槽(144 时段)均值, K 由全年交叉验证选出;
7     2) 情景生成: 整日残差 bootstrap (scenarios.residual_scenarios), 负荷与光伏同一天成对,
8         S=30 个情景, 抽样种子 = 2026 + 日序 (与日期绑定、确定性、重跑逐位一致);
9         2025-01-01 无历史, 按拍板规格用附件1 典型日剖面做冷启动情景基准;
10    3) 计划 P: lp.solve_two_stage(lam=LAM_OFFICIAL, CVaR 风险权衡口径: 接受总费用略升、
11        换取紧急购电显著下降; * 由 q2_lambda_sweep.py 全年权衡曲线数据驱动选定);
12    4) 结算: 固定 P, 用当日实际负荷/光伏调 lp.solve_day_fixedP (紧急价 5p) 得当日实际
13        紧急购电 Q 与充放电 c/f、储能 E; E_end 结转次日 (跨日连续, 1200~10800 kWh)。
14    全年链自 2025-01-01 (E=6000) 逐日滚动, 仅 1 月作储能状态预热, 输出 2.1-12.31。
15
16 旧口径: perfect_info_legacy() (把当日实际负荷/光伏当作 0:00 已知 → 全年 Q0),
17 仅作对比保留, 不再写入 result2.xlsx。
18 """
19 import os, sys, time, hashlib
20 sys.path.insert(0, os.path.dirname(__file__))
21 import numpy as np
22 import pandas as pd

```

```

23 import config as C
24 import data, lp, results as R
25 import scenarios
26
27 K_CANDIDATES = [7, 15, 30] # K 候选集（数据驱动选优）
28 SEED_BASE = 2026          # 抽样种子基数：2026 + 日序
29 LEGACY_FEE_RECORD = 12260572.90 # 旧完美信息口径计划费（16.0 附件列序修复后复算值）
30 LAM_OFFICIAL = 0.5        # 官方风险权衡权重 *：由 q2_lambda_sweep.py 全年权衡曲线数据驱动选定
31                            # （=0.5 拐点：紧急购电费较 =0 下降 55.4%，合计 16,209,098.16
32                            # 为全网格最低；
33                            # 1.5 后计划费饱和不再改善。用户拍板：Q2 采用风险权衡——
34                            # 接受计划费略升、换取紧急购电显著下降；Q3/Q4 不引入 CVaR，
35                            # 理由见 main_q3.py 头部注释，论文正文点明）
36
37 def _seed_for_day(d):
38     """与日期绑定的确定性随机种子（重跑逐位一致）。"""
39     return SEED_BASE + int(d.dayofyear)
40
41
42 def cross_validate_K(dates, load, pv):
43     """交叉验证选 K（候选 {7,15,30}）。
44
45     对 2025-02-01~12-31 每天 d：用 d 前 K 天逐槽均值预报 d，计算 144 时段 RMSE，
46     除以该日实际均值归一化（当日均值 0 的天跳过），全年取平均；
47     负荷、光伏各自选出平均归一化 RMSE 最小的 K（记为 K_L、K_P）。
48     返回 (KL, KP, cv 表)。
49     """
50     start = int(np.searchsorted(dates, pd.Timestamp('2025-02-01')))
51     rows, skip = [], 0
52     for K in K_CANDIDATES:
53         nrmse_L, nrmse_P = [], []
54         for i in range(start, len(dates)):
55             lo = max(0, i - K)
56             fcL = load[lo:i].mean(axis=0) if i > lo else load[i] * 0.0
57             fcP = pv[lo:i].mean(axis=0) if i > lo else pv[i] * 0.0
58             mL = float(load[i].mean()); mP = float(pv[i].mean())
59             if mL > 1e-6:
60                 nrmse_L.append(float(np.sqrt(np.mean((fcL - load[i]) ** 2)) / mL))
61             else:
62                 skip += 1
63             if mP > 1e-6:
64                 nrmse_P.append(float(np.sqrt(np.mean((fcP - pv[i]) ** 2)) / mP))
65             else:
66                 skip += 1
67         rows.append(dict(K=K, nrmse_load=float(np.mean(nrmse_L)),
68                         nrmse_pv=float(np.mean(nrmse_P))))

```

```

69 cv = pd.DataFrame(rows)
70 KL = int(cv.loc[cv['nrmse_load'].idxmin(), 'K'])
71 KP = int(cv.loc[cv['nrmse_pv'].idxmin(), 'K'])
72 if skip:
73     print(f'[K 交叉验证] 跳过日均值 0 的天次（负荷/光伏合计）: {skip}')
74 return KL, KP, cv
75
76
77 def perfect_info_legacy(price, load, pv, dates):
78     """【旧口径，仅作对比】完美信息：把当日实际负荷/光伏当作 0:00 已知，
79     确定性 LP 计划 → 全年紧急购电 Q 0。不再用于官方 result2.xlsx。"""
80     rec = {}
81     E_start = C.E_INIT
82     for i in range(len(dates)):
83         rp = lp.solve_day_plan(price, pv[i], load[i], E_start)
84         rec[dates[i]] = dict(P=rp['P'], Q=np.zeros(C.T_IN_DAY), c=rp['c'], f=rp['f'],
85                             E_start=E_start, E_end=rp['E'][-1])
86         E_start = rp['E'][-1]
87     return rec
88
89
90 def _run_forecast_chain(price, dates, load, pv, KL, KP, S=30, lam=0.0,
91                         max_days=None, verbose=True):
92     """预报式两阶段随机规划全年滚动链（官方口径）。
93
94     每日: residual_scenarios(load[:i], pv[:i], (KL,KP), S, seed=2026+日序)
95     → solve_two_stage(lam) 得计划 P → solve_day_fixedP 按当日实际结算
96     → E_end 结转次日。断言每日储能 E 全程 [1200,10800]。
97     max_days: 仅跑前 N 天（冒烟测试用），默认全年。
98     """
99     n = len(dates) if max_days is None else int(max_days)
100     rec = {}
101     E_start = C.E_INIT
102     t0 = time.time()
103     a1 = data.load_attach1() # 冷启动先验: 2025-01-01 无历史可用
104     for i in range(n):
105         d = dates[i]
106         if i == 0:
107             # 冷启动（拍板规格）：用附件1 典型日负荷/光伏剖面作当日点预报与情景基准
108             # （无历史可抽残差，S 个情景同形）。
109             scen_ld = np.tile(a1['load'], (S, 1))
110             scen_pv = np.tile(a1['pv'], (S, 1))
111         else:
112             fc = scenarios.residual_scenarios(load[:i], pv[:i], (KL, KP), S=S,
113                                               seed=_seed_for_day(d))
114             scen_ld, scen_pv = fc['scen_ld'], fc['scen_pv']
115         two = lp.solve_two_stage(price, scen_ld, scen_pv, E_start,

```

```

116         lam=lam, beta=C.BETA)
117     settle = lp.solve_day_fixedP(price, pv[i], load[i], E_start, two['P'])
118     E_t = np.asarray(settle['E'], float)
119     assert np.all(E_t >= C.SOC_MIN - 1e-6) and np.all(E_t <= C.SOC_HIGH_BOUND + 1e-6), \
120         f'{d.date()} 储能越界: min={E_t.min():.2f} max={E_t.max():.2f}'
121     rec[d] = dict(P=two['P'], Q=settle['Q'], c=settle['c'], f=settle['f'],
122                 E_start=float(E_start), E_end=float(E_t[-1]))
123     E_start = float(E_t[-1])
124     if verbose and (i + 1) % 30 == 0:
125         el = time.time() - t0
126         print(f' 滚动链进度 {i + 1}/{n} 日 ({d.date()}) E={E_start:8.1f} kWh'
127             f' 用时 {el:6.1f}s', flush=True)
128     return rec
129
130
131 def _result_digest(price, dates_out, rec):
132     """对全部输出序列做确定性哈希, 供跨次运行逐位一致性校验。"""
133     h = hashlib.sha256()
134     for d in dates_out:
135         h.update(np.round(np.asarray(rec[d]['P'], float), 6).tobytes())
136         h.update(np.round(np.asarray(rec[d]['Q'], float), 6).tobytes())
137         h.update(np.round(np.asarray(rec[d]['c'], float), 6).tobytes())
138         h.update(np.round(np.asarray(rec[d]['f'], float), 6).tobytes())
139         h.update(np.round(np.float64(rec[d]['E_end']), 6).tobytes())
140     return h.hexdigest()
141
142
143 def run(outfile='result2.xlsx'):
144     a1 = data.load_attach1()
145     price = a1['price']
146     dates, load, pv = data.load_attach2()
147
148     # ---- 1) 交叉验证选 K (数据驱动) ----
149     KL, KP, cv = cross_validate_K(dates, load, pv)
150     print('[K 交叉验证] 全年平均归一化 RMSE (2025-02-01~12-31, 144 时段):')
151     print(cv.to_string(index=False))
152     print(f'所选: K_L = {KL}, K_P = {KP}')
153
154     # ---- 2) 预报式两阶段随机规划全年链 (1 月预热, 不输出; =* 风险权衡口径) ----
155     print(f'开始全年滚动链: 365 天 ×S={C.S_NUM} 情景两阶段 LP (lam={LAM_OFFICIAL}) .....')
156     rec = _run_forecast_chain(price, dates, load, pv, KL, KP, S=C.S_NUM, lam=LAM_OFFICIAL)
157
158     # ---- 3) 汇总与写 result2.xlsx ----
159     dates_out = [d for d in dates if d >= pd.Timestamp('2025-02-01')]
160     P = {d: rec[d]['P'] for d in dates_out}
161     Q = {d: rec[d]['Q'] for d in dates_out}
162     c = {d: rec[d]['c'] for d in dates_out}

```



```

163 f = {d: rec[d]['f'] for d in dates_out}
164 E0 = {d: rec[d]['E_start'] for d in dates_out}
165 Ee = {d: rec[d]['E_end'] for d in dates_out}
166 plan_fee = sum(float(np.sum(price * rec[d]['P'])) for d in dates_out)
167 emerg_fee = sum(float(np.sum(C.EMERG_MULT * price * rec[d]['Q'])) for d in dates_out)
168 emerg_kwh = sum(float(rec[d]['Q'].sum()) for d in dates_out)
169 total_fee = plan_fee + emerg_fee
170 assert emerg_fee > 0, '全年紧急购电费应 > 0 (旧完美信息口径为 0, 已切换预报表口径)'
171 path = os.path.join(C.RES_DIR, outfile)
172 R.write_result2(path, dates_out, P, Q, c, f, E0, Ee)
173 path = R.strip_personal_meta(path) # 清除模板带入的个人元数据 (提交合规)
174 print('已写出', path)
175 print(f'[Q2 官方·预报表] 计划购电费 = {plan_fee:,.2f} 元 | '
176       f'紧急购电费 = {emerg_fee:,.2f} 元 ({emerg_kwh:,.0f} kWh) | '
177       f'合计 = {total_fee:,.2f} 元')
178
179 # ---- 4) 指定日期 144 时段 Q 序列 (论文表3 用) ----
180 spec_q = {}
181 print('[Q2 指定日期紧急购电 (论文表3)]')
182 for ds in C.SPEC_DAYS:
183     d = pd.Timestamp(ds)
184     spec_q[d] = np.asarray(rec[d]['Q'], float)
185     if spec_q[d].sum() <= 0:
186         # 合法零值: 当日实际净需求低于全部情景 (如周末低负荷日), 计划按情景包络
187         # 多购, 缺电由计划+储能完全吸收, 表现为高弃光而非紧急购电。
188         print(f'注意: {ds} 紧急购电 = 0 kWh (合法结果: 当日实际净需求低于全部 S 个'
189              f'情景, 计划按情景包络多购, 缺电由计划+储能吸收, 弃光较高)')
190     else:
191         print(f'{ds}: 紧急购电 {spec_q[d].sum():,.1f} kWh, 紧急费 '
192              f'{float(np.sum(C.EMERG_MULT * price * spec_q[d])):,.1f} 元')
193 qtab = pd.DataFrame({'时间段': C.INTERVAL_LABELS})
194 for ds in C.SPEC_DAYS:
195     qtab[ds] = np.round(spec_q[pd.Timestamp(ds)], 4)
196 qtab.to_csv(os.path.join(C.RES_DIR, 'q2_specdays_Q.csv'),
197             index=False, encoding='utf-8-sig')
198
199 # ---- 5) 摘要 CSV ----
200 summ_rows = [
201     ('全年计划购电费_元', round(plan_fee, 2)),
202     ('全年紧急购电费_元', round(emerg_fee, 2)),
203     ('合计_元', round(total_fee, 2)),
204     ('全年紧急购电量_kWh', round(emerg_kwh, 2)),
205     ('K_L', KL), ('K_P', KP),
206 ]
207 for ds in C.SPEC_DAYS:
208     d = pd.Timestamp(ds)
209     summ_rows.append((f'{ds}_紧急购电量_kWh', round(float(spec_q[d].sum()), 4)))

```

```

210     summ_rows.append((f'{ds}_紧急购电费_元',
211                      round(float(np.sum(C.EMERG_MULT * price * spec_q[d])), 4)))
212 summ = pd.DataFrame(summ_rows, columns=['指标', '数值'])
213 summ.to_csv(os.path.join(C.RES_DIR, 'q2_new_summary.csv'),
214             index=False, encoding='utf-8-sig')
215 cv.to_csv(os.path.join(C.RES_DIR, 'q2_K_cv.csv'), index=False, encoding='utf-8-sig')
216
217 # ---- 6) 重跑一致性校验 (seed 固定 → 逐位一致) ----
218 digest_path = os.path.join(C.RES_DIR, 'q2_digest.txt')
219 dg = _result_digest(price, dates_out, rec)
220 if os.path.exists(digest_path):
221     old = open(digest_path, encoding='utf-8').read().strip()
222     assert old == dg, f'重跑结果与上次不一致! 旧 digest={old[:16]}... 新 digest={dg[:16]}...'
223     print(f'[重跑一致性] digest 校验通过: {dg[:16]}...')
224 else:
225     with open(digest_path, 'w', encoding='utf-8') as fp:
226         fp.write(dg)
227     print(f'[重跑一致性] 首次运行, 已记录 digest: {dg[:16]}...')
228
229 # ---- 7) 与旧完美信息口径对比 (仅对比, 不写结果) ----
230 leg = perfect_info_legacy(price, load, pv, dates)
231 leg_fee = sum(float(np.sum(price * leg[d]['P']))) for d in dates_out)
232 diff = plan_fee - leg_fee
233 print('\n[新旧口径对比]')
234 print(f'旧完美信息计划费 = {leg_fee:,.2f} 元'
235       f' (历史记录 {LEGACY_FEE_RECORD:,.2f} 元, '
236       f'{"复算吻合" if abs(leg_fee - LEGACY_FEE_RECORD) < 1.0 else "复算不一致, 请检查"})')
237 print(f'新预报表式计划费 = {plan_fee:,.2f} 元, 差 = {diff:+,.2f} 元')
238 if diff > 0:
239     print('解释: 完美信息口径下计划即当日实际最优购电 (全年紧急购电恒为 0), 是计划费的'
240           '下界; 预报表式口径下计划基于历史均值与整日残差情景做期望最小化, 为对冲负荷/光伏'
241           '不确定性会在低价时段多购电充当“保险”, 把缺电风险尽量转移到 1 倍计划电价而非 '
242           '5 倍紧急电价, 因此计划费略高 (保险性多购), 多出的计划费换来的是把实际缺电'
243           f'风险真实计量为全年 {emerg_fee:,.0f} 元紧急购电费 (旧口径 Q0, 该风险成本'
244           '为零计量)。')
245 else:
246     print('解释: 预报表式口径计划费不高于完美信息口径, 说明历史均值预报在低价时段购电更'
247           '克制; 完美信息计划因精确贴合当日曲线而把更多购电安排在低价时段, 两者均无紧急'
248           '购电的完美信息是理论下界, 预报表式口径的全部费用 = 计划费 + 紧急费才是真实可'
249           '支付成本。')
250 print(f'[Q2 完成] 全年合计费用 (计划+紧急) = {total_fee:,.2f} 元')
251 return dict(price=price, dates=dates, load=load, pv=pv, rec=rec,
252            dates_out=dates_out, KL=KL, KP=KP, plan_fee=plan_fee,
253            emerg_fee=emerg_fee, total_fee=total_fee, leg_fee=leg_fee)
254
255
256 def method_analysis(price, dates, load, pv, rec, KL, KP):

```

```

257 """ (CVaR 权重)敏感性 (论文用, 不计入 result2.xlsx) 。
258
259 4 个指定日期 × {0,0.5,1,2,5}: 新情景生成器 ((K_L,K_P) 整日成对残差 bootstrap,
260 seed=2026+日序, 与官方链同源) → solve_two_stage(lam) → 当日实际结算。
261 输出 plan_fee / emerg_fee / total / emerg_kwh 表 (CSV) 并画 3.20 权衡图。
262 """
263 rows = []
264 for dd in C.SPEC_DAYS:
265     d = pd.Timestamp(dd)
266     i = int(np.where(dates == d)[0][0])
267     E_start = rec[d]['E_start']
268     fc = scenarios.residual_scenarios(load[:i], pv[:i], (KL, KP), S=C.S_NUM,
269                                     seed=_seed_for_day(d))
270     for lam in sorted(set([0.0, 0.5, 1.0, 2.0, 5.0, LAM_OFFICIAL])):
271         two = lp.solve_two_stage(price, fc['scen_ld'], fc['scen_pv'], E_start,
272                                 lam=lam, beta=C.BETA)
273         settle = lp.solve_day_fixedP(price, pv[i], load[i], E_start, two['P'])
274         plan_fee = float(np.sum(price * two['P']))
275         emerg_fee = float(np.sum(C.EMERG_MULT * price * settle['Q']))
276         rows.append(dict(day=dd, lam=lam, plan_fee=plan_fee, emerg_fee=emerg_fee,
277                         total=plan_fee + emerg_fee, emerg_kwh=float(settle['Q'].sum())))
278 df = pd.DataFrame(rows)
279 # 自检: =* 行应与官方链当日结算一致 (同种子、同情景、同 E_start)
280 for dd in C.SPEC_DAYS:
281     d = pd.Timestamp(dd)
282     r0 = df[(df.day == dd) & (df.lam == LAM_OFFICIAL)].iloc[0]
283     chain_plan = float(np.sum(price * rec[d]['P']))
284     chain_emer = float(np.sum(C.EMERG_MULT * price * rec[d]['Q']))
285     assert abs(r0.plan_fee - chain_plan) < 1e-3, f'{dd} = {LAM_OFFICIAL} 计划费与官方链不一致'
286     assert abs(r0.emerg_fee - chain_emer) < 1e-3, f'{dd} = {LAM_OFFICIAL}
287         紧急费与官方链不一致'
287 print('\nQ2 敏感性 • 两阶段随机+CVaR (新情景生成器)')
288 print(df.to_string(index=False))
289 df.to_csv(os.path.join(C.RES_DIR, 'q2_two_stage_comparison.csv'),
290           index=False, encoding='utf-8-sig')
291
292 import matplotlib
293 matplotlib.use('Agg')
294 C.setup_plot_style()
295 import matplotlib.pyplot as plt
296 df20 = df[df.day == C.SPEC_DAYS[0]].sort_values('lam')
297 fig, ax = plt.subplots(figsize=(8, 5))
298 ax.plot(df20.lam, df20.plan_fee, '-o', label='计划购电费')
299 ax.plot(df20.lam, df20.emerg_fee, '-s', label='紧急购电费')
300 ax.plot(df20.lam, df20.total, '--^', label='合计')
301 ax.set_xlabel('CVaR 权重'); ax.set_ylabel('费用(元)')
302 ax.set_title(f'问题2 两阶段随机+CVaR: {C.SPEC_DAYS[0]} 风险-费用权衡')

```

```

303     ax.legend(); ax.grid(alpha=0.4)
304     fig.tight_layout(); fig.savefig(os.path.join(C.FIG_DIR, 'q2_cvar_tradeoff.png'), dpi=150)
305     plt.close(fig)
306     print('已写 report/q2_two_stage_comparison.csv 与 figures/q2_cvar_tradeoff.png')
307
308
309 if __name__ == '__main__':
310     A = run()
311     method_analysis(A['price'], A['dates'], A['load'], A['pv'], A['rec'], A['KL'], A['KP'])

```

2.7 B.7 问题三入口 (main_q3.py)

滚动随机模型预测控制：0:00 计划、6/12/18 调整、实际光伏结算与违约/超额惩罚，写出 result3.xlsx。

```

1  # -*- coding: utf-8 -*-
2  """问题3：滚动随机 MPC (v2 —— 因果滚动 + 情景鲁棒 + 负荷预报化)。
3
4  相对旧版的口径修正：
5  1) 负荷预报化：0/6/12/18 决策只用历史信息（负荷点预报 = 前 KL 天逐槽均值 + 成对残差情景），
6     不再把当日实际负荷当作已知（与 Q2 预报式口径统一）；
7  2) 光伏情景化：以附件3 各时点官方预报为基准，叠加"实际-当日同点时点预报"的历史整日残差
8     bootstrap (S=30, 负荷与光伏同一天成对抽取)，调整决策对情景做期望最小化（随机 MPC 名副其实）；
9  3) SOC 实际轨迹：每个调整时点，用"已执行段 [0,t0) 固定购电 + 实际光伏/负荷"因果重放，
10     得到真实 SOC 初值，不再沿用 0:00 计划轨迹；
11  4) 调整目标计费口径修正： $\Sigma_t [p \cdot A + 0.5p \cdot u + 1.5p \cdot w]$  且  $A+u-w=P$  精确绑定违约/超额量
12     （旧版 LP 目标把 u 推到上界 P、调整购电 A 退化免费，已修正）；
13  5) 结算口径与 Q2 一致：固定最终调整购电 A，对当日实际光伏/负荷重放电池调度，
14     紧急购电 Q 按 5 倍价吸收偏差；E_end 结转次日（跨日连续 + 1 月预热）。
15
16  风险口径：Q3 不引入 CVaR 风险权重 (=0)。理由：0/6/12/18 四次预报更新已把不确定性
17  逐次吸收，滚动再计划本身即风险对冲手段；CVaR 权重留作 Q2 单次决策的风险权衡工具
18  （论文正文点明该设计）。
19  """
20  import os, sys
21  sys.path.insert(0, os.path.dirname(__file__))
22  import numpy as np
23  import pandas as pd
24  import config as C
25  import data, lp, results as R
26  import scenarios
27  C.setup_plot_style()
28  import matplotlib
29  matplotlib.use('Agg')

```

```

30 import matplotlib.pyplot as plt
31
32 TAU = [0, 6, 12, 18]
33 KL_Q3 = 15          # 负荷点预报/残差池窗口（与 Q2 的 K_L 一致）
34 S_Q3 = 30           # 情景数（与 Q2 一致）
35 SEED_BASE = 2026    # 抽样种子基数：2026 + 日序（确定性、重跑逐位一致）
36
37
38 def _seed_for_day(d):
39     return SEED_BASE + int(d.dayofyear)
40
41
42 def build_fc3_hist(fc3, dates):
43     """预计算历史 时点光伏预报矩阵 {tau: (N,144)}, 避免逐日重复插值。"""
44     return {tau: np.stack([data.pv_forecast_kwh(fc3, dd, tau)[0] for dd in dates])
45             for tau in TAU}
46
47
48 def _adjust(price, P, A, Q):
49     """结算费用口径:  $p \cdot \min(P, A) + 0.5p(P-A)^+ + 1.5p(A-P)^+ + 5p \cdot Q$ 。"""
50     u = np.maximum(P - A, 0.0)
51     w = np.maximum(A - P, 0.0)
52     grid_fee = float(np.sum(price * np.minimum(P, A) + C.DEFAULT_MULT * price * u +
53                             C.EXCESS_MULT * price * w))
54     emerg_fee = float(np.sum(C.EMERG_MULT * price * Q))
55     return grid_fee, emerg_fee
56
57 def _rolling(price, load, pv, dates, fc3, i, E_start, a1, FC3_HIST,
58             KL=KL_Q3, S=S_Q3):
59     """单日因果滚动随机 MPC。返回 dict(P,A,Q,c,f,grid_fee,emerg_fee,E_end)。"""
60     d = dates[i]
61     seed = _seed_for_day(d)
62     # ---- 0:00 计划：两阶段随机（负荷历史残差情景 + 光伏附件3 0:00预报残差情景）----
63     if i == 0:
64         # 冷启动：无历史可抽残差，用附件1 典型日剖面作负荷情景基准、0:00 预报作光伏基准
65         scen_ld = np.tile(a1['load'], (S, 1))
66         scen_pv = np.tile(FC3_HIST[0][0], (S, 1))
67     else:
68         fc = scenarios.residual_scenarios_fc3(load[:i], pv[:i], FC3_HIST[0][:i],
69                                             FC3_HIST[0][i], KL, S=S, seed=seed)
70         scen_ld, scen_pv = fc['scen_ld'], fc['scen_pv']
71     plan = lp.solve_two_stage(price, scen_ld, scen_pv, E_start, lam=0.0, beta=C.BETA)
72     P = plan['P']
73     A = P.copy()
74     # ---- 6/12/18 滚动调整 ----
75     for tau in TAU[1:]:

```

```

76     t0 = tau * 6
77     # 实际 SOC: 已执行段 [0,t0) 用实际光伏/负荷, 未来段 [t0,T) 用 时点点预报
78     # (电池对剩余时段有期望需求, 保留"为晚间高峰充电"的动机; 未来置 0 会导致电池躺平、
79     # SOC 恒为下限, 调整购电被迫超买)
80     G_mix = pv[i].copy(); G_mix[t0:] = FC3_HIST[tau][i][t0:]
81     D_hat = scenarios.point_forecast(load[:,i], KL) if i > 0 else a1['load']
82     D_mix = load[i].copy(); D_mix[t0:] = D_hat[t0:]
83     rep = lp.solve_day_fixedP(price, G_mix, D_mix, E_start, A)
84     E_tau = float(rep['E'][t0 - 1])
85     # 时点情景 (截取 [t0,144))
86     if i == 0:
87         scenD = np.tile(a1['load'][t0:], (S, 1))
88         scenG = np.tile(FC3_HIST[tau][0][t0:], (S, 1))
89     else:
90         fct = scenarios.residual_scenarios_fc3(load[:,i], pv[:,i], FC3_HIST[tau][:i],
91                                                FC3_HIST[tau][i], KL, S=S, seed=seed)
92         scenD = fct['scen_ld'][:, t0:]
93         scenG = fct['scen_pv'][:, t0:]
94         rp = lp.solve_adjust_scen(price, scenD, scenG, E_tau, P, t0)
95         A[t0:] = rp['A']
96     # ---- 日终结算: 固定 A, 实际光伏/负荷重放电池 (电池日内实时调度口径, 同 Q2) ----
97     settle = lp.solve_day_fixedP(price, pv[i], load[i], E_start, A)
98     grid_fee, emerg_fee = _adjust(price, P, A, settle['Q'])
99     return dict(P=P, A=A, Q=settle['Q'], c=settle['c'], f=settle['f'],
100               grid_fee=grid_fee, emerg_fee=emerg_fee, E_end=settle['E'][-1])
101
102
103 def run(outfile='result3.xlsx'):
104     a1 = data.load_attach1()
105     price = a1['price']
106     dates, load, pv = data.load_attach2()
107     fc3 = data.load_attach3()
108     FC3_HIST = build_fc3_hist(fc3, dates)
109     rec = {}
110     E_start = C.E_INIT
111     for i in range(len(dates)):
112         res = _rolling(price, load, pv, dates, fc3, i, E_start, a1, FC3_HIST)
113         res['E_start'] = E_start
114         rec[dates[i]] = res
115         E_start = res['E_end']
116         if (i + 1) % 30 == 0:
117             print(f' 滚动进度 {i+1}/{len(dates)} 日 ({dates[i].date()}) E={E_start:8.1f} kWh',
118                   flush=True)
119
120     dates_out = [d for d in dates if d >= pd.Timestamp('2025-02-01')]
121     P = {d: rec[d]['P'] for d in dates_out}
122     A = {d: rec[d]['A'] for d in dates_out}

```

```

122 Q = {d: rec[d]['Q'] for d in dates_out}
123 c = {d: rec[d]['c'] for d in dates_out}
124 f = {d: rec[d]['f'] for d in dates_out}
125 E0 = {d: rec[d]['E_start'] for d in dates_out}
126 Ee = {d: rec[d]['E_end'] for d in dates_out}
127 path = os.path.join(C.RES_DIR, outfile)
128 R.write_result3(path, dates_out, P, A, Q, c, f, E0, Ee)
129 path = R.strip_personal_meta(path)
130 tot_grid = sum(rec[d]['grid_fee'] for d in dates_out)
131 tot_emerg = sum(rec[d]['emerg_fee'] for d in dates_out)
132 print('已写出', path)
133 print(f'[Q3 v2 滚动随机MPC] 计划+调整电网费={tot_grid:.2f} 元, 紧急购电费={tot_emerg:.2f} 元,
      合计={tot_grid + tot_emerg:.2f} 元')
134 return dict(price=price, dates=dates, load=load, pv=pv, fc3=fc3, rec=rec,
135             dates_out=dates_out, a1=a1, FC3_HIST=FC3_HIST)
136
137
138 def analysis(price, dates, load, pv, fc3, rec, a1, FC3_HIST):
139     """是否需要其他时刻预报: 比较 仅0:00(不调整) vs 0/6/12/18 滚动。"""
140     rows = []
141     for dd in C.SPEC_DAYS:
142         d = pd.Timestamp(dd)
143         i = int(np.where(dates == d)[0][0])
144         E_start = rec[d]['E_start']
145         seed = _seed_for_day(d)
146         # 仅 0:00: 两阶段随机计划 (与滚动链同口径、同种子), 固定 P 结算
147         if i == 0:
148             scen_ld = np.tile(a1['load'], (S_Q3, 1))
149             scen_pv = np.tile(FC3_HIST[0][0], (S_Q3, 1))
150         else:
151             fc = scenarios.residual_scenarios_fc3(load[:i], pv[:i], FC3_HIST[0][:i],
152                                                  FC3_HIST[0][i], KL_Q3, S=S_Q3, seed=seed)
153             scen_ld, scen_pv = fc['scen_ld'], fc['scen_pv']
154         plan = lp.solve_two_stage(price, scen_ld, scen_pv, E_start, lam=0.0, beta=C.BETA)
155         st = lp.solve_day_fixedP(price, pv[i], load[i], E_start, plan['P'])
156         g0, e0 = _adjust(price, plan['P'], plan['P'], st['Q'])
157         r = _rolling(price, load, pv, dates, fc3, i, E_start, a1, FC3_HIST)
158         rows.append(dict(day=dd, only0_grid=g0, only0_emerg=e0, only0_total=g0 + e0,
159                        roll_grid=r['grid_fee'], roll_emerg=r['emerg_fee'],
160                        roll_total=r['grid_fee'] + r['emerg_fee']))
161     df = pd.DataFrame(rows)
162     print('\n[Q3 调整时点分析]')
163     print(df.to_string(index=False))
164     df.to_csv(os.path.join(C.RES_DIR, 'q3_rolling_comparison.csv'), index=False,
165              encoding='utf-8-sig')
166     x = np.arange(len(df))
167     w = 0.32

```

```

167     fig, ax = plt.subplots(figsize=(9, 5))
168     ax.bar(x - w / 2, df.only0_total, w, label='仅0:00(不调整)')
169     ax.bar(x + w / 2, df.roll_total, w, label='0/6/12/18 滚动')
170     ax.set_xticks(x); ax.set_xticklabels(df.day, fontsize=11)
171     ax.set_ylabel('总购电费用(元)', fontsize=12); ax.set_title('问题3: 是否引入多时刻预报',
172                                     fontsize=13)
173     ax.legend(fontsize=11); ax.grid(alpha=0.4, axis='y')
174     ax.tick_params(labelsize=11)
175     fig.tight_layout(); fig.savefig(os.path.join(C.FIG_DIR, 'q3_forecast_time.png'), dpi=150);
176     plt.close(fig)
177
178 if __name__ == '__main__':
179     A = run()
180     analysis(A['price'], A['dates'], A['load'], A['pv'], A['fc3'], A['rec'], A['a1'],
181             A['FC3_HIST'])

```

2.8 B.8 问题四入口 (main_q4.py)

基于附件4波动电价重算问题二、三，输出 result4-2.xlsx / result4-3.xlsx 并做季节对比。

```

1  # -*- coding: utf-8 -*-
2  """问题4: 波动电价(附件4)重算 问题2/问题3 → result4-2.xlsx / result4-3.xlsx + 对比图。
3
4  口径沿用 (与 15.0 之后的 Q2/Q3 v2 完全一致):
5      Q4-2 = 问题2 预报式两阶段随机规划 (K_L=15/K_P=7, 整日成对残差 bootstrap, S=30,
6              风险权衡 = * 与 Q2 官方一致), 仅把固定电价(附件1)换成当日波动电价(附件4);
7      Q4-3 = 问题3 因果滚动随机 MPC (负荷预报化 + 附件3 预报情景 + SOC 实际轨迹 + 情景鲁棒调整),
8              按当日波动电价结算, =0 (滚动预报更新本身对冲不确定性, 同 Q3)。
9  附件2(负载/光伏实际)与附件4 全年日期必须逐日对齐。
10 """
11 import os, sys
12 sys.path.insert(0, os.path.dirname(__file__))
13 import numpy as np
14 import pandas as pd
15 import config as C
16 import data, lp, results as R
17 import scenarios
18 import main_q2
19 import main_q3
20 C.setup_plot_style()
21 import matplotlib
22 matplotlib.use('Agg')

```



```

23 import matplotlib.pyplot as plt
24
25 Q2_LAM = main_q2.LAM_OFFICIAL # 与 Q2 官方链同一风险权重 (由 q2_lambda_sweep 数据驱动选定)
26
27
28 # ----- Q4-2: 波动电价 + 预报式两阶段随机 (同 Q2 新口径) -----
29 def run_q42():
30     dates4, price4 = data.load_attach4() # price4[365,144]
31     dates, load, pv = data.load_attach2()
32     data.assert_aligned(dates4, dates, '附件4', '附件2')
33     a1 = data.load_attach1()
34     S = C.S_NUM
35     KL, KP, _ = main_q2.cross_validate_K(dates, load, pv) # 与 Q2 官方链同源的 K
36     rec = {}
37     E_start = C.E_INIT
38     for i in range(len(dates)):
39         d = dates[i]
40         if i == 0:
41             scen_ld = np.tile(a1['load'], (S, 1))
42             scen_pv = np.tile(a1['pv'], (S, 1))
43         else:
44             fc = scenarios.residual_scenarios(load[:i], pv[:i], (KL, KP),
45                                               S=S, seed=main_q2._seed_for_day(d))
46             scen_ld, scen_pv = fc['scen_ld'], fc['scen_pv']
47             two = lp.solve_two_stage(price4[i], scen_ld, scen_pv, E_start,
48                                     lam=Q2_LAM, beta=C.BETA)
49             settle = lp.solve_day_fixedP(price4[i], pv[i], load[i], E_start, two['P'])
50             E_t = np.asarray(settle['E'], float)
51             assert np.all(E_t >= C.SOC_MIN - 1e-6) and np.all(E_t <= C.SOC_HIGH_BOUND + 1e-6), \
52                 f'{d.date()} 储能越界'
53             rec[d] = dict(P=two['P'], Q=settle['Q'], c=settle['c'], f=settle['f'],
54                           E_start=float(E_start), E_end=float(E_t[-1]))
55             E_start = float(E_t[-1])
56             if (i + 1) % 30 == 0:
57                 print(f' Q4-2 进度 {i+1}/{len(dates)} 日 ({d.date()}) E={E_start:8.1f} kWh',
58                       flush=True)
59
60     dates_out = [d for d in dates if d >= pd.Timestamp('2025-02-01')]
61     P = {d: rec[d]['P'] for d in dates_out}
62     Q = {d: rec[d]['Q'] for d in dates_out}
63     c = {d: rec[d]['c'] for d in dates_out}
64     f = {d: rec[d]['f'] for d in dates_out}
65     E0 = {d: rec[d]['E_start'] for d in dates_out}
66     Ee = {d: rec[d]['E_end'] for d in dates_out}
67     path = os.path.join(C.RES_DIR, 'result4-2.xlsx')
68     R.write_result2(path, dates_out, P, Q, c, f, E0, Ee, tpl_name='result4-2.xlsx')
69     path = R.strip_personal_meta(path)
70     idx_of = {d: i for i, d in enumerate(dates)}

```

```

69 plan_fee = sum(float(np.sum(price4[idx_of[d]] * rec[d]['P'])) for d in dates_out)
70 emerg_fee = sum(float(np.sum(C.EMERG_MULT * price4[idx_of[d]] * rec[d]['Q'])) for d in
    dates_out)
71 emerg_kwh = sum(float(rec[d]['Q'].sum()) for d in dates_out)
72 print('已写出', path)
73 print(f'[Q4-2 波动电价+预报式两阶段随机]={Q2_LAM}] 计划费={plan_fee:,.2f} 元, '
74       f'紧急费={emerg_fee:,.2f} 元 ({emerg_kwh:,.0f} kWh), 合计={plan_fee + emerg_fee:,.2f}
    元')
75 return dict(dates=dates, load=load, pv=pv, price4=price4, rec=rec,
76            dates_out=dates_out, plan_fee=plan_fee, emerg_fee=emerg_fee)
77
78
79 # ----- Q4-3: 波动电价 + 因果滚动随机 MPC (复用 main_q3) -----
80 def run_q43():
81     dates4, price4 = data.load_attach4()
82     dates, load, pv = data.load_attach2()
83     fc3 = data.load_attach3()
84     data.assert_aligned(dates4, dates, '附件4', '附件2')
85     a1 = data.load_attach1()
86     FC3_HIST = main_q3.build_fc3_hist(fc3, dates)
87     rec = {}
88     E_start = C.E_INIT
89     for i in range(len(dates)):
90         res = main_q3.rolling(price4[i], load, pv, dates, fc3, i, E_start, a1, FC3_HIST)
91         res['E_start'] = E_start
92         rec[dates[i]] = res
93         E_start = res['E_end']
94         if (i + 1) % 30 == 0:
95             print(f' Q4-3 进度 {i+1}/{len(dates)} 日 ({dates[i].date()}) E={E_start:8.1f} kWh',
100                  flush=True)
101
102     dates_out = [d for d in dates if d >= pd.Timestamp('2025-02-01')]
103     P = {d: rec[d]['P'] for d in dates_out}
104     A = {d: rec[d]['A'] for d in dates_out}
105     Q = {d: rec[d]['Q'] for d in dates_out}
106     c = {d: rec[d]['c'] for d in dates_out}
107     f = {d: rec[d]['f'] for d in dates_out}
108     E0 = {d: rec[d]['E_start'] for d in dates_out}
109     Ee = {d: rec[d]['E_end'] for d in dates_out}
110     path = os.path.join(C.RES_DIR, 'result4-3.xlsx')
111     R.write_result3(path, dates_out, P, A, Q, c, f, E0, Ee, tpl_name='result4-3.xlsx')
112     path = R.strip_personal_meta(path)
113     tot_grid = sum(rec[d]['grid_fee'] for d in dates_out)
114     tot_emerg = sum(rec[d]['emerg_fee'] for d in dates_out)
115     print('已写出', path)
116     print(f'[Q4-3 波动电价+因果滚动随机MPC] 电网费={tot_grid:,.2f} 紧急费={tot_emerg:,.2f} '
117           f'合计={tot_grid + tot_emerg:,.2f} 元')
118     return dict(dates=dates, dates_out=dates_out, rec=rec,

```

```

113         tot_grid=tot_grid, tot_emerg=tot_emerg)
114
115
116 def shell():
117     q42 = run_q42()
118     q43 = run_q43()
119     dates4, price4 = data.load_attach4()
120     mean_p = price4.mean(axis=1)
121     x = np.arange(len(mean_p))
122     fig, ax = plt.subplots(figsize=(10, 4.5))
123     ax.plot(x, mean_p, lw=1.2, label='附件4 波动电价(日均)')
124     ax.axhline(C.EMERG_MULT * np.median(mean_p), color='r', ls='--', lw=1,
125               label='固定电价(附件1,均值省略)')
126     ax.set_xlabel('日期索引(1.1起)', fontsize=12); ax.set_ylabel('平均电价(元/kWh)', fontsize=12)
127     ax.set_title('问题4: 波动电价(附件4)与求解区间示意', fontsize=13)
128     ax.legend(fontsize=11); ax.grid(alpha=0.3)
129     ax.tick_params(labelsize=11)
130     fig.tight_layout(); fig.savefig(os.path.join(C.FIG_DIR, 'q4_price.png'), dpi=150);
131     plt.close(fig)
132     print('图已存', os.path.join(C.FIG_DIR, 'q4_price.png'))
133
134 if __name__ == '__main__':
135     shell()

```

2.9 B.9 结果模板回填 (results.py)

按附件 5 模板结构用 openpyxl 将计划/调整购电、充放电、紧急购电与储电量写入结果 Excel。

```

1  # -*- coding: utf-8 -*-
2  """结果写入：按 附件5 模板结构用 openpyxl 填值。策略：
3  - 计划购电量/调整购电量：模板已预置行(2.1-12.31 共334天)，就地填 144 列。
4  - 充放电量/紧急购电量：模板仅示例，按天数扩展行。
5  """
6  import os
7  import numpy as np
8  import openpyxl
9  import config as C
10
11
12 def _save(wb, path):
13     try:
14         wb.save(path); return path

```

```

15 except PermissionError:
16     alt = os.path.join(os.path.dirname(path),
17                         os.path.splitext(os.path.basename(path))[0] + '_new.xlsx')
18     wb.save(alt)
19     print('[警告] 目标被占用, 已写副本:', alt)
20     return alt
21
22
23 def _fill_plan_sheet(ws, data):
24     """data: {datetime/date日期: arr[144]}, 就地填 计划/调整购电量。"""
25     lookup = {}
26     for k, arr in data.items():
27         key = k.date() if hasattr(k, 'date') else k
28         lookup[key] = arr
29     for row in range(2, ws.max_row + 1):
30         dt = ws.cell(row=row, column=1).value
31         if dt is None or isinstance(dt, str):
32             continue
33         key = dt.date() if hasattr(dt, 'date') else dt
34         arr = lookup.get(key)
35         if arr is None:
36             continue
37         for t in range(C.T_IN_DAY):
38             ws.cell(row=row, column=2 + t, value=round(float(arr[t]), 4))
39
40
41 def _clear_data_rows(ws, keep_first_col=True):
42     """清空模板中残留的数据(第2行起)。
43     keep_first_col=True(默认): 保留第1列(时间段标签), 只清数据列。"""
44     for row in range(2, ws.max_row + 1):
45         start = 2 if keep_first_col else 1
46         for col in range(start, ws.max_column + 1):
47             ws.cell(row=row, column=col).value = None
48
49
50 def _fill_chongfang(ws, dates_out, c_all, f_all, EO_all, Eend_all):
51     """充放电量: 日期|时间段|充电量|放电量|时刻|储电量。每日期 6块+0:00+24:00=8行。"""
52     from datetime import time as tm
53     _clear_data_rows(ws, keep_first_col=False)
54     r = 2
55     for d in dates_out:
56         for h0, lab in C.BLOCK4:
57             lo = h0 * 6; hi = lo + 24
58             ws.cell(row=r, column=1, value=str(d))
59             ws.cell(row=r, column=2, value=lab)
60             ws.cell(row=r, column=3, value=round(float(np.sum(c_all[d][lo:hi])), 4))
61             ws.cell(row=r, column=4, value=round(float(np.sum(f_all[d][lo:hi])), 4))

```

```

62         r += 1
63         ws.cell(row=r, column=1, value=str(d)); ws.cell(row=r, column=5, value='0:00')
64         ws.cell(row=r, column=6, value=round(float(E0_all[d]), 4)); r += 1
65         ws.cell(row=r, column=1, value=str(d)); ws.cell(row=r, column=5, value='24:00')
66         ws.cell(row=r, column=6, value=round(float(Eend_all[d]), 4)); r += 1
67
68
69 def _fill_jinji(ws, dates_out, Q_all):
70     _clear_data_rows(ws, keep_first_col=False)
71     r = 2
72     for d in dates_out:
73         for t in range(C.T_IN_DAY):
74             q = Q_all[d][t]
75             if q > 1e-6:
76                 ws.cell(row=r, column=1, value=str(d))
77                 ws.cell(row=r, column=2, value=C.INTERVAL_LABELS[t])
78                 ws.cell(row=r, column=3, value=round(float(q), 4))
79                 r += 1
80
81
82 def write_result1(path, P, c, f, E_start, E_end, price):
83     """问题1 result1.xlsx —— 共需填 2 张表。
84
85     表1【计划购电量】：模板已预置 144 行时间段(A列2-145)，就地填 B 列购电量。
86     表2【充放电量】：模板 7 行(2-7块 + 0:00/24:00 储电)，就地填 充电量/放电量(B/C列
87         2-7行) 与 储电量(E列, 0:00在2行、24:00在3行)。
88
89     """
90     tpl = os.path.join(C.TPL_DIR, 'result1.xlsx')
91     wb = openpyxl.load_workbook(tpl)
92
93     # 表1 计划购电量: B 列(2-145) 填购电量, A 列时间段保留
94     ws = wb['计划购电量']
95     for t in range(C.T_IN_DAY):
96         ws.cell(row=2 + t, column=2, value=round(float(P[t]), 4))
97
98     # 表2 充放电量: 就地填 6 块充电/放电量 + 0:00/24:00 储电量
99     ws = wb['充放电量']
100     for h0, lab in C.BLOCK4:
101         lo = h0 * 6; hi = lo + 24
102         r = 2 + h0 // 4 # 块索引 0..5 -> 行 2..7
103         ws.cell(row=r, column=2, value=round(float(np.sum(c[lo:hi])), 4))
104         ws.cell(row=r, column=3, value=round(float(np.sum(f[lo:hi])), 4))
105         ws.cell(row=2, column=4, value='0:00'); ws.cell(row=2, column=5,
106             value=round(float(E_start), 4))
107         ws.cell(row=3, column=4, value='24:00'); ws.cell(row=3, column=5, value=round(float(E_end),
108             4))
109     wb._template = None

```

```

107     return _save(wb, path)
108
109
110 def write_result2(path, dates_out, P, Q, c, f, E0, Eend, tpl_name='result2.xlsx'):
111     """问题2/问题4-2。dates_out: 日期列表(2.1-12.31)。各 dict: date->arr/标量。"""
112     tpl = os.path.join(C.TPL_DIR, tpl_name)
113     wb = openpyxl.load_workbook(tpl)
114     _fill_plan_sheet(wb['计划购电量'], P)
115     _fill_jinji(wb['紧急购电量'], dates_out, Q)
116     _fill_chongfang(wb['充放电量'], dates_out, c, f, E0, Eend)
117     wb._template = None
118     return _save(wb, path)
119
120
121 def write_result3(path, dates_out, P, A, Q, c, f, E0, Eend, tpl_name='result3.xlsx'):
122     """问题3/问题4-3。"""
123     tpl = os.path.join(C.TPL_DIR, tpl_name)
124     wb = openpyxl.load_workbook(tpl)
125     _fill_plan_sheet(wb['计划购电量'], P)
126     _fill_plan_sheet(wb['调整购电量'], A)
127     _fill_jinji(wb['紧急购电量'], dates_out, Q)
128     _fill_chongfang(wb['充放电量'], dates_out, c, f, E0, Eend)
129     wb._template = None
130     return _save(wb, path)
131
132
133 def strip_personal_meta(path):
134     """【新增】清除结果文件的文档属性中的个人元数据（creator/lastModifiedBy），
135     避免提交文件泄露个人信息（模板作者字段曾为个人名）。不动其他内容。"""
136     wb = openpyxl.load_workbook(path)
137     wb.properties.creator = ''
138     wb.properties.lastModifiedBy = ''
139     try:
140         wb.save(path)
141     except PermissionError:
142         alt = os.path.join(os.path.dirname(path),
143                             os.path.splitext(os.path.basename(path))[0] + '_new.xlsx')
144         wb.save(alt)
145         print(['警告'] 元数据清除时目标被占用，已写副本:', alt)
146         return alt
147     return path

```

2.10 B.10 季节费用拆分（season_split.py）

固定/波动电价 × 季节的四类日费用真实计算，验证电价波动风险的季节差异。

```

1  # -*- coding: utf-8 -*-
2  """问题四季节费用拆分 (v2 口径)：固定/波动电价 ×季节 的日费用。
3
4  口径与官方链一致：
5      Q2 列 = 预报式两阶段随机规划 (K_L=15/K_P=7、成对残差 bootstrap、=* 同 Q2 官方)；
6      Q3 列 = 因果滚动随机 MPC (main_q3._rolling v2, =0)。
7  输出 report/season_cost_split.csv (供 R 绘制 A10) + 全年汇总打印。
8  """
9  import os, sys
10 sys.path.insert(0, os.path.dirname(__file__))
11 import numpy as np
12 import pandas as pd
13 import config as C
14 import data, lp
15 import scenarios
16 import main_q2
17 import main_q3
18
19 SORDER = ['春', '夏', '秋', '冬']
20
21
22 def season(d):
23     m = d.month
24     return {12: '冬', 1: '冬', 2: '冬', 3: '春', 4: '春', 5: '春',
25             6: '夏', 7: '夏', 8: '夏', 9: '秋', 10: '秋', 11: '秋'}[m]
26
27
28 def q2_chain(price_day, dates, load, pv, a1, lam, label):
29     """预报式两阶段随机全年链 (同 Q2 官方口径)，price_day(dates[i]) 返回当日 144 电价。
30     返回 {date: 日总费用}。"""
31     S = C.S_NUM
32     KL, KP = 15, 7      # 与 Q2 官方链一致的 K
33     cost = {}
34     E = C.E_INIT
35     for i, d in enumerate(dates):
36         if i == 0:
37             scen_ld = np.tile(a1['load'], (S, 1))
38             scen_pv = np.tile(a1['pv'], (S, 1))
39         else:
40             fc = scenarios.residual_scenarios(load[:i], pv[:i], (KL, KP), S=S,
41                                                seed=main_q2._seed_for_day(d))
42             scen_ld, scen_pv = fc['scen_ld'], fc['scen_pv']
43         pr = price_day(i)
44         two = lp.solve_two_stage(pr, scen_ld, scen_pv, E, lam=lam, beta=C.BETA)
45         settle = lp.solve_day_fixedP(pr, pv[i], load[i], E, two['P'])
46         cost[d] = float(np.sum(pr * two['P']) + np.sum(C.EMERG_MULT * pr * settle['Q']))

```

```

47     E = float(settle['E'][-1])
48     if (i + 1) % 90 == 0:
49         print(f' [{label}] {i+1}/{len(dates)} 日 ({d.date()}) ', flush=True)
50     return cost
51
52
53 def main():
54     a1 = data.load_attach1()
55     fp = a1['price']
56     dates, load, pv = data.load_attach2()
57     dates4, p4 = data.load_attach4()
58     fc3 = data.load_attach3()
59     FC3_HIST = main_q3.build_fc3_hist(fc3, dates)
60     lam = main_q2.LAM_OFFICIAL
61
62     print(f'[Q2 固定电价 预报式两阶段随机 ={lam}]')
63     cost_fix2 = q2_chain(lambda i: fp, dates, load, pv, a1, lam, 'Q2固定')
64     print(f'[Q2 波动电价 ={lam}]')
65     cost_var2 = q2_chain(lambda i: p4[i], dates, load, pv, a1, lam, 'Q2波动')
66
67     cost_fix3, cost_var3 = {}, {}
68     E = C.E_INIT
69     print('[Q3 固定电价 因果滚动随机MPC]')
70     for i, d in enumerate(dates):
71         r = main_q3._rolling(fp, load, pv, dates, fc3, i, E, a1, FC3_HIST)
72         cost_fix3[d] = r['grid_fee'] + r['emerg_fee']
73         E = r['E_end']
74         if (i + 1) % 90 == 0:
75             print(f' [Q3固定] {i+1}/{len(dates)} 日 ({d.date()}) ', flush=True)
76     E = C.E_INIT
77     print('[Q3 波动电价 因果滚动随机MPC]')
78     for i, d in enumerate(dates):
79         r = main_q3._rolling(p4[i], load, pv, dates, fc3, i, E, a1, FC3_HIST)
80         cost_var3[d] = r['grid_fee'] + r['emerg_fee']
81         E = r['E_end']
82         if (i + 1) % 90 == 0:
83             print(f' [Q3波动] {i+1}/{len(dates)} 日 ({d.date()}) ', flush=True)
84
85     rows = []
86     # 季节统计口径与全年费用一致：只用 2.1-12.31 (334 天, 1 月仅作储能预热, 不参与季节均值)
87     do_all = [d for d in dates if d >= pd.Timestamp('2025-02-01')]
88     for s in SORDER:
89         idx = [d for d in do_all if season(d) == s]
90         a = np.array([cost_fix2[d] for d in idx]); b = np.array([cost_var2[d] for d in idx])
91         c = np.array([cost_fix3[d] for d in idx]); e = np.array([cost_var3[d] for d in idx])
92         rows.append(dict(seas=s, n=len(idx),
93                         q2_fix=a.mean() / 1e4, q2_var=b.mean() / 1e4,

```



```

94         q3_fix=c.mean() / 1e4, q3_var=e.mean() / 1e4,
95         up3=(e.mean() - c.mean()) / c.mean() * 100))
96 out = pd.DataFrame(rows)
97 print(out.round(2).to_string(index=False))
98 out.to_csv(os.path.join(C.RES_DIR, 'season_cost_split.csv'), index=False,
99             encoding='utf-8-sig')
100
101 do = [d for d in dates if d >= pd.Timestamp('2025-02-01')]
102 print('\n[全年总 (2.1-12.31, 万元)]')
103 for name, dd in [('Q2固定', cost_fix2), ('Q2波动', cost_var2),
104                 ('Q3固定', cost_fix3), ('Q3波动', cost_var3)]:
105     print(f' {name}: {sum(dd[d] for d in do)/1e4:,.1f}')
106
107 if __name__ == '__main__':
108     main()

```

2.11 B.11 季节特征分析 (seasonal_check.py)

对负荷、光伏与电价做季节分组与统计，支撑问题分析中的季节分型与 ANOVA 检验。

```

1  # -*- coding: utf-8 -*-
2  """季节差异实证：负荷/光伏/优化结果按季节分组统计，评估'按季节分类'创新点可行性。"""
3  import os, sys, json
4  sys.path.insert(0, os.path.dirname(__file__))
5  import numpy as np
6  import pandas as pd
7  import config as C
8  import data, lp
9
10 def season(d):
11     m = d.month
12     return
13     {12:'冬',1:'冬',2:'冬',3:'春',4:'春',5:'春',6:'夏',7:'夏',8:'夏',9:'秋',10:'秋',11:'秋'}[m]
14
15 a1 = data.load_attach1()
16 dates, load, pv = data.load_attach2() # kWh/时段 (365,144)
17 dates4, p4 = data.load_attach4()
18
19 # 1) 季节负荷/光伏总量
20 day_ld = load.sum(axis=1) # kWh/日
21 day_pv = pv.sum(axis=1)
22 seas = [season(d) for d in dates]

```

```

22 df = pd.DataFrame({'seas': seas, 'ld': day_ld, 'pv': day_pv})
23 g = df.groupby('seas').agg(ld_mean=('ld', 'mean'), ld_std=('ld', 'std'),
24                             pv_mean=('pv', 'mean'),
25                             pv_std=('pv', 'std')).reindex(['春', '夏', '秋', '冬'])
26 print('=== 季节负荷/光伏(日 kWh) ===')
27 print(g.round(1).to_string())
28
29 # 2) 季节日内峰谷特征 (平均日负荷曲线峰值/谷值)
30 day_peak = load.max(axis=1); day_trough = load.min(axis=1)
31 df['pk'] = day_peak; df['tr'] = day_trough
32 g2 = df.groupby('seas')[['pk', 'tr']].mean().reindex(['春', '夏', '秋', '冬'])
33 print('\n=== 季节平均日峰/谷负荷(kWh/时段) ===')
34 print(g2.round(1).to_string())
35
36 # 3) 完美信息(Q2口径) 下季节购电费
37 cost_seas = {}
38 E = C.E_INIT
39 for i, d in enumerate(dates):
40     rp = lp.solve_day_plan(a1['price'], pv[i], load[i], E)
41     cost_seas.setdefault(season(d), []).append(float(np.sum(a1['price']*rp['P'])))
42     E = rp['E'][-1]
43 rows = []
44 for s in ['春', '夏', '秋', '冬']:
45     a = np.array(cost_seas[s])
46     rows.append(dict(seas=s, n=len(a), mean=a.mean(), std=a.std(),
47                     mn=a.min(), mx=a.max()))
48 g3 = pd.DataFrame(rows)
49 print('\n=== 季节日购电费分布(元) ===')
50 print(g3.round(0).to_string(index=False))
51
52 # 4) 波动电价季节均值
53 p_mean = p4.mean(axis=1)
54 df['pm'] = p_mean
55 g4 = df.groupby('seas')['pm'].mean().reindex(['春', '夏', '秋', '冬'])
56 print('\n=== 季节平均电价(元/kWh) ===')
57 print(g4.round(4).to_string())
58
59 # 5) 光伏季节日峰值时刻 (负荷峰谷错位程度)
60 # 光伏正午最高、负荷晚峰, 计算 负荷峰谷比
61 ratio = day_peak / np.maximum(day_trough, 1e-6)
62 df['ratio'] = ratio
63 g5 = df.groupby('seas')['ratio'].mean().reindex(['春', '夏', '秋', '冬'])
64 print('\n=== 季节日负荷峰谷比 ===')
65 print(g5.round(2).to_string())
66
67 print('\nANOVA(季节间购电费差异显著性):')
68 from scipy import stats

```

```

68 f, p = stats.f_oneway(cost_seas['春'], cost_seas['夏'], cost_seas['秋'], cost_seas['冬'])
69 print(f'F={f:.1f}, p={p:.2e} ->', '季节差异显著' if p < 0.05 else '差异不显著')

```

2.12 B.12 进阶图表 (chart_advanced.py)

生成箱线图、小提琴图、相关热图、帕累托前沿、雷达图、QQ图、桑基图等分析图（图A1–A8）。

```

1  # -*- coding: utf-8 -*-
2  """子任务3: 高级图表生成（全部基于真实数据 result/附件，服务明确结论）。
3
4  生成 8 张图 → C题/figures_adv/:
5      图A1 季节购电费箱型图      （结论：冬>夏>秋>春, ANOVA p<1e-16 → 支持'按季节分类'创新点）
6      图A2 季节负荷/光伏柱+误差条 （结论：冬光伏最低、夏负荷最高 → 季节分类动因）
7      图A3 波动电价季节分布小提琴图 （结论：冬电价均值最高且右尾重 → 电价季节风险）
8      图A4 - 敏感性热力图      （结论：↑总费↓; 影响小 → 模型对 稳健）
9      图A5 帕累托前沿: 计划费 vs 紧急费 （结论： 提供风险-经济折中前沿）
10     图A6 四问雷达图          （结论：随机/滚动模型以计算复杂度换可靠性与鲁棒性）
11     图A7 光伏预报误差 Q-Q 图   （结论：预报误差近似正态但右尾重 → 随机情景必要性）
12     图A8 典型日能量流向桑基图   （结论：购电结构按季节/时段显著不同）
13
14     """
15     import os, sys
16     sys.path.insert(0, os.path.dirname(__file__))
17     import numpy as np
18     import pandas as pd
19     import config as C
20     import data, lp, scenarios
21     C.setup_plot_style()
22     import matplotlib
23     matplotlib.use('Agg')
24     import matplotlib.pyplot as plt
25     from matplotlib import colors as mcolors
26
27     OUT = os.path.join(C.C_BASE, 'figures_adv')
28     os.makedirs(OUT, exist_ok=True)
29     CB = ['#0072B2', '#D55E00', '#009E73', '#CC79A7', '#F0E442', '#56B4E9', '#E69F00'] # 色盲友好
30
31     def season(d):
32         m = d.month
33         return
34
35         {12: '冬', 1: '冬', 2: '冬', 3: '春', 4: '春', 5: '春', 6: '夏', 7: '夏', 8: '夏', 9: '秋', 10: '秋', 11: '秋'}[m]
36
37     SORDER = ['春', '夏', '秋', '冬']
38
39     a1 = data.load_attach1(); fp = a1['price']

```

```

36 dates, load, pv = data.load_attach2()
37 dates4, p4 = data.load_attach4()
38 fc3 = data.load_attach3()
39
40 # ----- 公共：逐日完美信息购电费 -----
41 day_cost = {}
42 E = C.E_INIT
43 for i, d in enumerate(dates):
44     rp = lp.solve_day_plan(fp, pv[i], load[i], E)
45     day_cost[d] = float(np.sum(fp * rp['P']))
46     E = rp['E'][-1]
47
48 df = pd.DataFrame({'date': dates, 'seas': [season(d) for d in dates],
49                  'cost': [day_cost[d] for d in dates],
50                  'ld': load.sum(axis=1), 'pv': pv.sum(axis=1),
51                  'pm': p4.mean(axis=1)})
52
53 # ===== 图A1 季节购电费箱型图 =====
54 fig, ax = plt.subplots(figsize=(8, 4.6))
55 data_s = [df[df.seas == s]['cost'] / 1e4 for s in SORDER]
56 bp = ax.boxplot(data_s, tick_labels=SORDER, patch_artist=True, widths=0.55,
57                 medianprops=dict(color='k', lw=1.4))
58 for patch, cc in zip(bp['boxes'], CB[:4]):
59     patch.set_facecolor(cc); patch.set_alpha(0.65)
60 ax.set_ylabel('日购电费（万元）'); ax.set_xlabel('季节')
61 ax.set_title('图A1 各季节日购电费分布（完美信息口径）')
62 ax.grid(alpha=0.3, axis='y')
63 fig.tight_layout(); fig.savefig(os.path.join(OUT, 'A1_season_cost_box.png'), dpi=150);
64 plt.close(fig)
65
66 # ===== 图A2 季节负荷/光伏 =====
67 g = df.groupby('seas')[['ld', 'pv']].mean().reindex(SORDER) / 1e4
68 gstd = df.groupby('seas')[['ld', 'pv']].std().reindex(SORDER) / 1e4
69 x = np.arange(4); w = 0.36
70 fig, ax = plt.subplots(figsize=(8, 4.6))
71 ax.bar(x - w/2, g['ld'], w, yerr=gstd['ld'], capsize=3, color=CB[0], alpha=0.85,
72       label='日负荷均值')
73 ax.bar(x + w/2, g['pv'], w, yerr=gstd['pv'], capsize=3, color=CB[1], alpha=0.85,
74       label='日光伏均值')
75 ax.set_xticks(x); ax.set_xticklabels(SORDER)
76 ax.set_ylabel('日能量（万 kWh）'); ax.set_title('图A2 各季节日负荷与光伏出力（均值±标准差）')
77 ax.legend(); ax.grid(alpha=0.3, axis='y')
78 fig.tight_layout(); fig.savefig(os.path.join(OUT, 'A2_season_ld_pv.png'), dpi=150);
79 plt.close(fig)
80
81 # ===== 图A3 季节电价小提琴图 =====
82 fig, ax = plt.subplots(figsize=(8, 4.6))

```

```

79 vp = ax.violinplot([df[df.seas == s]['pm'] for s in SORDER],
80                     positions=[0,1,2,3], widths=0.7, showmedians=True)
81 for b, cc in zip(vp['bodies'], CB[:4]):
82     b.set_facecolor(cc); b.set_alpha(0.6)
83 ax.set_xticks([0,1,2,3]); ax.set_xticklabels(SORDER)
84 ax.set_ylabel('日平均电价 (元/kWh)')
85 ax.set_title('图A3 各季节日平均电价分布 (附件4)')
86 ax.grid(alpha=0.3, axis='y')
87 fig.tight_layout(); fig.savefig(os.path.join(OUT, 'A3_price_violin.png'), dpi=150);
88     plt.close(fig)
89
90 # ===== 图A4/A5: - 敏感性 (两阶段随机+CVaR, 2025-03-20) =====
91 D0 = pd.Timestamp('2025-03-20'); i0 = int(np.where(dates == D0)[0][0])
92 # 与 Q2 官方链同源: E0 取 result2.xlsx 中 3-20 的 0:00 储电量 (* 预热链实际值);
93 # 情景用整日成对残差 bootstrap (K_L=15/K_P=7, seed=2026+日序), 替代旧逐槽独立抽样。
94 import openpyxl
95 _wb = openpyxl.load_workbook(os.path.join(C.RES_DIR, 'result2.xlsx'), read_only=True)
96 _ws = _wb['充放电量']
97 E0 = None
98 for _row in _ws.iter_rows(min_row=2, values_only=True):
99     if isinstance(_row[0], str) and _row[0].startswith('2025-03-20') and _row[4] == '0:00':
100         E0 = float(_row[5]); break
101 _wb.close()
102 assert E0 is not None, 'result2.xlsx 中未找到 2025-03-20 的 0:00 储电量'
103 import main_q2
104 fc = scenarios.residual_scenarios(load[:,i0], pv[:,i0], (15, 7), S=C.S_NUM,
105                                   seed=main_q2._seed_for_day(D0))
106
107 LAMS = [0, 0.2, 0.5, 1, 2, 5]
108 BETS = [0.80, 0.85, 0.90, 0.95, 0.99]
109 grid = np.zeros((len(LAMS), len(BETS)))
110 plan_arr = np.zeros((len(LAMS), len(BETS)))
111 emerg_arr = np.zeros((len(LAMS), len(BETS)))
112 for ai, lam in enumerate(LAMS):
113     for bi, beta in enumerate(BETS):
114         two = lp.solve_two_stage(fp, fc['scen_ld'], fc['scen_pv'], E0, lam=lam, beta=beta)
115         settle = lp.solve_day_fixedP(fp, pv[i0], load[i0], E0, two['P'])
116         pf = float(np.sum(fp * two['P']))
117         ef = float(np.sum(5 * fp * settle['Q']))
118         plan_arr[ai, bi] = pf; emerg_arr[ai, bi] = ef
119         grid[ai, bi] = pf + ef
120         print(f'lam={lam} beta={beta} plan={pf:.0f} emerg={ef:.0f} total={pf+ef:.0f}')
121
122 # 热力图 (总费用)
123 fig, ax = plt.subplots(figsize=(8.2, 5.4))
124 im = ax.imshow(grid, aspect='auto', cmap='YlOrRd')
125 ax.set_xticks(range(len(BETS))); ax.set_xticklabels([str(b) for b in BETS], fontsize=11)

```

```

125 ax.set_yticks(range(len(LAMS))); ax.set_yticklabels([str(l) for l in LAMS], fontsize=11)
126 for r in range(len(LAMS)):
127     for c in range(len(BETS)):
128         ax.text(c, r, f'{grid[r,c]/1e4:.1f}', ha='center', va='center', fontsize=11)
129 ax.set_xlabel('CVaR 置信水平 ', fontsize=12); ax.set_ylabel('风险权重 ', fontsize=12)
130 ax.set_title('图A4 - 对当日总费用（万元）的敏感性（2025-03-20）', fontsize=13)
131 cb = fig.colorbar(im, ax=ax); cb.set_label('总费用（万元）', fontsize=12)
132 fig.tight_layout(); fig.savefig(os.path.join(OUT, 'A4_heatmap_lambda_beta.png'), dpi=150);
    plt.close(fig)
133
134 # 帕累托前沿（=0.9）
135 fig, ax = plt.subplots(figsize=(8.2, 5.4))
136 ax.plot(plan_arr[:, 2]/1e4, emerg_arr[:, 2]/1e4, '-o', color=CB[0], lw=2.0)
137 for ai, lam in enumerate(LAMS):
138     px, py = plan_arr[ai, 2]/1e4, emerg_arr[ai, 2]/1e4
139     dy = (8 if ai % 2 == 0 else -14) if ai < len(LAMS) - 1 else -14
140     ax.annotate(f'={lam}', (px, py), textcoords='offset points', xytext=(10, dy),
141                fontsize=11, bbox=dict(fc='white', ec='none', alpha=0.7, pad=1))
142 ax.set_xlabel('计划购电费（万元）', fontsize=12); ax.set_ylabel('紧急购电费（万元）',
    fontsize=12)
143 ax.set_title('图A5 计划购电费—紧急购电费 帕累托前沿（=0.9, 2025-03-20）', fontsize=13)
144 ax.tick_params(labelsize=11)
145 ax.grid(alpha=0.3)
146 fig.tight_layout(); fig.savefig(os.path.join(OUT, 'A5_pareto.png'), dpi=150); plt.close(fig)
147
148 # ===== 图A6 雷达图 =====
149 fig, ax = plt.subplots(figsize=(7.4, 6.6), subplot_kw=dict(polar=True))
150 dims = ['经济性', '供电可靠性', '鲁棒性', '计算复杂度', '信息利用度']
151 # 半定量打分 0-10: 确定性LP / 两阶段随机 / 滚动MPC / 波动电价重算
152 vals = dict(LP=[8, 4, 2, 9, 3], 随机=[6, 9, 8, 4, 7], MPC=[7, 8, 9, 3, 9], 波动=[5, 8, 8, 3, 9])
153 N = len(dims); ang = np.linspace(0, 2*np.pi, N, endpoint=False).tolist(); ang += ang[:1]
154 for k, (name, v) in enumerate(vals.items()):
155     vv = v + v[:1]
156     ax.plot(ang, vv, label=name, color=CB[k], lw=1.8)
157     ax.fill(ang, vv, color=CB[k], alpha=0.12)
158 ax.set_xticks(ang[:-1]); ax.set_xticklabels(dims, fontsize=12)
159 ax.set_ylim(0, 10); ax.set_yticks([2,4,6,8,10]); ax.set_yticklabels(['2','4','6','8','10'],
    fontsize=10)
160 ax.set_title('图A6 四种模型在五维度上的对比（半定量评分）', fontsize=13)
161 ax.legend(loc='upper right', bbox_to_anchor=(1.32, 1.12), fontsize=11)
162 fig.tight_layout(); fig.savefig(os.path.join(OUT, 'A6_radar.png'), dpi=150); plt.close(fig)
163
164 # ===== 图A7 光伏预报误差 Q-Q =====
165 # 用附件3 0:00 预报 与 附件2 实际 的日总量误差（取全年）
166 day_fc = {}
167 for d in dates:
168     g0, _ = data.pv_forecast_kwh(fc3, d, 0)

```

```

169     day_fc[d] = g0.sum()
170 err = np.array([day_fc[d] - pv[i].sum() for i, d in enumerate(dates)])
171 err = err / np.maximum(np.array([pv[i].sum() for i in range(len(dates))]), 1e-6) * 100 #
    %相对误差
172 err = err[np.isfinite(err)]
173 from scipy import stats
174 fig, ax = plt.subplots(figsize=(7, 5.2))
175 stats.probplot(err, dist='norm', plot=ax)
176 ax.set_title('图A7 光伏日预报相对误差的 Q-Q 图 (附件3 vs 附件2)')
177 ax.set_xlabel('理论分位数'); ax.set_ylabel('样本分位数 (%)')
178 ax.grid(alpha=0.3)
179 fig.tight_layout(); fig.savefig(os.path.join(OUT, 'A7_pv_err_qq.png'), dpi=150); plt.close(fig)
180
181 # ===== 图A8 桑基图: 典型日能量流向 =====
182 try:
183     from matplotlib.sankey import Sankey
184 except Exception:
185     print('sankey unavailable'); A8 = False
186 else:
187     # 用 2025-06-21 完美信息日: 电网购电 + 光伏 + 储能放电 → 负荷 + 充电 + 弃光
188     d = pd.Timestamp('2025-06-21'); i = int(np.where(dates == d)[0][0])
189     E = C.E_INIT
190     for j in range(i):
191         rp = lp.solve_day_plan(fp, pv[j], load[j], E); E = rp['E'][-1]
192     rp = lp.solve_day_plan(fp, pv[i], load[i], E)
193     P = rp['P'].sum(); G = pv[i].sum(); F = (C.ETA*rp['f']).sum() # 放电(输出侧)
194     D = load[i].sum(); Ch = rp['c'].sum(); R = rp['R'].sum()
195     # 标准两列能量流向图 (自绘, 标签外置零遮挡; 替代 matplotlib.sankey 楔形旧图)
196     from matplotlib.path import Path
197     from matplotlib.patches import PathPatch, Rectangle
198     A8C = ['#0072B2', '#D55E00', '#009E73']
199     # 流带分解 (守恒: P + (G-R-Ch) + F = D; Ch、R 记光伏)
200     assert G - R - Ch > 0, '光伏直供为负, 分带方案不适用'
201     bands = [(0, 0, P), (1, 0, G - R - Ch), (1, 1, Ch), (1, 2, R), (2, 0, F)]
202     L_NAMES, R_NAMES = ['电网购电', '光伏出力', '储能放电'], ['负荷', '储能充电', '弃光']
203     L_VALS, R_VALS = [P, G, F], [D, Ch, R]
204     SRC_COLOR = {0: A8C[0], 1: A8C[1], 2: A8C[2]}
205     TP = sum(v for _, _, v in bands)
206     Y0, H, GAP = 0.12, 0.74, 0.035
207     def _stack(vals):
208         hs = [v / TP * (H - GAP * (len(vals) - 1)) for v in vals]
209         out, y = [], 1.0
210         for h in hs:
211             out.append((y, y - h)); y -= h + GAP
212         return out
213     LT, RT = _stack(L_VALS), _stack(R_VALS)
214     X0, X1, W = 0.30, 0.70, 0.028

```

```

215 fig, ax = plt.subplots(figsize=(9.2, 5.0))
216 for k, (t, b) in enumerate(LT):
217     ax.add_patch(Rectangle((X0 - W, b), W, t - b, facecolor=SRC_COLOR[k],
218                             edgecolor='black', linewidth=0.8, alpha=0.95))
219     ax.text(X0 - W - 0.02, (t + b) / 2, f'{L_NAMES[k]}\n{L_VALS[k]:.0f} kWh',
220             ha='right', va='center', fontsize=12)
221 for k, (t, b) in enumerate(RT):
222     ax.add_patch(Rectangle((X1, b), W, t - b,
223                             facecolor=[ '#444444', A8C[2], '#888888' ][k],
224                             edgecolor='black', linewidth=0.8, alpha=0.95))
225     ax.text(X1 + W + 0.02, (t + b) / 2, f'{R_NAMES[k]}\n{R_VALS[k]:.0f} kWh',
226             ha='left', va='center', fontsize=12)
227 rcur = {k: RT[k][0] for k in range(3)}
228 lcur = {k: LT[k][0] for k in range(3)}
229 for (s, t, v) in bands:
230     h = v / TP * (H - GAP * (len(R_VALS) - 1))
231     y0t, y0b = lcur[s], lcur[s] - h
232     y1t, y1b = rcur[t], rcur[t] - h
233     lcur[s] -= h; rcur[t] -= h
234     xm = (X0 + X1) / 2
235     pth = Path([(X0, y0t), (xm, y0t), (xm, y1t), (X1, y1t),
236                 (X1, y1b), (xm, y1b), (xm, y0b), (X0, y0b), (X0, y0t)]),
237               [Path.MOVETO, Path.CURVE4, Path.CURVE4, Path.CURVE4,
238                Path.LINETO, Path.CURVE4, Path.CURVE4, Path.CURVE4, Path.CLOSEPOLY])
239     ax.add_patch(PathPatch(pth, facecolor=SRC_COLOR[s], alpha=0.35, edgecolor='none'))
240 ax.set_xlim(0, 1); ax.set_ylim(0.07, 1.02); ax.axis('off')
241 fig.tight_layout()
242 for out in {OUT, os.path.join(C.C_BASE, 'paper', 'figures_adv')}:
243     os.makedirs(out, exist_ok=True)
244     fig.savefig(os.path.join(out, 'A8_sankey.png'), dpi=200)
245 plt.close(fig)
246 A8 = True
247
248 print('\n全部图表已输出至', OUT)
249 print(sorted(os.listdir(OUT)))

```

2.13 B.13 问题四图表 (chart_4_0.py)

图 A9–A12 已迁移至 R (ggplot2, 见支撑材料 scripts_r/), 本文件仅保留迁移说明。

```

1 # -*- coding: utf-8 -*-
2 """【已迁移】A9–A12 四张图的生成已迁移至 R (ggplot2), 本文件不再生成图表。
3

```


原因：15.0 之后的重绘环节按"清晰直观"标准做工具选型，A9（对比堆叠柱）、A10（季节费用对比）、A11（Q3 滚动对比）、A12（全年费用结构）改用 R 绘制：

- scripts_r/plot_A9_cvar.R → paper/figures_adv/A9_q2_cvar_compare.png
- scripts_r/plot_A10_season.R → paper/figures_adv/A10_season_cost.png
- scripts_r/plot_A11_roll.R → paper/figures_adv/A11_q3_roll_compare.png
- scripts_r/plot_A12_annual.R → paper/figures_adv/A12_annual_cost.png

数据依赖：report/q2_two_stage_comparison.csv、report/season_cost_split.csv、report/q3_rolling_comparison.csv、report/plot_annual.csv。

请勿再运行本文件（历史版本可查 git 记录：12.0优化 分支）。

```

"""
import os, sys

if __name__ == '__main__':
    print('A9-A12 已迁移至 scripts_r/ (R/ggplot2)，请运行对应 R 脚本。')

```

2.14 B.14 论文化取数（extract_paper_day.py）

从官方结果附件读回指定四日的计划/调整/紧急购电与储电数据，供论文表格使用。

```

# -*- coding: utf-8 -*-
"""抓取指定日期4天在 Q2/Q3/Q4-2/Q4-3 官方结果文件中的详细结果，供论文表格。

直接从 report/result*.xlsx 读回（与官方提交附件逐位一致，不重新求解）。
"""
import os, sys, json
sys.path.insert(0, os.path.dirname(__file__))
import numpy as np
import pandas as pd
import openpyxl
import config as C

SLOT = [m // 10 for m in C.SPEC_MINUTES]
SLOT_LAB = ['10:00-10:10', '12:00-12:10', '14:00-14:10', '16:00-16:10', '18:00-18:10',
            '20:00-20:10']
BLK = [(0, '0-4'), (4, '4-8'), (8, '8-12'), (12, '12-16'), (16, '16-20'), (20, '20-24')]

def blk(arr):
    return [round(float(np.sum(arr[h * 6:(h + 4) * 6])), 1) for h, _ in BLK]

def _read_day(path, sheets):
    """读回某日各 sheet 数据。sheets: {'P': 名, 'A': 名或None, 'Q': 名, 'CF': 名}。"""
    wb = openpyxl.load_workbook(path, read_only=True)
    out = {}

```

```

26 for key, sn in sheets.items():
27     if sn is None:
28         continue
29     ws = wb[sn]
30     if key == 'Q':
31         q = np.zeros(C.T_IN_DAY)
32         for row in ws.iter_rows(min_row=2, values_only=True):
33             if row[0] is not None and isinstance(row[0], str) and
34                 row[0].startswith(str(d.date())):
35                 lab = str(row[1])
36                 t = C.INTERVAL_LABELS.index(lab)
37                 q[t] = float(row[2] or 0.0)
38         out['Q'] = q
39     elif key == 'CF':
40         e0 = e24 = None
41         c = f = None
42         for row in ws.iter_rows(min_row=2, values_only=True):
43             if row[0] is not None and isinstance(row[0], str) and
44                 row[0].startswith(str(d.date())):
45                 if row[1] in ('0:00-4:00', '4:00-8:00', '8:00-12:00',
46                             '12:00-16:00', '16:00-20:00', '20:00-24:00'):
47                     pass
48                     if row[4] == '0:00':
49                         e0 = float(row[5])
50                     if row[4] == '24:00':
51                         e24 = float(row[5])
52         out['E0'], out['E24'] = e0, e24
53     else:
54         for row in ws.iter_rows(min_row=2, values_only=True):
55             if row[0] is not None and not isinstance(row[0], str):
56                 dt = pd.to_datetime(row[0])
57                 if dt.date() == d.date():
58                     out[key] = np.array([float(v or 0.0) for v in row[1:1 + C.T_IN_DAY]])
59                     break
60
61 wb.close()
62 return out
63
64 out = {}
65 for dd in C.SPEC_DAYS:
66     d = pd.Timestamp(dd)
67     r2 = _read_day(os.path.join(C.RES_DIR, 'result2.xlsx'),
68                     dict(P='计划购电量', Q='紧急购电量', CF='充放电量'))
69     r3 = _read_day(os.path.join(C.RES_DIR, 'result3.xlsx'),
70                     dict(P='计划购电量', A='调整购电量', Q='紧急购电量', CF='充放电量'))
71     r42 = _read_day(os.path.join(C.RES_DIR, 'result4-2.xlsx'),
72                     dict(P='计划购电量', Q='紧急购电量', CF='充放电量'))

```

```

71 r43 = _read_day(os.path.join(C.RES_DIR, 'result4-3.xlsx'),
72                 dict(P='计划购电量', A='调整购电量', Q='紧急购电量', CF='充放电电量'))
73
74 def qsum(q):
75     return round(float(q.sum()), 1)
76
77 out[dd] = dict(
78     Q2=dict(e0=r2['E0'], e24=r2['E24'], p_slots=[round(float(r2['P'][t]), 1) for t in SLOT],
79            q_sum=qsum(r2['Q']), nq=int((r2['Q'] > 1e-6).sum())),
80     Q4_2=dict(e0=r42['E0'], e24=r42['E24'], p_slots=[round(float(r42['P'][t]), 1) for t in
81            SLOT],
82            q_sum=qsum(r42['Q']), nq=int((r42['Q'] > 1e-6).sum())),
83     Q3=dict(e0=r3['E0'], e24=r3['E24'],
84            p_slots=[round(float(r3['P'][t]), 1) for t in SLOT],
85            a_slots=[round(float(r3['A'][t]), 1) for t in SLOT],
86            q_sum=qsum(r3['Q']), nq=int((r3['Q'] > 1e-6).sum())),
87     Q4_3=dict(e0=r43['E0'], e24=r43['E24'],
88            p_slots=[round(float(r43['P'][t]), 1) for t in SLOT],
89            a_slots=[round(float(r43['A'][t]), 1) for t in SLOT],
90            q_sum=qsum(r43['Q']), nq=int((r43['Q'] > 1e-6).sum())),
91 )
92
93 print(json.dumps(out, ensure_ascii=False, indent=1))
94 print('SLOT_LAB', SLOT_LAB)

```

2.15 B.15 论文化取数 (extract_paper_nums.py)

汇总全年成本、误差等关键数值，供论文定量表述使用。

```

1  # -*- coding: utf-8 -*-
2  """抓取论文所需全部数值 → 供 tex 引用."""
3  import os, sys, json
4  sys.path.insert(0, os.path.dirname(__file__))
5  import numpy as np
6  import pandas as pd
7  import config as C
8  import data, lp
9
10 SPEC_MIN_SLOT = [m // 10 for m in C.SPEC_MINUTES] # 时段索引: 10:00,12:00,...
11 BLK = [(0, '0-4'), (4, '4-8'), (8, '8-12'), (12, '12-16'), (16, '16-20'), (20, '20-24')]
12
13 def blk_sums(arr):
14     return [float(np.sum(arr[h0 * 6:(h0 + 4) * 6])) for h0, _ in BLK]
15
16 def q1():

```

```

17     a1 = data.load_attach1()
18     rp = lp.solve_day_plan(a1['price'], a1['pv'], a1['load'], C.E_INIT, force_end=C.E_INIT)
19     P = rp['P']; c = rp['c']; f = rp['f']
20     return dict(
21         slots=[float(P[t]) for t in SPEC_MIN_SLOT],
22         total=float(P.sum()), cost=float(np.sum(a1['price'] * P)),
23         c_blk=blk_sums(c), f_blk=blk_sums(f), E0=float(rp['E'][0]), E24=float(rp['E'][-1]))
24
25 def day_rows(price, load, pv, dates, out_dates, plan_rec):
26     out = {}
27     for dd in C.SPEC_DAYS:
28         d = pd.Timestamp(dd)
29         i = int(np.where(dates == d)[0][0])
30         pr = price[d] if d in plan_rec else None
31         r = plan_rec[d] if d in plan_rec else None
32         out[dd] = dict(idx=i)
33     return out
34
35 def compute():
36     a1 = data.load_attach1()
37     fp = a1['price']
38     dates, load, pv = data.load_attach2()
39     fc3 = data.load_attach3()
40     dates4, p4 = data.load_attach4()
41
42     # ---- Q1 ----
43     r1 = q1()
44
45     # ---- Q2 (固定电价 完美信息) 全年 + 指定4日块数据 ----
46     rec2 = {}; E = C.E_INIT
47     for i in range(len(dates)):
48         rp = lp.solve_day_plan(fp, pv[i], load[i], E)
49         rec2[dates[i]] = rp
50         E = rp['E'][-1]
51     do2 = [d for d in dates if d >= pd.Timestamp('2025-02-01')]
52     cost2 = float(sum(np.sum(fp * rec2[d]['P']) for d in do2))
53
54     # ---- Q4-2 (波动电价 完美信息) ----
55     rec42 = {}; E = C.E_INIT
56     for i in range(len(dates)):
57         pr = p4[i]
58         rp = lp.solve_day_plan(pr, pv[i], load[i], E)
59         rec42[dates[i]] = rp
60         E = rp['E'][-1]
61     cost42 = float(sum(np.sum(p4[i] * rec42[d]['P']) for i, d in enumerate(do2)))
62
63     return dict(r1=r1, cost2=cost2, cost42=cost42,

```

```

64         dates=[str(d.date()) for d in dates],
65         spec=[dict(d=str(x.date()), i=int(np.where(dates == pd.Timestamp(x))[0][0]),
66                 E2=rec2[x]['E'][-1], E42=rec42[x]['E'][-1]) for x in map(pd.Timestamp,
67                               C.SPEC_DAYS))]
68
69 d = compute()
70 print(json.dumps(d, ensure_ascii=False, indent=1))

```

2.16 B.16 页数校验工具 (page_check.py)

统计编译后 PDF 页数，辅助核验”正文 ≤ 30 页”等篇幅要求。

```

1  # -*- coding: utf-8 -*-
2  """统计 main.pdf 各章节页码分布。"""
3  import PyPDF2, re
4
5  r = PyPDF2.PdfReader(r'E:\desktop\2026建模\C题\paper\main.pdf')
6  print('总页数:', len(r.pages))
7  marks = ['摘要', '摘要', '问题重述', '问题分析', '模型假设', '符号说明',
8          '模型建立与求解', '模型检验与分析', '模型评价与推广', '参考文献',
9          '人工智能工具使用声明', '附录']
10 for i, p in enumerate(r.pages, 1):
11     t = p.extract_text() or ''
12     t1 = re.sub(r'\s+', ' ', t)[:120]
13     hits = [m for m in marks if m in t[:200]]
14     print(f'p{i:>2} | {" ".join(hits):<16} | {t1[:70]}')

```

2.17 B.17 λ 全年权衡扫描 (q2_lambda_sweep.py)

在 $\lambda \in \{0, 0.5, 1, 1.5, 2, 3, 5\}$ 上逐一遍历全年预报式两阶段随机规划链，数据驱动选定官方风险权重 λ^* （图 10 数据来源）。

```

1  # -*- coding: utf-8 -*-
2  """Q2 全年 网格扫描（数据驱动选风险权衡权重 *）。
3
4  对 {0,0.5,1,1.5,2,3,5} 各跑一遍全年预报式两阶段随机规划链
5  （复用 main_q2._run_forecast_chain, K_L=15/K_P=7, S=30, seed=2026+日序），
6  汇总全年计划购电费/紧急购电费/紧急购电量/合计 → report/q2_lambda_sweep.csv。
7  用于在"紧急购电削减 vs 总费用上浮"权衡曲线上选官方 *（论文正文引用）。
8  """
9  import os, sys, time
10 sys.path.insert(0, os.path.dirname(__file__))

```

```

11 import numpy as np
12 import pandas as pd
13 import config as C
14 import data
15 from main_q2 import _run_forecast_chain, cross_validate_K
16
17 LAM_GRID = [0.0, 0.5, 1.0, 1.5, 2.0, 3.0, 5.0]
18
19
20 def main():
21     a1 = data.load_attach1()
22     price = a1['price']
23     dates, load, pv = data.load_attach2()
24     KL, KP, cv = cross_validate_K(dates, load, pv)
25     print(f'[K 交叉验证] K_L={KL}, K_P={KP}')
26     dates_out = [d for d in dates if d >= pd.Timestamp('2025-02-01')]
27     rows = []
28     for lam in LAM_GRID:
29         t0 = time.time()
30         rec = _run_forecast_chain(price, dates, load, pv, KL, KP, S=C.S_NUM,
31                                 lam=lam, verbose=True)
32         plan_fee = sum(float(np.sum(price * rec[d]['P']))) for d in dates_out)
33         emerg_fee = sum(float(np.sum(C.EMERG_MULT * price * rec[d]['Q']))) for d in dates_out)
34         emerg_kwh = sum(float(rec[d]['Q'].sum())) for d in dates_out)
35         total = plan_fee + emerg_fee
36         rows.append(dict(lam=lam, plan_fee=round(plan_fee, 2), emerg_fee=round(emerg_fee, 2),
37                         emerg_kwh=round(emerg_kwh, 2), total=round(total, 2)))
38         print(f'[{lam}] 计划费={plan_fee:.2f} 紧急费={emerg_fee:.2f}'
39               f'({emerg_kwh:.0f} kWh) 合计={total:.2f} 用时 {time.time()-t0:.0f}s', flush=True)
40     df = pd.DataFrame(rows)
41     df.to_csv(os.path.join(C.RES_DIR, 'q2_lambda_sweep.csv'), index=False, encoding='utf-8-sig')
42     print('\n[ 网格扫描完成] 已写 report/q2_lambda_sweep.csv')
43     print(df.to_string(index=False))
44
45
46 if __name__ == '__main__':
47     main()

```

2.18 B.18 结果附件读回与汇总 (export_plot_data.py)

从 result2/3/4-2/4-3.xlsx 读回计划/调整/紧急购电并重算全年费用，输出论文数字锚点 (plot_annual.csv)，保证正文数字与提交附件逐位一致。

```

1 # -*- coding: utf-8 -*-

```

```

2  """从结果附件(result2/3/4-2/4-3.xlsx)读回 P/A/Q, 重算全年费用汇总 → report/plot_annual.csv。
3
4  用途:
5  1) 供 R 脚本绘制 A12 (全年费用结构堆叠柱);
6  2) 验收: 附件数字与求解输出逐位一致 (论文摘要/表格数字以本脚本输出为准)。
7  费用口径:
8       $Q2 = \sum p \cdot P + \sum 5p \cdot Q$ 
9       $Q3 = \sum [p \cdot \min(P, A) + 0.5p(P-A)^+ + 1.5p(A-P)^+ + 5p \cdot Q]$ 
10      $Q4 =$  同上, 价格取附件4 当日波动电价。
11
12     """
13     import os, sys
14     sys.path.insert(0, os.path.dirname(__file__))
15     import numpy as np
16     import pandas as pd
17     import openpyxl
18     import config as C
19     import data
20
21     # 附件5 模板 sheet 名
22     S_PLAN, S_ADJ, S_JINJI = '计划购电量', '调整购电量', '紧急购电量'
23
24     def _read_day_matrix(path, sheet):
25         """读回 计划/调整购电量 sheet → {date: arr144}。"""
26         wb = openpyxl.load_workbook(path, read_only=True)
27         ws = wb[sheet]
28         out = {}
29         for row in ws.iter_rows(min_row=2, values_only=True):
30             if row[0] is None or isinstance(row[0], str):
31                 continue
32             dt = pd.to_datetime(row[0])
33             vals = [float(v) if v is not None else 0.0 for v in row[1:1 + C.T_IN_DAY]]
34             out[dt] = np.array(vals)
35         wb.close()
36         return out
37
38
39     def _read_jinji(path):
40         """读回 紧急购电量 sheet → {date: {t: q}}。"""
41         wb = openpyxl.load_workbook(path, read_only=True)
42         ws = wb[S_JINJI]
43         out = {}
44         for row in ws.iter_rows(min_row=2, values_only=True):
45             if row[0] is None or not isinstance(row[0], str):
46                 continue
47             try:
48                 dt = pd.to_datetime(row[0])

```

```

49     except Exception:
50         continue
51     lab = str(row[1])
52     q = float(row[2]) if row[2] is not None else 0.0
53     t = C.INTERVAL_LABELS.index(lab)
54     out.setdefault(dt, {})[t] = q
55 wb.close()
56 return out
57
58
59 def _q_vec(jinji_day, n=144):
60     q = np.zeros(n)
61     for t, v in jinji_day.items():
62         q[t] = v
63     return q
64
65
66 def _q2_fees(path, price_map):
67     """Q2/Q4-2:  $\Sigma p \cdot P + \Sigma 5p \cdot Q$ . price_map: {date: arr144}."""
68     P = _read_day_matrix(path, S_PLAN)
69     J = _read_jinji(path)
70     plan = emerg = 0.0
71     for d, arr in P.items():
72         pr = price_map[pd.Timestamp(d)]
73         plan += float(np.sum(pr * arr))
74         emerg += float(np.sum(5.0 * pr * _q_vec(J.get(pd.Timestamp(d), {}))))
75     return plan, emerg
76
77
78 def _q3_fees(path, price_map):
79     """Q3/Q4-3:  $\Sigma [p \cdot \min(P, A) + 0.5p(P-A)^+ + 1.5p(A-P)^+ + 5p \cdot Q]$ ."""
80     P = _read_day_matrix(path, S_PLAN)
81     A = _read_day_matrix(path, S_ADJ)
82     J = _read_jinji(path)
83     grid = emerg = 0.0
84     for d, pv in P.items():
85         dt = pd.Timestamp(d)
86         pr = price_map[dt]
87         av = A[dt]
88         u = np.maximum(pv - av, 0.0)
89         w = np.maximum(av - pv, 0.0)
90         grid += float(np.sum(pr * np.minimum(pv, av) + 0.5 * pr * u + 1.5 * pr * w))
91         emerg += float(np.sum(5.0 * pr * _q_vec(J.get(dt, {}))))
92     return grid, emerg
93
94
95 def main():

```



```

96 a1 = data.load_attach1()
97 fp = a1['price']
98 dates4, p4 = data.load_attach4()
99 price_fix = {d: fp for d in dates4}
100 price_var = {d: p4[i] for i, d in enumerate(dates4)}
101
102 g2, e2 = _q2_fees(os.path.join(C.RES_DIR, 'result2.xlsx'), price_fix)
103 g3, e3 = _q3_fees(os.path.join(C.RES_DIR, 'result3.xlsx'), price_fix)
104 g42, e42 = _q2_fees(os.path.join(C.RES_DIR, 'result4-2.xlsx'), price_var)
105 g43, e43 = _q3_fees(os.path.join(C.RES_DIR, 'result4-3.xlsx'), price_var)
106
107 rows = [
108     dict(label='问题二\n(固定电价·随机)', grid_fee=round(g2, 2), emerg_fee=round(e2, 2)),
109     dict(label='问题三\n(固定电价·滚动)', grid_fee=round(g3, 2), emerg_fee=round(e3, 2)),
110     dict(label='问题四(二)\n(波动电价·随机)', grid_fee=round(g42, 2), emerg_fee=round(e42,
111         2)),
112     dict(label='问题四(三)\n(波动电价·滚动)', grid_fee=round(g43, 2), emerg_fee=round(e43,
113         2)),
114 ]
115 df = pd.DataFrame(rows)
116 df.to_csv(os.path.join(C.RES_DIR, 'plot_annual.csv'), index=False, encoding='utf-8-sig')
117 print('[plot_annual.csv 已写]')
118 print(df.to_string(index=False))
119 print(f'\n[附件读回汇总] Q2 合计={g2+e2:,.2f} | Q3 合计={g3+e3:,.2f} | '
120     f'Q4-2 合计={g42+e42:,.2f} | Q4-3 合计={g43+e43:,.2f}')
121 print(f'[对比] Q4-2 较 Q2 上浮 {(g42+e42)/(g2+e2)*100-100:+.2f}% | '
122     f'Q4-3 较 Q3 上浮 {(g43+e43)/(g3+e3)*100-100:+.2f}%')
123 return dict(g2=g2, e2=e2, g3=g3, e3=e3, g42=g42, e42=e42, g43=g43, e43=e43)
124
125 if __name__ == '__main__':
126     main()

```